

CCAO-F — Claude Certified Associate Foundations Study Pack

Contents

Claude Certified Associate – Foundations (CCAO-F)	5
Original Exam-Prep Study Pack (Non-Developer / claude.ai Product Focus)	5
Public availability note	5
Exam mechanics (from published guide)	6
Domain map (user order ↔ official CCAO-F)	6
Recommended study order (by exam weight)	7
Product scope (what this pack covers)	7
Pack maintenance (updated 2026-08-23)	7
How each domain file is structured	8
Suggested 3-week cadence	8
Disclaimer	8
Domain 01 — Claude Platform & Model Foundations	9
Maps to official CCAO-F Product and Model Selection (~12%, ~7 questions)	9
Disclaimer	9
Overview	9
Key map (objectives ↔ exam verbs)	9
Deep notes	10
Decision trees	13
Exam traps	14
Practice Q&A (18) — scenario stems with answers and rationales	14
Quick review checklist (pre-exam)	16
Glossary	16
Domain 01 — Claude Platform & Model Foundations	16
Maps to official CCAO-F Product and Model Selection (~12%, ~7 questions)	16
Disclaimer	17
Overview	17
Key map (objectives ↔ exam verbs)	17
Deep notes	17
Decision trees	21
Exam traps	21
Practice Q&A (18) — scenario stems with answers and rationales	21
Quick review checklist (pre-exam)	23
Glossary	23
Domain 02 — Prompting & Task Execution	24
Maps to official CCAO-F Prompting and Task Execution (~14%, ~8 questions)	24

Disclaimer	24
Overview	24
Key map (objectives ↔ exam verbs)	24
Deep notes	25
Decision trees	28
Exam traps	28
Practice Q&A (18) — scenario stems with answers and rationales	29
Quick review checklist (pre-exam)	30
Glossary	30
Domain 02 — Prompting & Task Execution	31
Maps to official CCAO-F Prompting and Task Execution (~14%, ~8 questions)	31
Disclaimer	31
Overview	31
Key map (objectives ↔ exam verbs)	31
Deep notes	32
Decision trees	34
Exam traps	35
Practice Q&A (18) — scenario stems with answers and rationales	35
Quick review checklist (pre-exam)	36
Glossary	37
Domain 03 — Evaluating & Validating Claude's Output	37
Maps to official CCAO-F Output Evaluation and Validation (~21%, heaviest, ~13 questions)	37
Disclaimer	37
Overview	37
Key map (objectives ↔ exam verbs)	38
Deep notes	38
Decision trees	42
Timed claim-map drills (90-second stems)	42
Exam traps	45
Practice Q&A (25) — answers with brief rationales	45
Quick review checklist (pre-exam)	46
Glossary	47
Worked evaluations (unique examples — once)	47
Validation scripts (once)	47
Personal SOP lab (once)	47
Domain 03 — Evaluating & Validating Claude's Output	48
Maps to official CCAO-F Output Evaluation and Validation (~21%, heaviest, ~13 questions)	48
Disclaimer	48
Overview	48
Key map (objectives ↔ exam verbs)	48
Deep notes	49
Decision trees	52
Timed claim-map drills (90-second stems)	53
Exam traps	55
Practice Q&A (25) — answers with brief rationales	55
Quick review checklist (pre-exam)	57
Glossary	57

Worked evaluations (unique examples — once)	58
Validation scripts (once)	58
Personal SOP lab (once)	58
Domain 04 — Workflow Integration & Solution Design	58
Maps to official CCAO-F Workflow Integration and Solution Design (~16%, ~10 questions)	58
Disclaimer	58
Overview	59
Key map (objectives ↔ exam verbs)	59
Deep notes	59
Decision trees	61
Exam traps	62
Practice Q&A (18) — scenario stems with answers and rationales	62
Quick review checklist (pre-exam)	64
Glossary	64
Domain 04 — Workflow Integration & Solution Design	64
Maps to official CCAO-F Workflow Integration and Solution Design (~16%, ~10 questions)	64
Disclaimer	64
Overview	65
Key map (objectives ↔ exam verbs)	65
Deep notes	65
Decision trees	67
Exam traps	68
Practice Q&A (18) — scenario stems with answers and rationales	68
Quick review checklist (pre-exam)	69
Glossary	70
Domain 05 — Configuration & Knowledge Management	70
Maps to official CCAO-F Configuration and Knowledge Management (~12%, ~7 questions)	70
Disclaimer	70
Overview	70
Key map (objectives ↔ exam verbs)	71
Deep notes	71
Decision trees	74
Exam traps	74
Practice Q&A (18) — scenario stems with answers and rationales	75
Quick review checklist (pre-exam)	76
Glossary	76
Domain 05 — Configuration & Knowledge Management	77
Maps to official CCAO-F Configuration and Knowledge Management (~12%, ~7 questions)	77
Disclaimer	77
Overview	77
Key map (objectives ↔ exam verbs)	77
Deep notes	77
Decision trees	80
Exam traps	80
Practice Q&A (18) — scenario stems with answers and rationales	81
Quick review checklist (pre-exam)	82

Glossary	82
Domain 06 — Governance, Risk & Responsible Use	83
Maps to official CCAO-F Governance, Risk, and Responsible Use (~15%, ~9 questions)	83
Disclaimer	83
Overview	83
Key map (objectives ↔ exam verbs)	83
Deep notes	84
Decision trees	86
Exam traps	86
Practice Q&A (18) — scenario stems with answers and rationales	87
Quick review checklist (pre-exam)	88
Glossary	88
Domain 06 — Governance, Risk & Responsible Use	89
Maps to official CCAO-F Governance, Risk, and Responsible Use (~15%, ~9 questions)	89
Disclaimer	89
Overview	89
Key map (objectives ↔ exam verbs)	89
Deep notes	90
Decision trees	92
Exam traps	92
Practice Q&A (18) — scenario stems with answers and rationales	93
Quick review checklist (pre-exam)	94
Glossary	94
Domain 07 — Troubleshooting & Optimization	95
Maps to official CCAO-F Troubleshooting and Optimization (~10%, ~6 questions)	95
Disclaimer	95
Overview	95
Key map (objectives ↔ exam verbs)	95
Deep notes	96
Decision trees	98
Exam traps	98
Practice Q&A (16) — scenario stems with answers and rationales	99
Quick review checklist (pre-exam)	100
Glossary	100
Domain 07 — Troubleshooting & Optimization	100
Maps to official CCAO-F Troubleshooting and Optimization (~10%, ~6 questions)	100
Disclaimer	101
Overview	101
Key map (objectives ↔ exam verbs)	101
Deep notes	101
Decision trees	103
Exam traps	104
Practice Q&A (16) — scenario stems with answers and rationales	104
Quick review checklist (pre-exam)	105
Glossary	105

CCAO-F Pro Tips — how to take this exam well

106

The numbers that drive strategy

106

The four master instincts (most items reduce to one of these)

106

If the stem says X → think Y

106

Distractor archetypes to expect

107

Freshness watchlist (re-verify the week of your exam)

107

The 48-hour plan

107

Day-of (online proctored)

108

Claude Certified Associate – Foundations (CCAO-F)

Original Exam-Prep Study Pack (Non-Developer / claude.ai Product Focus)

Exam code: CCAO-F **Audience:** Business professionals who use Claude as a productivity tool (ops, marketing, PM, education, communications, knowledge work). **Not** for API/coding tracks. **Pack type:** Original study notes only — synthesized from **public** Anthropic product docs, Help Center, public Claude Academy courses, and the published CCAO-F blueprint. This pack is **not** affiliated with, endorsed by, or sponsored by Anthropic.

Public availability note

Source	Public?	How this pack uses it
Published CCAO-F exam blueprint / exam guide (weights, mechanics, domain statements)	Yes — official Exam Guide v1.0 (July 2026); download from the official Pearson VUE Anthropic program page	Domain weights, exam mechanics, skill statements — cross-checked (2026-08-23) against the public Pearson VUE Anthropic program page
Claude Academy public courses (e.g. Claude 101 , AI Fluency: Framework & Foundations , model-choice tutorials)	Yes — free on academy.claude.com / claude.com/resources	Prompting, 4D fluency, product feature mental models
Anthropic Help Center & product docs (Projects, Artifacts, Research, Connectors, Skills, privacy)	Yes	Feature behavior, plan differences, configuration facts
Official gated prep courses	Not used	These notes are original and grounded in public sources only.

Registration note: Exams are delivered via Pearson VUE (online proctored or test center). Academy **course completion badges** (Claude 101, AI Fluency, etc.) are **not** the same as the paid, proctored CCAO-F credential.

Exam mechanics (from published guide)

Item	Detail
Questions	60 (all scored)
Time	120 minutes (~2 min/question average)
Formats	Single-answer MCQ + multiple-response; each item states how many to select (the “~¼ multiple-response” share is anecdotal, not from the guide)
Pass score	720 on a 100–1000 scaled score
Language	English
Delivery	Pearson VUE — online proctored or test center
Fee	~USD \$99 per attempt (confirm at registration)
Validity	12 months ; free on-time renewal assessment; lapsed → full exam
Retakes	Waiting periods commonly cited: 14 / 30 / 90 days; attempt caps in a rolling year apply
Score report	Pass/fail + scaled score; domain % correct is informational

Tip: Study domain files in order of **weight** (below), not file number. File numbers follow the user’s 7-domain map; official blueprint order differs slightly in numbering but matches the same seven topics.

Domain map (user order ↔ official CCAO-F)

File	Your domain	Official CCAO-F name	Weight	~Q
01-platform-model-foundations.md	Claude Platform & Model Foundations	Product and Model Selection	~12%	~7
02-prompting-task-execution.md	Prompting & Task Execution	Prompting and Task Execution	~14%	~8
03-evaluating-validating-output.md	Evaluating & Validating Claude’s Output	Output Evaluation and Validation	~21% (heaviest)	~13
04-workflow-integration-solution-design.md	Workflow Integration & Solution Design	Workflow Integration and Solution Design	~16%	~10
05-configuration-knowledge-management.md	Configuration & Knowledge Management	Configuration and Knowledge Management	~12%	~7

File	Your domain	Official CCAO-F name	Weight	~Q
06-governance-risk-responsible-use.md	Governance, Risk & Responsible Use	Governance, Risk, and Responsible Use	~15%	~9
07-troubleshooting-optimization.md	Troubleshooting & Optimization	Troubleshooting and Optimization	~10%	~6

Recommended study order (by exam weight)

1. **Domain 03 — Evaluating & Validating Output (21%)** — start here; judgment questions dominate.
2. **Domain 04 — Workflow Integration & Solution Design (16%)** — scenarios, stakeholders, redesign.
3. **Domain 06 — Governance, Risk & Responsible Use (15%)** — policy, privacy, high-risk use.
4. **Domain 02 — Prompting & Task Execution (14%)** — craft, decompose, iterate.
5. **Domain 01 — Platform & Model Foundations (12%)** — features + Haiku/Sonnet/Opus/Fable).
6. **Domain 05 — Configuration & Knowledge Management (12%)** — Projects, instructions, connectors.
7. **Domain 07 — Troubleshooting & Optimization (10%)** — diagnose weak outputs/workflows.

Hands-on (public product): Practice on claude.ai / Claude Desktop with Projects, Artifacts, Skills, Connectors, Research (paid plans for Research / RAG capacity). Pair study with public Academy: Claude 101 + AI Fluency (4D: Delegation, Description, Discernment, Diligence).

Product scope (what this pack covers)

In scope (non-developer): claude.ai chat, Projects (instructions + knowledge + RAG), Artifacts, Skills, Connectors / MCP, Research, Enterprise Search (Team/Enterprise), **Memory & incognito chats, code execution & file creation**, model picker, Styles / preferences, usage limits, responsible use.

Out of scope: API coding, Claude Code as a developer tool track, Bedrock/Vertex implementation, writing exploits or dumps of official course content.

Pack maintenance (updated 2026-08-23)

- **All seven domain files are now deduped, single primary-study copies.** The earlier padding pattern (the same drill blocks pasted 7–15× per file) has been removed from files 01, 02, 04, 05, 06, and 07, matching the clean Domain 03 rewrite. Every file now carries deep notes, decision trees, exam traps, and 15–25 real scenario Q&A with rationales.
- **Coverage gaps closed:** claude.ai **Memory** (named in the official exam guide’s prep list and Domain 3 objectives) now has its main treatment in 05—... with a selection-side section in 01—...;

code execution & file creation (also named in the guide) is covered in 01–... (feature) and 04–.../07–... (workflow and troubleshooting use).

- **PRO-TIPS.md added (2026-08-23):** exam-craft companion — the four master instincts, reflex table, distractor archetypes, 48-hour plan. Read it after the domain files.
- **Plan facts re-verified against support.claude.com (Aug 2026):** Free = max five Projects and one custom connector; Project RAG = paid plans, up to ~10× knowledge capacity; Memory generation on Free/Pro/Max (Enterprise legacy experience) with chat search on Pro+; incognito chats on all plans. Plan caps change — re-verify before exam day.

How each domain file is structured

1. Disclaimer
 2. Overview
 3. Key map (objectives ↔ exam verbs)
 4. Deep notes
 5. Decision trees / tables
 6. Exam traps
 7. Practice Q&A (15–25) with brief rationales
 8. Pre-exam checklist
 9. Glossary
-

Suggested 3-week cadence

Week	Focus	Practice
1	Domains 03 + 04 + skim 06	Daily: evaluate 3 Claude outputs with a written “ship / revise / reject” note
2	Domains 02 + 01 + 05	Build one Project; one Artifact workflow; one Research brief with citations
3	Domain 07 + full review	Timed 60Q simulation; drill multiple-response; re-read traps

Disclaimer

This is an **independent original study aid**. Product UI and model names change; always verify against current support.claude.com and academy.claude.com. Exam facts are from the official Exam Guide v1.0 (July 2026), cross-checked against the public Pearson VUE Anthropic program page; Anthropic may publish updated guide versions.

Domain 01 — Claude Platform & Model Foundations

Maps to official CCAO-F Product and Model Selection (~12%, ~7 questions)

Note: This edition follows the ASD-STE100 Simplified Technical English writing rules. Simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Haiku, Sonnet, Opus, Fable, MCP, Projects, Artifacts, Skills, Connectors, Research, RAG, Memory, prompting, effort) are exceptions and stay as written.

Dedup note (2026-08-23): This is one primary-study copy. Earlier builds repeated the same drill blocks about 7 times. Those duplicates are gone. The content now matches the Domain 03 depth. Check Product facts against support.claude.com (marked below).

Disclaimer

These notes are original CCAO-F study notes. They are for people who are not developers. They use claude.ai (Projects, Artifacts, Skills, Connectors, Research). The notes use public Anthropic Help Center and product docs. They also use public Claude Academy (Claude 101, AI Fluency 4D framework, model-choice tutorials). They use the published CCAO-F blueprint. The notes are independent. They are not affiliated with Anthropic. Check live product details on support.claude.com.

Overview

Select the correct Claude feature and model for the task. Official blueprint verbs follow. Select the right product features (Projects, research mode, chat, artifacts). Differentiate model types (**Haiku, Sonnet, Opus** — the guide names these three). Align model selection with task requirements (cost, speed, quality). Understand and manage context limitations and **memory considerations**. Know when to restart, summarize, or persist. This domain has about 7 questions. This domain is also vocabulary for the whole exam. Domains 04–07 assume you can name the right feature at once.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	Anchors
Feature selection	select, recommend	Chat, Projects, Artifacts, Skills, Connectors, Research, Enterprise Search, code execution/file creation
Model differentiation	differentiate, compare	Haiku / Sonnet / Opus tiers (Fable exists above Opus — see note)
Task alignment	align, escalate	Cost, speed, quality. Effort. Lightest-shippable rule
Context & memory	restart, summarize, persist	Context window, Project knowledge, Memory feature, incognito, handoff briefs

Official objective	Exam verbs	Anchors
Plans	identify constraints	Free / Pro / Max / Team / Enterprise caps that stems hide

Deep notes

1. Three shapes of work

- **Turn-by-turn chat** — you steer every step. This shape is best for iteration, quick answers, and drafts with feedback.
- **Hand-off agentic** — you give a brief. Claude then works in multiple steps (Research. Long agentic tasks). This shape costs more usage. It needs a better brief. You also need to check citations.
- **Reusable systems** — Projects, Skills, Artifacts, and Connectors hold context and procedure once. Then the whole team does not re-explain the same context.

Most exam feature stems ask this: which shape is this job?

2. Feature catalog (what to use, and when)

Chat. Use chat for fast iteration on one thread. Chat is weak for recurring shared context. That work is a Project. Chats stay in your history. With Memory on, they can inform later chats (see §5). Chats are not a team knowledge base.

Projects. A Project is a self-contained workspace. It has **instructions** (standing behavior) plus **knowledge** (reference files). It also has separate chat histories. Projects exist on **all plans**. **Free accounts cap at five projects** (verified support.claude.com, Aug 2026). **Enhanced knowledge via RAG is paid-plan only.** RAG expands knowledge capacity by up to about **10x**. RAG starts when project knowledge approaches context limits (verified Aug 2026). Sharing is a Team/Enterprise capability. Sharing uses “Can view” vs “Can edit.” You share individually, in bulk, or org-wide. Your chats inside a shared Project stay yours. Sharing distributes knowledge and instructions. Sharing does not distribute your transcripts.

Artifacts. An Artifact is a standalone editable deliverable (docs, code, HTML/SVG, interactive apps). The product shows it beside the chat. You can publish or share an Artifact. AI-powered artifacts consume the **viewer’s** usage limits, not the creator’s. Artifacts are not pre-validated. Domain 03 rules still apply.

Skills. A Skill is a packaged task procedure. The product loads it when it is relevant. Contrast the three. Project knowledge is always-available reference. A Skill is a *repeatable procedure*. A Connector is *live data*.

Connectors (MCP). Attach live systems (Drive, Gmail, Slack, custom remote MCP servers). On Team/Enterprise, an admin enables the connector. Then each user authenticates. That is the normal pattern. **Custom connectors exist on every plan. Free accounts are limited to one custom connector** (verified Aug 2026). Domain 06 covers governance.

Research. Research is agentic multi-angle investigation with citations. It needs web search on. It can use connectors. It needs a paid plan. It costs more usage than chat. For a single already-uploaded PDF, quote the file instead.

Enterprise Search. Enterprise Search is org-internal search. It cites connected sources (Team/Enterprise). “What did we decide?” → Enterprise Search. “What is the public landscape?” → Research.

Code execution & file creation. Claude has a **sandboxed computing environment** on claude.ai. Claude writes and runs code. It analyzes data. It builds charts. It generates downloadable files: Excel, PowerPoint, Word, PDF, CSV. **This feature exists on all plans (Free–Enterprise) on web, Desktop, and Mobile** (verified Aug 2026). It replaced the older “analysis tool.” Use it when the stem involves *computation on data* (totals, pivots, trends from a spreadsheet) or a *file deliverable*. Pattern-matching in plain chat is not reliable for arithmetic at scale. This is distinct from Artifacts. An Artifact is the interactive/editable deliverable. Code execution is the compute. It can produce files and verified numbers.

3. Model ladder and task alignment

The exam guide names three tiers. Differentiate them clearly:

Tier	Character	Reach for
Haiku	Fastest, lightest, cheapest	Extraction, classification, short summaries, high-volume simple tasks
Sonnet	Balanced daily default	Most drafts, analysis, everyday knowledge work
Opus	Deepest reasoning	Hard multi-constraint strategy, subtle analysis — <i>after</i> a lighter model or a better brief fails

Note: The current product also offers a tier above Opus (Fable-class). Use it for the longest-horizon, most complex work. The guide’s objective language is Haiku/Sonnet/Opus. Answer in those terms. “Escalate past Opus” appears in stems only as the prestige-bias distractor.

Rules that decide exam answers:

- **Select the lightest model that you can ship.** Evaluate from lighter models upward. Stop at the cheapest output that you would actually ship. The official sample question rewards a “faster, lower-cost model for straightforward, high-volume tasks.”
- **Escalate on demonstrated failure, not prestige.** “Use the model with the most capability to maximize quality” is the standard wrong option. It wastes cost and latency budget.
- **Effort / extended thinking tune depth *inside* a model.** Use lower effort for quick lookups. Use higher effort for genuinely hard problems. Sometimes the right fix is more effort on Sonnet. It is not a jump to Opus.
- **Match the failure to the remedy.** Haiku misses nuance → move up. Sonnet is shallow on adversarial strategy → Opus or a better decomposition. Opus “fails” on cost, latency, or limits → move *down* or restructure. Bad brief → Fix the Description (Domain 02) before you change the model selection.
- **Try-two-models method.** Run the same known task on Haiku, then on Sonnet. Compare substance, not length. If Haiku already shows the claims you care about, stay with the light model.

4. Context limitations (restart / summarize / persist)

The context window is finite. Long threads degrade. You get drift, contradictions, and forgotten constraints.

- **Restart** when contradictions accumulate, roles drift, or quality collapses. A fresh thread plus a tight brief is better than a damaged thread.
- **Summarize** when you need decision continuity without the draft noise. Ask for a handoff brief (decisions, open questions, constraints). Start a new chat from it.
- **Persist** when reuse is predictable. Put standing rules in Project instructions. Put reference files in Project knowledge. Do not use a long first message every Monday.

Instructions compete with task tokens. Keep them short and stable. Put bulky policy in knowledge files with clear names. Ask by filename.

5. Memory (the persist layer built into the product)

claude.ai now has a first-class **Memory** feature (verified support.claude.com, Aug 2026). Details evolve. Re-check before exam day:

- Claude builds **memory entries** from your chats. Entries hold professional context, preferences, and ongoing work. Claude applies them in later conversations. Memory generation exists on **Free, Pro, and Max** (newer experience) and **Enterprise** (legacy synthesis experience). **Chat search** (Claude retrieves your past chats) is Pro and above.
- **Project-scoped:** each Project gets its own separate memory. Context from client A does not enter the memory of client B. This is a direct answer to the exam's "memory considerations" objective.
- **You control it:** view, edit, or delete individual entries, or reset all in Settings. Search and Memory toggle independently.
- Exclude **Incognito chats** (ghost icon, all plans, outside Projects) from history, memory, and search. Use them for one-off sensitive queries.

Exam framing: "restart, summarize, persist" is still the skill. Memory reduces how often you re-explain across chats. Memory is *not* a team knowledge base (it is per-user). Memory is *not* a substitute for Project knowledge. Curated files are better than inferred memories for policy questions. Memory is *not* guaranteed complete. Load-bearing context still belongs in Projects. Configuration and governance of Memory live in Domain 05 (see `05-configuration-knowledge-management.md` §Memory).

6. Plans and hidden constraints

Exam stems hide plan constraints in subordinate clauses. Solve the constrained world in the stem. Do not solve an unconstrained world that the stem does not give:

- Free: five-project cap. One custom connector. No Research.
- Research needs a paid plan **and** web search enabled.
- Enterprise Search needs Team/Enterprise with admin-connected sources.
- Team member $\neq$ admin: the admin enables. Authentication is usually per-user.
- Artifact viewers consume their own limits. Capabilities that you toggle off block artifact/code features.

7. Fluency bridge (4D)

Delegation is this whole domain. You select automation vs augmentation vs agency. You also select which feature and model carries it. **Description** includes the choice of Research vs chat, not just wording. **Discernment** applies hardest to heavy models. Long fluent Opus output still needs Domain 03 evaluation. **Diligence** treats these acts as a deployment: you publish an Artifact, or you enable a connector.

Wrong feature looks like a prompting failure. Wrong model looks like “Claude is dumb.” Domain 07 asks which layer failed. Keep a one-page note per recurring workflow: Feature | Model | Context strategy.

8. Mini-cases (feature and model together)

PMO Friday status: Use a Project with template instructions and a Drive connector. Use Sonnet as the default.

Board strategy interrogation: Use Opus. Domain 03 evaluation is still mandatory.

Support refund question: Answer from Project policy or Enterprise Search. Never use general model knowledge.

Workshop icebreakers: Use chat plus Haiku or Sonnet. Skip Research.

RFP compare: Put the rubric in a Project. Use Research for the public landscape. Use code execution for computed scoring. Output as a table or Artifact.

Quarter-end deck: Upload the data. Use code execution for figures. Sonnet drafts commentary. A human validates.

Anti-patterns follow. Do not mix legal, HR, and marketing in one mega-Project. Do not use Research for a single uploaded PDF. Do not use write-capable connectors for read-only analysis. Do not use the top tier to increase creativity. Do not assume Artifact viewers use the creator’s credentials or limits. Do not use Memory to hold a team’s policy truth.

9. Multiple-response pattern bank

Select-TWO/THREE stems pair one right feature/model move with distractors. Practice this. Remove these wrong options:

- “Use Opus for everything important” (prestige bias)
- “Projects share your chat transcripts” (false mechanics)
- “Research works on Free” / “Research without web search” (plan/capability misses)
- “Memory replaces Project knowledge” (persist-layer confusion)
- “Have chat compute the 10,000-row totals” (wrong tool — code execution)

Correct combinations usually pair a **feature-fit** choice (Project / Research / code execution / Artifact) with a **constraint-respecting** choice (plan cap, auth step, lightest-shippable model).

Decision trees

Feature: Reusable/recurring context → Project. Cited multi-source public investigation → Research. Org-internal “what did we decide” → Enterprise Search. Standalone interactive deliverable → Artifact.

Repeatable procedure → Skill. Live system of record → Connector. Computation on data / file deliverable → code execution. One-off question → chat.

Model: Instant extract/classify → Haiku. Default knowledge work → Sonnet. Deep reasoning after a lighter attempt fails → Opus. Adjust effort before you move to a higher tier.

Context: A thread that drifts → restart. Need continuity, not noise → summarize into a handoff. Predictable reuse → persist in Project. Cross-chat personal context → Memory (check entries. Sensitive one-offs → incognito).

Exam traps

1. Model with the most capability “to be safe” (prestige bias. The official sample question marks it wrong)
 2. The belief that Project sharing shares your chat transcripts (it shares knowledge + instructions)
 3. Research without web search enabled, or Research for one already-uploaded PDF
 4. “Enabled = access”: you forget per-user connector authentication
 5. “Free has no Projects” (false — capped at five) / you ignore the Free one-custom-connector cap
 6. You treat Artifacts as pre-validated because they look like products
 7. Enterprise Search ≠ Research (internal cited lookup vs public agentic investigation)
 8. You ask chat to “calculate” large-table arithmetic instead of code execution
 9. You treat Memory as a shared team knowledge base or as a policy source of truth
 10. You escalate the model when the brief, knowledge, or feature choice is what failed
-

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A marketing lead rebuilds the same brand-voice briefing every Monday. They paste the style guide into a new chat. What should they set up? **A:** A Project with the style guide in knowledge and standing rules in instructions. Recurring context belongs in a persistent workspace. Do not paste it every week.

Q2. An ops team must pull totals and a pivot by region from a 40,000-row CSV. They must deliver an Excel file. What is the best approach? **A:** Upload the file. Have Claude use code execution to compute the numbers and generate the spreadsheet. Chat-only “reading” of large tables risks arithmetic errors. Code execution runs real code. It produces the downloadable file.

Q3. A Free-plan user wants a sixth project for a new client. What is the reality? **A:** Free accounts cap at five projects. Archive or merge one, or upgrade. (Plan caps change. Check at support.claude.com.)

Q4. An associate runs Research and gets nothing useful. Web search is toggled off. What is the first fix? **A:** Enable web search. Research is an agentic web investigation and depends on it. Diagnose capability toggles before you blame the model.

Q5. A consultant wants Claude to remember their reporting format across chats. Client-confidential details must never carry over between two clients. What product behavior helps? **A:** Memory is project-scoped. Each Project keeps separate memory. Keep each client in its own Project. General preferences can live in account-level memory. You can view, edit, or delete entries in Settings.

Q6. A teammate asks a one-off sensitive HR question. They do not want it in history or memory. What do you recommend? **A:** An incognito chat (ghost icon). Exclude It from history, memory, and search. It is available on all plans, outside Projects.

Q7. Select TWO tasks that fit Haiku. **A:** Classify inbound tickets by type. Extract fields from invoices. High-volume, well-specified, low-nuance work is the light-model best fit.

Q8. Sonnet’s competitive analysis feels shallow on a genuinely multi-constraint strategy question after two solid iterations. What is the next move? **A:** Escalate to Opus (or raise effort). A demonstrated failure on hard reasoning is the escalation trigger. Prestige is not the trigger.

Q9. Legal needs “what did we decide about data retention in Q2?” answered from Slack and Drive. What feature? **A:** Enterprise Search — org-internal cited lookup across connected sources. Research is for the public landscape.

Q10. A 300-message thread now contradicts its own constraints. What is the best context move? **A:** Ask for a handoff summary (decisions, constraints, open questions). Restart a fresh chat from it. Summarize, then restart. Do not continue a drifted thread.

Q11. An exec wants an interactive ROI calculator the sales team can use. What feature? **A:** An Artifact (interactive app). Viewers consume their own usage limits. The calculator’s formulas still need Domain 03 validation.

Q12. A Team-plan user shares a Project org-wide. They worry colleagues will read their draft chats. What is true? **A:** Sharing distributes knowledge and instructions with view/edit permissions. Do not give Their own chats in the Project to every member as transcripts. (Check current sharing behavior on support.claude.com.)

Q13. Select TWO reasons the “always use the most capable model” policy fails. **A:** It wastes cost and latency on tasks a light model ships. It substitutes prestige for the evaluate-upward method. (Quality on simple tasks does not improve enough to justify it.)

Q14. The team repeats a 12-step QBR-prep procedure monthly. They teach it again in chat each month. What feature? **A:** A Skill — a packaged, reusable procedure. Project knowledge holds the reference material. The Skill holds the *how*.

Q15. A paid-plan Project’s knowledge has grown near context limits. What keeps it usable? **A:** Enhanced knowledge with RAG (paid plans). Retrieval expands capacity up to about 10×. Help it: name files clearly and ask with a narrow question.

Q16. A user says “Claude forgot everything from yesterday” in a fresh chat on a Pro plan. What is the diagnosis? **A:** Check whether Memory/chat search is enabled. Check whether yesterday’s chat was incognito or in a different Project’s scope. Then decide. Persist standing context in a Project. Do not rely on recall.

Q17. A director asks whether Claude can draft board-pack prose “with zero hallucination risk if we buy Opus.” What is the correct framing? **A:** No tier eliminates hallucination. Model choice trades cost, speed, and quality. Validation (Domain 03) manages accuracy. Opus plus skipped review is worse than Sonnet plus a claim map.

Q18. Select TWO correct pairings of need → surface. **A:** Live Drive data in answers → Connector. Downloadable PowerPoint from analyzed data → code execution/file creation. (Research is not for single uploaded files. Memory is not a team knowledge base.)

Quick review checklist (pre-exam)

- ☐ Define every feature in one sentence, including code execution/file creation and Memory
 - ☐ Free caps: five projects, one custom connector. RAG = paid, ~10×
 - ☐ Model ladder Haiku/Sonnet/Opus + lightest-shippable + escalate-on-failure
 - ☐ Effort tunes depth inside a model
 - ☐ Restart / summarize / persist — and where Memory does and does not help
 - ☐ Incognito = out of history, memory, search
 - ☐ Research needs paid plan + web search. Enterprise Search needs admin-connected sources
 - ☐ Connector pattern: admin enables, user authenticates
-

Glossary

Term	Meaning
Project	A workspace with instructions, knowledge, and its own chats
RAG (in Projects)	Paid-plan retrieval that expands knowledge capacity (~10×) near context limits
Artifact	A standalone editable/interactive deliverable beside the chat
Skill	A packaged reusable task procedure
Connector (MCP)	A live integration to an external system. Admin-enabled, user-authenticated
Research	Paid agentic multi-source investigation with citations
Enterprise Search	Org-internal cited search (Team/Enterprise)
Code execution / file creation	A sandboxed environment where Claude runs code and generates files (all plans)
Memory	Per-user, project-scoped remembered context. View/edit/delete in Settings
Incognito chat	Ghost-icon chat excluded from history, memory, and search
Effort	Depth control inside a model tier
Context window	Finite working memory of a conversation

Domain 01 — Claude Platform & Model Foundations

Maps to official CCAO-F Product and Model Selection (~12%, ~7 questions)

Dedup note (2026-08-23): Rewritten as a single primary-study copy. Earlier builds repeated the same drill blocks ~7×; duplicates removed, content deepened to the Domain 03 standard, and product facts re-verified against support.claude.com (marked below).

Disclaimer

Original CCAO-F study notes for non-developers using claude.ai (Projects, Artifacts, Skills, Connectors, Research). Grounded in public Anthropic Help Center & product docs, public Claude Academy (Claude 101, AI Fluency 4D framework, model-choice tutorials), and the published CCAO-F blueprint. Independent; not affiliated with Anthropic. Verify live product details on support.claude.com.

Overview

Pick the right Claude **feature** and **model** for the job. Official blueprint verbs: select appropriate product features (Projects, research mode, chat, artifacts); differentiate model types (**Haiku, Sonnet, Opus** — the guide names these three); align model selection with task requirements (cost, speed, quality); understand and manage context limitations and **memory considerations** (when to restart, summarize, or persist). ~7 questions, but this domain is vocabulary for the whole exam: Domains 04–07 assume you can name the right feature instantly.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	Anchors
Feature selection	select, recommend	Chat, Projects, Artifacts, Skills, Connectors, Research, Enterprise Search, code execution/file creation
Model differentiation	differentiate, compare	Haiku / Sonnet / Opus tiers (Fable exists above Opus — see note)
Task alignment	align, escalate	Cost, speed, quality; effort; lightest-shippable rule
Context & memory	restart, summarize, persist	Context window, Project knowledge, Memory feature, incognito, handoff briefs
Plans	identify constraints	Free / Pro / Max / Team / Enterprise caps hidden in stems

Deep notes

1. Three shapes of work

- **Turn-by-turn chat** — you steer every step. Best for iteration, quick answers, drafting with feedback.
- **Hand-off agentic** — you brief, Claude works multi-step (Research; long agentic tasks). Costs more usage; needs a better brief and citation checking.
- **Reusable systems** — Projects, Skills, Artifacts, Connectors encode context and procedure once so the whole team stops re-explaining.

Most exam feature stems are really asking: which shape is this job?

2. Feature catalog (what to reach for, and when)

Chat. Fast iteration on one thread. Weak for recurring shared context — that’s a Project. Chats persist in your history (and, with Memory on, can inform later chats — see §5); they are not a team knowledge base.

Projects. Self-contained workspaces with **instructions** (standing behavior) + **knowledge** (reference files) and separate chat histories. Available on **all plans**; **Free accounts are capped at five projects** (verified support.claude.com, Aug 2026). **Enhanced knowledge via RAG is paid-plan only and expands knowledge capacity by up to ~10x**, activating when project knowledge approaches context limits (verified Aug 2026). Sharing is a Team/Enterprise capability: “Can view” vs “Can edit,” shared individually, in bulk, or org-wide. Your chats inside a shared Project stay yours — sharing distributes knowledge and instructions, not your transcripts.

Artifacts. Standalone editable deliverables (docs, code, HTML/SVG, interactive apps) rendered beside the chat. Can be published/shared; AI-powered artifacts consume the **viewer’s** usage limits, not the creator’s. Not pre-validated — Domain 03 rules still apply.

Skills. Packaged task procedures loaded when relevant. Contrast: Project knowledge is always-available reference; a Skill is a *repeatable procedure*; a Connector is *live data*.

Connectors (MCP). Attach live systems (Drive, Gmail, Slack, custom remote MCP servers). Admin enablement plus per-user authentication is the normal pattern on Team/Enterprise. **Custom connectors are available on every plan, but Free accounts are limited to one custom connector** (verified Aug 2026). Governance lives in Domain 06.

Research. Agentic multi-angle investigation with citations; needs web search on; can use connectors; paid plans; burns usage faster than chat. For a single already-uploaded PDF, quote the file instead.

Enterprise Search. Org-internal search with citations back to connected sources (Team/Enterprise). “What did we decide?” → Enterprise Search. “What’s the public landscape?” → Research.

Code execution & file creation. Claude has a **sandboxed computing environment** on claude.ai: it writes and runs code to analyze data, build charts, and generate downloadable files — Excel, PowerPoint, Word, PDF, CSV. **Available on all plans (Free–Enterprise) on web, Desktop, and Mobile** (verified Aug 2026); it replaced the older “analysis tool.” Reach for it when the stem involves *computation on data* (totals, pivots, trends from a spreadsheet) or a *file deliverable* — pattern-matching in plain chat is unreliable for arithmetic at scale. Distinct from Artifacts: an Artifact is the interactive/editable deliverable; code execution is the compute that can produce files and verified numbers.

3. Model ladder and task alignment

The exam guide names three tiers — differentiate them cleanly:

Tier	Character	Reach for
Haiku	Fastest, lightest, cheapest	Extraction, classification, short summaries, high-volume simple tasks
Sonnet	Balanced daily default	Most drafting, analysis, everyday knowledge work

Tier	Character	Reach for
Opus	Deepest reasoning	Hard multi-constraint strategy, subtle analysis — <i>after</i> a lighter model or better brief has failed

Note: the current product also offers a tier above Opus (Fable-class) for the longest-horizon, most complex work. The guide’s objective language is Haiku/Sonnet/Opus — answer in those terms; “escalate past Opus” appears in stems only as the prestige-bias distractor.

Rules that decide exam answers:

- **Lightest shippable model wins.** Evaluate from lighter models upward; stop at the cheapest output you would actually ship. The official sample question rewards “faster, lower-cost model for straightforward, high-volume tasks.”
- **Escalate on demonstrated failure, not prestige.** “Use the biggest model to maximize quality” is the canonical wrong option — it wastes cost and latency budget.
- **Effort / extended thinking tune depth *inside* a model.** Lower effort for quick lookups; higher for genuinely hard problems. Sometimes the right fix is more effort on Sonnet, not a jump to Opus.
- **Match the failure to the remedy.** Haiku missing nuance → move up. Sonnet shallow on adversarial strategy → Opus or a better decomposition. Opus “failing” on cost/latency/limits → move *down* or restructure. Bad brief → fix Description (Domain 02) before touching the model picker.
- **Try-two-models method.** Run the same known task on Haiku then Sonnet; compare substance, not length. If Haiku already surfaces the claims you care about, stay light.

4. Context limitations (restart / summarize / persist)

The context window is finite; long threads degrade — drift, contradictions, forgotten constraints.

- **Restart** when contradictions accumulate, roles drift, or quality collapses. Fresh thread + a tight brief beats nursing a polluted one.
- **Summarize** when you need decision continuity without the draft noise: ask for a handoff brief (decisions, open questions, constraints) and start a new chat from it.
- **Persist** when reuse is predictable: put standing rules in Project instructions and reference files in Project knowledge — not in a heroic first message every Monday.

Instructions compete with task tokens: keep them short and stable; put bulky policy in knowledge files with clear names, and ask by filename.

5. Memory (the persist layer built into the product)

claude.ai now has a first-class **Memory** feature (verified support.claude.com, Aug 2026 — details evolve; re-check before exam day):

- Claude builds **memory entries** from your chats (professional context, preferences, ongoing work) and applies them in later conversations. Memory generation is available on **Free, Pro, and Max** (newer experience) and **Enterprise** (legacy synthesis experience); **chat search** (retrieving past chats) is Pro and above.

- **Project-scoped:** each Project gets its own separate memory, so client A's context doesn't bleed into client B's — a direct answer to the exam's "memory considerations" objective.
- **You control it:** view, edit, delete individual entries, or reset all in Settings; search and memory toggle independently.
- **Incognito chats** (ghost icon, all plans, outside Projects) are excluded from history, memory, and search — use them for one-off sensitive queries.

Exam framing: "restart, summarize, persist" is still the skill. Memory reduces re-explaining across chats, but it is *not* a team knowledge base (it's per-user), *not* a substitute for Project knowledge (curated files beat inferred memories for policy questions), and *not* guaranteed complete — load-bearing context still belongs in Projects. Configuration and governance of Memory live in Domain 05 (see 05-configuration-knowledge-management.md §Memory).

6. Plans and hidden constraints

Exam stems hide plan constraints in subordinate clauses. Solve the constrained world in the stem, not the ideal world in your head:

- Free: five-project cap; one custom connector; no Research.
- Research needs a paid plan **and** web search enabled.
- Enterprise Search needs Team/Enterprise with admin-connected sources.
- Team member ≠ admin: enablement is admin's; authentication is usually per-user.
- Artifact viewers burn their own limits; capabilities toggled off block artifact/code features.

7. Fluency bridge (4D)

Delegation is this whole domain: choosing automation vs augmentation vs agency, and which feature/model carries it. **Description** includes choosing Research vs chat, not just wording. **Discernment** applies hardest to heavy models — long fluent Opus output still needs Domain 03 evaluation. **Diligence** treats publishing an Artifact or enabling a connector as a deployment act.

Wrong feature masquerades as a prompting failure; wrong model masquerades as "Claude is dumb." Domain 07 will ask which layer broke — keep a one-pager per recurring workflow: Feature | Model | Context strategy.

8. Mini-cases (feature + model in one breath)

PMO Friday status: Project with template instructions + Drive connector; Sonnet default. **Board strategy interrogation:** Opus (with Domain 03 evaluation still mandatory). **Support refund question:** answer from Project policy or Enterprise Search — never general model lore. **Workshop icebreakers:** chat + Haiku/Sonnet; skip Research. **RFP compare:** rubric in Project, public landscape via Research, computed scoring via code execution, output as table/Artifact. **Quarter-end deck:** data upload → code execution for figures → Sonnet drafts commentary → human validates.

Anti-patterns: one mega-Project mixing legal + HR + marketing; Research for a single uploaded PDF; write-capable connectors for read-only analysis; the top tier as a creativity dial; assuming an Artifact's viewers use the creator's credentials or limits; relying on Memory to carry a team's policy truth.

9. Multiple-response pattern bank

Select-TWO/THREE stems here pair one right feature/model move with distractors. Rehearse eliminating: “use Opus for everything important” (prestige bias); “Projects share your chat transcripts” (false mechanics); “Research works on Free” / “Research without web search” (plan/capability misses); “Memory replaces Project knowledge” (persist-layer confusion); “have chat compute the 10,000-row totals” (wrong tool — code execution). Correct combinations usually pair a **feature-fit** choice (Project / Research / code execution / Artifact) with a **constraint-respecting** choice (plan cap, auth step, lightest-shippable model).

Decision trees

Feature: Reusable/recurring context → Project. Cited multi-source public investigation → Research. Org-internal “what did we decide” → Enterprise Search. Standalone interactive deliverable → Artifact. Repeatable procedure → Skill. Live system of record → Connector. Computation on data / file deliverable → code execution. One-off question → chat.

Model: Instant extract/classify → Haiku. Default knowledge work → Sonnet. Deep reasoning after a lighter attempt failed → Opus. Tune effort before jumping tiers.

Context: Drifting thread → restart. Need continuity, not noise → summarize into a handoff. Predictable reuse → persist in Project. Cross-chat personal context → Memory (verify entries; sensitive one-offs → incognito).

Exam traps

1. Biggest model “to be safe” (prestige bias — the official sample question punishes it)
 2. Believing Project sharing shares your chat transcripts (it shares knowledge + instructions)
 3. Research without web search enabled, or Research for one already-uploaded PDF
 4. “Enabled = access”: forgetting per-user connector authentication
 5. “Free has no Projects” (false — capped at five) / ignoring the Free one-custom-connector cap
 6. Treating Artifacts as pre-validated because they look like products
 7. Enterprise Search ≠ Research (internal cited lookup vs public agentic investigation)
 8. Asking chat to “calculate” large-table arithmetic instead of using code execution
 9. Treating Memory as a shared team brain or as a policy source of truth
 10. Escalating the model when the brief, knowledge, or feature choice is what’s broken
-

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A marketing lead rebuilds the same brand-voice briefing every Monday by pasting the style guide into a new chat. What should they set up? **A:** A Project with the style guide in knowledge and standing rules in instructions. Recurring context belongs in a persistent workspace, not weekly heroic pasting.

Q2. An ops team must pull totals and a pivot by region from a 40,000-row CSV and deliver an Excel file. Best approach? **A:** Upload the file and have Claude use code execution to compute and generate

the spreadsheet. Chat-only “reading” of large tables risks arithmetic errors; code execution runs real code and produces the downloadable file.

Q3. A Free-plan user wants a sixth project for a new client. What’s the reality? **A:** Free accounts cap at five projects — archive or merge one, or upgrade. (Plan caps change; verify at support.claude.com.)

Q4. An associate runs Research and gets nothing useful; web search is toggled off. First fix? **A:** Enable web search — Research is an agentic web investigation and depends on it. Diagnose capability toggles before blaming the model.

Q5. A consultant wants Claude to remember their reporting format across chats, but client-confidential details must never carry over between two clients. What product behavior helps? **A:** Memory is project-scoped — each Project keeps separate memory. Keep each client in its own Project; general preferences can live in account-level memory, and entries can be viewed/edited/deleted in Settings.

Q6. A teammate asks a one-off sensitive HR question and doesn’t want it in history or memory. What do you recommend? **A:** An incognito chat (ghost icon) — excluded from history, memory, and search; available on all plans, outside Projects.

Q7. Select TWO tasks that fit Haiku. **A:** Classifying inbound tickets by type; extracting fields from invoices. High-volume, well-specified, low-nuance work is the light-model sweet spot.

Q8. Sonnet’s competitive analysis feels shallow on a genuinely multi-constraint strategy question after two solid iterations. Next move? **A:** Escalate to Opus (or raise effort) — a demonstrated failure on hard reasoning is the escalation trigger, not prestige.

Q9. Legal needs “what did we decide about data retention in Q2?” answered from Slack and Drive. Feature? **A:** Enterprise Search — org-internal cited lookup across connected sources. Research is for the public landscape.

Q10. A 300-message thread now contradicts its own constraints. Best context move? **A:** Ask for a handoff summary (decisions, constraints, open questions) and restart a fresh chat from it. Summarize-then-restart beats pushing a drifted thread.

Q11. An exec wants an interactive ROI calculator the sales team can use. Feature? **A:** An Artifact (interactive app). Note viewers consume their own usage limits, and the calculator’s formulas still need Domain 03 validation.

Q12. A Team-plan user shares a Project org-wide and worries colleagues will read their draft chats. What’s true? **A:** Sharing distributes knowledge and instructions with view/edit permissions; their own chats in the Project are not handed to every member as transcripts. (Verify current sharing behavior on support.claude.com.)

Q13. Select TWO reasons the “always use the most capable model” policy fails. **A:** It wastes cost/latency on tasks a light model ships; it substitutes prestige for the evaluate-upward method. (Quality on simple tasks does not improve enough to justify it.)

Q14. The team repeats a 12-step QBR-prep procedure monthly and keeps re-teaching it in chat. Feature? **A:** A Skill — packaged, reusable procedure. Project knowledge holds the reference material; the Skill holds the *how*.

Q15. A paid-plan Project’s knowledge has grown near context limits. What keeps it usable? **A:** Enhanced knowledge with RAG (paid plans) — retrieval expands capacity up to ~10×; help it by naming files clearly and asking narrowly.

Q16. A user says “Claude forgot everything from yesterday” in a fresh chat on a Pro plan. Diagnosis?
A: Check whether Memory/chat search is enabled and whether yesterday’s chat was incognito or in a different Project’s scope. Then decide: persist standing context in a Project rather than relying on recall.

Q17. A director asks whether Claude can draft board-pack prose “with zero hallucination risk if we buy Opus.” Correct framing? **A:** No tier eliminates hallucination — model choice trades cost/speed/quality, while validation (Domain 03) manages accuracy. Opus + skipped review is worse than Sonnet + claim map.

Q18. Select TWO correct pairings of need → surface. **A:** Live Drive data in answers → Connector; downloadable PowerPoint from analyzed data → code execution/file creation. (Research is not for single uploaded files; Memory is not a team knowledge base.)

Quick review checklist (pre-exam)

- ☐ Define every feature in one sentence, including code execution/file creation and Memory
 - ☐ Free caps: five projects, one custom connector; RAG = paid, ~10×
 - ☐ Model ladder Haiku/Sonnet/Opus + lightest-shippable + escalate-on-failure
 - ☐ Effort tunes depth inside a model
 - ☐ Restart / summarize / persist — and where Memory does and doesn’t help
 - ☐ Incognito = out of history, memory, search
 - ☐ Research needs paid plan + web search; Enterprise Search needs admin-connected sources
 - ☐ Connector pattern: admin enables, user authenticates
-

Glossary

Term	Meaning
Project	Workspace with instructions + knowledge + its own chats
RAG (in Projects)	Paid-plan retrieval that expands knowledge capacity (~10×) near context limits
Artifact	Standalone editable/interactive deliverable beside the chat
Skill	Packaged reusable task procedure
Connector (MCP)	Live integration to an external system; admin-enabled, user-authed
Research	Paid agentic multi-source investigation with citations
Enterprise Search	Org-internal cited search (Team/Enterprise)
Code execution / file creation	Sandboxed environment where Claude runs code and generates files (all plans)
Memory	Per-user, project-scoped remembered context; view/edit/delete in Settings
Incognito chat	Ghost-icon chat excluded from history, memory, and search

Term	Meaning
Effort	Depth control inside a model tier
Context window	Finite working memory of a conversation

Domain 02 — Prompting & Task Execution

Maps to official CCAO-F Prompting and Task Execution (~14%, ~8 questions)

Note: This edition follows the ASD-STE100 Simplified Technical English writing rules. Simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Projects, Artifacts, Skills, Connectors, Research, prompting, prompt, schema, eval) are exceptions and stay as written.

Dedup note (2026-08-23): This is one primary-study copy. Earlier builds repeated the same drill blocks about 10 times. Those duplicates are gone. The content now matches the Domain 03 depth.

Disclaimer

These notes are original CCAO-F study notes. They are for people who are not developers. They use claude.ai (Projects, Artifacts, Skills, Connectors, Research). The notes use public Anthropic Help Center and product docs. They also use public Claude Academy (Claude 101, AI Fluency 4D framework) and the published CCAO-F blueprint. The notes are independent. They are not affiliated with Anthropic. Check live product details on support.claude.com.

Overview

Prompting on CCAO-F equals professional briefing. Official blueprint verbs: **create** effective prompts for business and technical tasks. **Apply task decomposition** to structure complex requests. **Iterate** prompts to improve quality. **Adapt prompting strategies by task type**. The guide names *analysis*, *research*, *drafting*, *brainstorming*. The AI Fluency **Description** triad is Product, Process, Performance. This triad is the core structure. Product is what the deliverable is. Process is how you get there. Performance is what “good” looks like.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Effective prompts	create, structure	Prompt anatomy. The Description triad
Task decomposition	apply, sequence	Phased prompts with checkpoints
Iteration	iterate, refine	Named-defect feedback. Change one variable at a time

Official objective	Exam verbs	What you practice
Task-type adaptation	adapt, select	Analysis / research / drafting / brainstorming playbooks

Deep notes

1. Prompt anatomy

A complete brief carries these parts: **role, goal, inputs, constraints, process, format, verification.**

- **Role:** who Claude acts as.
- **Goal:** the deliverable plus the decision it supports.
- **Inputs:** files by name, pasted data, connector sources.
- **Constraints:** length, tone, banned claims, audience.
- **Process:** ordered steps and checkpoints.
- **Format:** schema, sections, table.
- **Verification:** what you check before you answer.

You rarely need all seven parts. Exam-best prompts have no *missing load-bearing* part. They also have no internal contradiction.

Description triad: *Product* Description names the deliverable. It also names the audience decision that the deliverable supports. *Process* Description sequences tools and checkpoints. *Performance* Description sets acceptance tests, examples, and counterexamples. A missing part creates a predictable failure. No Product Description gives empty, unusable output. No Process skips steps. No Performance gives stylish wrong answers.

Stakeholder translation is the associate skill. “Make it pop” needs voice samples. “Keep it legal” needs disclaimer files and banned claims. “Board ready” needs a Decision/Risks/Ask structure. “Quick thoughts” must have a speculative label. Stems use stakeholder slang. Translate the slang before you select an answer.

Contradictions must have an explicit resolution. Example: “explain like a beginner but keep undefined internal acronyms.” Unresolved contradictions give unstable outputs. People then blame the model.

Examples are better than adjectives. Two or three gold examples (and one counterexample) are better than paragraphs of style description. Few-shot is the strongest control a non-developer has.

Ambiguity: prefer *clarify* over *invent*. If you proceed, require an explicit ASSUMPTIONS block. Wrong guesses then stay visible. You can fix them at low cost.

2. Task decomposition

Break multi-deliverable or high-stakes work into phases. Put human checkpoints between phases:

- **Outline** → **fill:** Approve the structure before the prose.
- **Extract** → **analyze:** Separate facts from judgment. Then you can trace errors.
- **Options** → **decide:** Make alternatives without a commitment. Then select with criteria.
- **Draft** → **red-team:** Produce the draft. Then attack it as an adversary (this bridges Domain 03).

Worked patterns:

- **Ops postmortem:** Timeline facts. Then tag contributing factors as Evidence vs Hypothesis. Then write actions only with named owners.
- **Marketing email:** Facts plus legal must-includes. Then subjects plus body. Then a compliance pass against banned claims.
- **HR FAQ:** Answer only from knowledge. Quote the section. Write “Not found” if the answer is absent.
- **Offsite agenda:** Brainstorm. Then cluster. Then select. Then make a timed agenda. Then do a risk check.
- **Data cleanup:** Define a schema. Validate a 20-row sample via code execution. Then scale.

Decomposition is also governance. It inserts human gates before irreversible or external steps. Exams test this crossover in Domains 04 and 06.

Decompose when you have:

- multiple deliverables
- high stakes
- mixed task types (never brainstorm and analyze in the same prompt)
- a need for intermediate checkpoints

Single-shot is fine when the task is small, clear, and low-stakes. Decomposition of trivia wastes time and usage.

3. Iteration craft

Strong iteration **names the defect and the constraint to add**. Example: “the summary invented a Q3 figure — use only numbers from Q3-report.pdf, mark anything else UNKNOWN.” This is better than “make it better.” Change **one major variable at a time** when you debug, so you learn the cause. Prefer narrow edits (“fix only the table”) over a full regeneration. When a prompt fails twice with tighter wording, the problem is usually inputs, feature choice, or task framing. The problem is not word choice (Domain 07’s ladder).

Keep a micro-prompt library:

- assumptions list
- FACTS vs INFERENCES
- only the table
- NULL if missing
- strongest counterargument
- meaning-preserving diff
- outline only
- claim-to-source map
- three options without picking
- compress without changing numbers

4. Task-type playbooks (the named blueprint objective)

Task type	Strategy	Signature moves
Analysis	Criteria first. Separate evidence from inference.	Give a rubric. Demand FACTS vs INFERENCES sections. Require counterarguments and unknowns.
Research	Scope + sources + citations	Define the question, timeframe, and source rules. Require citation-support (Domain 03). Use the Research feature for a multi-source public landscape.
Drafting	Must-include facts + audience + format	Supply approved facts. Do not let Claude source them. Add voice samples, banned phrases, and length caps.
Brainstorming	Diverge, then converge — in separate steps	Ask for quantity and variety without judgment first. Rank with criteria in a second step. Label the output speculative.

Mode mixing is the common error. A prompt that asks for “creative ideas but strictly accurate to policy” gets neither result. Split the phases.

5. Prompting as a system (team scale)

Stable rules move out of prompts. Recurring instructions go to Project instructions. Reference detail goes to knowledge files (point to them by name). Rich procedures go to Skills. Shared Projects need priority rules for conflicts (example: legal > brand). Store gold examples and counterexamples in knowledge. Teach teammates to paste the request. Do not paste vague urgency. Vague urgency without specs wastes usage limits. A prompt that you repeat weekly is configuration, not typing (Domain 05).

6. Timed heuristics for prompt-comparison items

When the exam shows four candidate prompts, score each on Goal, Inputs, Constraints, Process, Format, Verification. The highest complete, non-contradictory score wins. Prefer clarify over invent. Prefer phased approvals over one mega-prompt for consequential transformations. API-parameter options (temperature, fine-tuning) are distractors on this product-focused exam. Before you “escalate the model,” ask this: could a careful intern succeed from this brief? If no, fix the Description first.

7. Worked briefs (before → after)

Weak: “Summarize this customer call.” → **Strong:** “From call-2026-08-12.txt, produce a one-page summary for the account exec deciding whether to escalate. 5 bullet facts with timestamps, customer sentiment with a supporting quote, commitments made by either side (quote exactly. UNKNOWN if unclear), and recommended next step with rationale. Do not infer facts not in the transcript.”

Weak: “Write a LinkedIn post about our launch.” → **Strong:** “Draft a LinkedIn post announcing the launch for prospective mid-market buyers. Must include: the two approved claims in launch-

facts.md (verbatim numbers), a customer-pain opening, ≤150 words, no superlatives from banned-phrases.md. Match the voice of the two examples in knowledge. Give two variants: safe and bold.”

Weak: “Is this vendor contract okay?” → **Strong:** “Phase 1: extract every clause in vendor-msa.pdf touching liability, termination, data handling, and auto-renewal — quote each. Phase 2 (after my confirmation): assess each extracted clause against the checklist in contract-redlines.md, tag OK / FLAG / ESCALATE with one-line reasons. Final legal judgment stays with counsel.”

The pattern: Name the file. Name the audience decision. Name the must-includes. Name the format. Name the verification rule. Phase anything consequential.

8. Multiple-response elimination bank

Select-TWO/THREE prompting items usually pair one correct craft move with distractors. Practice this: eliminate these options:

- “make it better” (no defect named)
- ‘add temperature=0’ (API parameter, incorrect approach)
- “use the model with the most capability” (Domain 01 prestige bias)
- “let Claude find the statistics” (Claude sources load-bearing facts)
- “combine brainstorm and compliance review into one prompt” (mode mixing)
- “re-paste the policy every message” (configuration failure)

Correct pairs typically combine a *structural* move with a *verification* move. Structural moves: schema, decomposition, examples. Verification moves: assumptions block, quote-the-source, checkpoint.

Decision trees

Situation	Action
Multi-deliverable or high stakes	Decompose with checkpoints
Extraction into a system	Define a schema. Use NULL for missing. Sample first.
Analysis request	Rubric plus Evidence/Unknowns separation
Vague stakeholder ask	Translate slang into Product/Process/Performance. Clarify.
Output misses style	Add examples and counterexamples. Do not add adjectives.
“Make it better” urge	Name the defect and the constraint instead
Same brief needed weekly	Move rules to Project instructions / Skill
Two tightened iterations still fail	Re-diagnose the layer (inputs, feature, knowledge) — Domain 07

Exam traps

1. A very large prompt with no checkpoints for consequential work

2. A role with no success criteria (“act as a CFO” alone)
 3. You let Claude source load-bearing numbers that a stakeholder must own
 4. Mix of brainstorm and analysis in one prompt
 5. “Make it better” / “be more professional” without concrete criteria
 6. Restatement of stable policies every turn instead of Project instructions
 7. Research asks with no scope, timeframe, or source rules
 8. Decomposition of trivial tasks (unnecessary process)
 9. Change of five prompt variables at once while you debug
 10. Answers to product-exam stems with API knobs (temperature, fine-tuning)
-

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A PM types: “Analyze our Q4 plan.” Claude returns generic strategy prose. What is the best next prompt? **A:** Rebuild the brief. Attach the plan file. State the decision it informs. Give evaluation criteria. Require Evidence vs Unknowns. The failure is a missing Product/Performance Description. The failure is not the model.

Q2. Select TWO Performance-Description elements for a customer-email draft. **A:** A gold example of an approved email. Explicit banned claims/phrases. (Adjectives like “professional” are not acceptance tests.)

Q3. Legal must review a 60-page contract for renewal risks. What is the strongest structure? **A:** Decompose. Extract clauses relevant to renewal. Tag each with evidence quotes. Then assess risk with a rubric. Then do a human legal review. Extract-then-judge keeps errors easy to trace.

Q4. Claude still ignores the uploaded pricing PDF. What is the best iteration? **A:** Name the file explicitly (“answer only from Pricing-v3.pdf. Quote the section. Say Not found otherwise”). A file pointer is better than a louder restatement of the question.

Q5. A team wants campaign ideas *and* a compliance-safe shortlist. How do you prompt? **A:** Use two phases. First diverge: many labeled-speculative ideas, no judgment. Then converge: rank against criteria plus a banned-claims file. Mode mixing in one prompt lowers quality of both.

Q6. Where do standing tone rules for every chat belong? **A:** Project instructions. Stable rules are configuration, not per-prompt typing.

Q7. A manager’s feedback is “make it better.” Why is this the exam’s wrong option, and what is the fix? **A:** It names no defect or constraint. Improvement is then luck. Fix: state what missed (audience, length, missing risks). Add the constraint.

Q8. Select TWO benefits of decomposition on consequential work. **A:** Human checkpoints before irreversible steps. Separation of extraction from judgment so errors stay localized.

Q9. A rewrite for executives must not change meaning. What constraint do you add? **A:** Meaning locks: “preserve all facts and numbers exactly. Change structure/length/jargon only. List any sentence whose meaning shifted.” Adapt is not invent (Domain 03 bridge).

Q10. The ask touches employment-law wording. The associate is not sure what is allowed. What is the prompt move? **A:** Require an ASSUMPTIONS block. Route the output to human legal review. Clarify or escalate. Do not invent on regulated content.

Q11. Claude must fill a tracker with vendor, date, amount, owner. What is the prompt pattern? **A:**

Define the schema. Use one row per record. Use NULL if missing. Then validate a sample before you scale. Schemas cut revision cycles.

Q12. A draft cites market stats that sound impressive. Nobody provided them. What is the iteration? **A:** Ban unsourced figures. Supply approved numbers, or require UNKNOWN plus an Unknowns section. Drafting strategy equals must-include facts. It does not equal model-sourced facts.

Q13. When is single-shot prompting the right answer? **A:** Small, clear, low-stakes tasks. Decomposition of an icebreaker request is unnecessary process.

Q14. “Act as a McKinsey partner and review our strategy” does not perform well. What is missing? **A:** Success criteria and inputs. A role without a rubric and documents is a role name only, not a complete brief.

Q15. Select TWO analysis-prompt requirements. **A:** An explicit rubric/criteria list. Separation of facts from inferences (with unknowns labeled).

Q16. The output is a long block of prose. Ops needs it in the ticket template. What is the fix? **A:** Add a format contract: exact sections/fields, table shape, length caps. Keep it in the Project so the format stays.

Q17. A researcher wants “everything about competitor X.” What is a better research prompt? **A:** Scope it. Name the questions, the timeframe, and the source types. Require citations. Then use Research mode. Validate citation support afterward.

Q18. After two tightened iterations, extraction still fails on scanned PDFs. What is the diagnosis? **A:** Stop more prompt edits. The input layer (OCR-poor scans) is the problem. Two failed tightened rounds signal a layer change, not more wording (Domain 07).

Quick review checklist (pre-exam)

- ☐ Recite the seven anatomy parts and the PPP triad
 - ☐ Decompose: outline-fill, extract-analyze, options-decide, draft-redteam
 - ☐ Iterate: name the defect plus the constraint. Change one variable at a time
 - ☐ Playbooks for analysis / research / drafting / brainstorming
 - ☐ Examples plus counterexamples over adjectives
 - ☐ ASSUMPTIONS block when you proceed under ambiguity
 - ☐ Stable rules → Project instructions / Skills, not repeated prompts
 - ☐ Prompt-comparison scoring: a complete, non-contradictory brief wins
-

Glossary

Term	Meaning
Prompt anatomy	Role, goal, inputs, constraints, process, format, verification
Description triad	Product / Process / Performance — what, how, and what counts as good
Decomposition	Split of work into phases with checkpoints

Term	Meaning
Iteration	Refinement that names the defect and adds a constraint
Few-shot	Control with worked examples (and counterexamples)
Schema	Fixed output structure for extraction/handoff
ASSUMPTIONS block	Explicit list of guesses made under ambiguity
Meaning lock	Rewrite constraint that keeps facts and numbers exactly

Domain 02 — Prompting & Task Execution

Maps to official CCAO-F Prompting and Task Execution (~14%, ~8 questions)

Dedup note (2026-08-23): Rewritten as a single primary-study copy. Earlier builds repeated the same drill blocks ~10x; duplicates removed and content deepened to the Domain 03 standard.

Disclaimer

Original CCAO-F study notes for non-developers using claude.ai (Projects, Artifacts, Skills, Connectors, Research). Grounded in public Anthropic Help Center & product docs, public Claude Academy (Claude 101, AI Fluency 4D framework), and the published CCAO-F blueprint. Independent; not affiliated with Anthropic. Verify live product details on support.claude.com.

Overview

Prompting on CCAO-F equals professional briefing. Official blueprint verbs: **create** effective prompts for business and technical tasks; **apply task decomposition** to structure complex requests; **iterate** prompts to improve quality; **adapt prompting strategies by task type** — the guide names *analysis*, *research*, *drafting*, *brainstorming*. The AI Fluency **Description** triad — Product, Process, Performance — is the spine: what the deliverable is, how to get there, and what “good” looks like.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Effective prompts	create, structure	Prompt anatomy; the Description triad
Task decomposition	apply, sequence	Phased prompts with checkpoints
Iteration	iterate, refine	Named-defect feedback; one variable at a time

Official objective	Exam verbs	What you practice
Task-type adaptation	adapt, select	Analysis / research / drafting / brainstorming playbooks

Deep notes

1. Prompt anatomy

A complete brief carries: **role** (who Claude acts as), **goal** (deliverable + the decision it supports), **inputs** (files by name, pasted data, connector sources), **constraints** (length, tone, banned claims, audience), **process** (ordered steps, checkpoints), **format** (schema, sections, table), **verification** (what to check before answering). You rarely need all seven — but exam-best prompts are the ones with no *missing load-bearing* part and no internal contradiction.

Description triad: *Product* Description names the deliverable and audience decision it supports. *Process* Description sequences tools and checkpoints. *Performance* Description sets acceptance tests, examples, and counterexamples. Missing one creates a predictable failure: fluff (no Product), skipped steps (no Process), or stylish wrongness (no Performance).

Stakeholder translation is the associate skill: “make it pop” needs voice samples; “keep it legal” needs disclaimer files and banned claims; “board ready” needs Decision/Risks/Ask structure; “quick thoughts” must be labeled speculative. Stems use stakeholder slang — translate before picking an answer.

Contradictions (“explain like a beginner but keep undefined internal acronyms”) must be resolved explicitly. Unresolved contradictions yield flaky outputs blamed on the model.

Examples beat adjectives. Two or three gold examples (and one counterexample) outperform paragraphs of style description. Few-shot is the highest-leverage lever a non-developer has.

Ambiguity: prefer *clarify* over *invent*. If proceeding, require an explicit ASSUMPTIONS block so wrong guesses are visible and cheap to fix.

2. Task decomposition

Break multi-deliverable or high-stakes work into phases with human checkpoints:

- **Outline** → **fill:** approve structure before prose.
- **Extract** → **analyze:** separate facts from judgment so errors are traceable.
- **Options** → **decide:** generate alternatives without commitment, then choose with criteria.
- **Draft** → **red-team:** produce, then adversarially attack (bridges Domain 03).

Worked patterns: ops postmortem (timeline facts → contributing factors tagged Evidence vs Hypothesis → actions only with named owners); marketing email (facts + legal must-includes → subjects + body → compliance pass against banned claims); HR FAQ (answer only from knowledge; quote section; “Not found” if absent); offsite agenda (brainstorm → cluster → pick → timed agenda → risk check); data cleanup (define schema → validate a 20-row sample via code execution → scale).

Decomposition is also governance: it inserts human gates before irreversible or external steps — the crossover tested in Domains 04 and 06. Decompose when: multiple deliverables, high stakes, mixed

task types (never brainstorm and analyze in one breath), or when you need intermediate checkpoints. Single-shot is fine when the task is small, clear, and low-stakes — decomposing trivia wastes time and usage.

3. Iteration craft

Strong iteration **names the defect and the constraint to add**: “the summary invented a Q3 figure — use only numbers from Q3-report . pdf, mark anything else UNKNOWN” beats “make it better.” Change **one major variable at a time** when debugging so you learn causality. Prefer surgical edits (“fix only the table”) over full regeneration. When a prompt fails twice with tightened wording, the problem is usually inputs, feature choice, or task framing — not word choice (Domain 07’s ladder).

Keep a micro-prompt library: assumptions list; FACTS vs INFERENCES; only the table; NULL if missing; strongest counterargument; meaning-preserving diff; outline only; claim-to-source map; three options without picking; compress without changing numbers.

4. Task-type playbooks (the named blueprint objective)

Task type	Strategy	Signature moves
Analysis	Criteria first, evidence separated from inference	Provide a rubric; demand FACTS vs INFERENCES sections; require counterarguments and unknowns
Research	Scope + sources + citations	Define question, timeframe, source rules; require citation-support (Domain 03); Research feature for multi-source public landscape
Drafting	Must-include facts + audience + format	Supply approved facts (don’t let Claude source them); voice samples; banned phrases; length caps
Brainstorming	Diverge, then converge — separately	Ask for quantity/variety without judgment first; rank with criteria in a second step; label output speculative

Mixing modes is the classic error: asking for “creative ideas but strictly accurate to policy” in one prompt gets neither. Split the phases.

5. Prompting as a system (team scale)

Stable rules migrate out of prompts: recurring instructions → Project instructions; reference detail → knowledge files (pointed to by name); rich procedures → Skills. Shared Projects need priority rules for conflicts (e.g., legal > brand). Store gold examples and counterexamples in knowledge. Teach teammates to paste the ask, not their anxiety — vague urgency without specs wastes usage limits. A prompt worth repeating weekly is configuration, not typing (Domain 05).

6. Timed heuristics for prompt-comparison items

When four candidate prompts are shown, score each on Goal, Inputs, Constraints, Process, Format, Verification; highest complete, non-contradictory score wins. Prefer clarify over invent; prefer phased approvals over one mega-prompt for consequential transformations. API-parameter options (temperature, fine-tuning) are distractors on this product-focused exam. Before “escalate the model,” ask: could a careful intern succeed from this brief? If no, fix the Description first.

7. Worked briefs (before → after)

Weak: “Summarize this customer call.” → **Strong:** “From call-2026-08-12.txt, produce a one-page summary for the account exec deciding whether to escalate: 5 bullet facts with timestamps, customer sentiment with a supporting quote, commitments made by either side (quote exactly; UNKNOWN if unclear), and recommended next step with rationale. Do not infer facts not in the transcript.”

Weak: “Write a LinkedIn post about our launch.” → **Strong:** “Draft a LinkedIn post announcing the launch for prospective mid-market buyers. Must include: the two approved claims in launch-facts.md (verbatim numbers), a customer-pain opening, ≤150 words, no superlatives from banned-phrases.md. Match the voice of the two examples in knowledge. Give two variants: safe and bold.”

Weak: “Is this vendor contract okay?” → **Strong:** “Phase 1: extract every clause in vendor-msa.pdf touching liability, termination, data handling, and auto-renewal — quote each. Phase 2 (after my confirmation): assess each extracted clause against the checklist in contract-redlines.md, tag OK / FLAG / ESCALATE with one-line reasons. Final legal judgment stays with counsel.”

The pattern: name the file, the audience decision, the must-includes, the format, the verification rule — and phase anything consequential.

8. Multiple-response elimination bank

Select-TWO/THREE prompting items usually pair one correct craft move with distractors. Practice eliminating: “make it better” (no defect named); “add temperature=0” (API knob, wrong track); “use the biggest model” (Domain 01 prestige bias); “let Claude find the statistics” (sourcing load-bearing facts); “combine brainstorm and compliance review into one prompt” (mode mixing); “re-paste the policy every message” (configuration failure). Correct pairs typically combine a *structural* move (schema, decomposition, examples) with a *verification* move (assumptions block, quote-the-source, checkpoint).

Decision trees

Situation	Action
Multi-deliverable or high stakes	Decompose with checkpoints
Extraction into a system	Define a schema; NULL for missing; sample first
Analysis request	Rubric + Evidence/Unknowns separation
Vague stakeholder ask	Translate slang into
	Product/Process/Performance; clarify
Output misses style	Add examples + counterexamples, not adjectives

Situation	Action
“Make it better” urge	Name the defect and the constraint instead
Same brief needed weekly	Move rules to Project instructions / Skill
Two tightened iterations still fail	Re-diagnose layer (inputs, feature, knowledge) — Domain 07

Exam traps

1. Mega-prompt with no checkpoints for consequential work
2. Role assigned but no success criteria (“act as a CFO” alone)
3. Letting Claude source load-bearing numbers a stakeholder must own
4. Mixing brainstorm and analysis in one prompt
5. “Make it better” / “be more professional” without concrete criteria
6. Restating stable policies every turn instead of Project instructions
7. Research asks with no scope, timeframe, or source rules
8. Decomposing trivial tasks (process theater)
9. Changing five prompt variables at once while “debugging”
10. Answering product-exam stems with API knobs (temperature, fine-tuning)

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A PM types: “Analyze our Q4 plan.” Claude returns generic strategy prose. Best next prompt?
A: Rebuild the brief: attach the plan file, state the decision it informs, give evaluation criteria, require Evidence vs Unknowns. The failure is a missing Product/Performance Description, not the model.

Q2. Select TWO Performance-Description elements for a customer-email draft. **A:** A gold example of an approved email; explicit banned claims/phrases. (Adjectives like “professional” are not acceptance tests.)

Q3. Legal must review a 60-page contract for renewal risks. Strongest structure? **A:** Decompose: extract clauses relevant to renewal → tag each with evidence quotes → then assess risk with a rubric → human legal review. Extract-then-judge keeps errors traceable.

Q4. Claude keeps ignoring the uploaded pricing PDF. Best iteration? **A:** Name the file explicitly (“answer only from Pricing-v3.pdf; quote the section; say Not found otherwise”). Pointing beats repeating the question louder.

Q5. A team wants campaign ideas *and* a compliance-safe shortlist. How do you prompt? **A:** Two phases: diverge (many labeled-speculative ideas, no judgment), then converge (rank against criteria + banned-claims file). Mixing modes in one prompt degrades both.

Q6. Where do standing tone rules used in every chat belong? **A:** Project instructions — stable rules are configuration, not per-prompt typing.

Q7. A manager’s feedback is “make it better.” Why is this the exam’s wrong option, and the fix? **A:** It names no defect or constraint, so improvement is luck. Fix: state what missed (audience, length, missing risks) and add the constraint.

- Q8.** Select TWO benefits of decomposition on consequential work. **A:** Human checkpoints before irreversible steps; separating extraction from judgment so errors are localized.
- Q9.** A rewrite for executives must not change meaning. What constraint do you add? **A:** Meaning locks: “preserve all facts and numbers exactly; change structure/length/jargon only; list any sentence whose meaning shifted.” Adapt ≠ invent (Domain 03 bridge).
- Q10.** The ask touches employment-law wording and the associate isn’t sure what’s allowed. Prompt move? **A:** Require an ASSUMPTIONS block and route the output to human legal review — clarify/escalate beats invention on regulated content.
- Q11.** Claude must fill a tracker with vendor, date, amount, owner. Prompt pattern? **A:** Define the schema, one row per record, NULL if missing, then validate a sample before scaling. Schemas cut revision cycles.
- Q12.** A draft cites impressive-sounding market stats nobody provided. Iteration? **A:** Ban unsourced figures; supply approved numbers or require UNKNOWN + an Unknowns section. Drafting strategy = must-include facts, not model-sourced facts.
- Q13.** When is single-shot prompting the right answer? **A:** Small, clear, low-stakes tasks — decomposing an icebreaker request is process theater.
- Q14.** “Act as a McKinsey partner and review our strategy” underperforms. Missing? **A:** Success criteria and inputs. Role without rubric and documents is costume, not brief.
- Q15.** Select TWO analysis-prompt requirements. **A:** An explicit rubric/criteria list; separation of facts from inferences (with unknowns labeled).
- Q16.** Output is a wall of prose but ops needs it in the ticket template. Fix? **A:** Add format contract: exact sections/fields, table shape, length caps — and keep it in the Project so it sticks.
- Q17.** A researcher wants “everything about competitor X.” Better research prompt? **A:** Scope it: which questions, which timeframe, which source types, citations required — then Research mode; validate citation support afterward.
- Q18.** After two tightened iterations, extraction still fails on scanned PDFs. Diagnosis? **A:** Stop prompt-tweaking — the input layer (OCR-poor scans) is the problem. Two failed tightened rounds signal a layer change, not more wording (Domain 07).
-

Quick review checklist (pre-exam)

- ☐ Recite the seven anatomy parts and the PPP triad
 - ☐ Decompose: outline-fill, extract-analyze, options-decide, draft-redteam
 - ☐ Iterate by naming defect + constraint; one variable at a time
 - ☐ Playbooks for analysis / research / drafting / brainstorming
 - ☐ Examples + counterexamples over adjectives
 - ☐ ASSUMPTIONS block when proceeding under ambiguity
 - ☐ Stable rules → Project instructions / Skills, not repeated prompts
 - ☐ Prompt-comparison scoring: complete, non-contradictory brief wins
-

Glossary

Term	Meaning
Prompt anatomy	Role, goal, inputs, constraints, process, format, verification
Description triad	Product / Process / Performance — what, how, what counts as good
Decomposition	Splitting work into phases with checkpoints
Iteration	Refinement that names the defect and adds a constraint
Few-shot	Steering with worked examples (and counterexamples)
Schema	Fixed output structure for extraction/handoff
ASSUMPTIONS block	Explicit list of guesses made under ambiguity
Meaning lock	Rewrite constraint preserving facts and numbers exactly

Domain 03 — Evaluating & Validating Claude's Output

Maps to official CCAO-F Output Evaluation and Validation (~21%, heaviest, ~13 questions)

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Artifacts, Skills, Connectors, Research, eval, RAG) are exceptions and stay as written.

Dedup note (Aug 2026): This file is one primary-study copy. Earlier pack builds repeated the same drill blocks about 15 times. This edition removes those duplicates.

Disclaimer

These are original CCAO-F study notes for non-developers who use claude.ai (Projects, Artifacts, Skills, Connectors, Research). They use public Anthropic Help Center and product docs, public Claude Academy (Claude 101, AI Fluency 4D), and the published CCAO-F blueprint. They are independent. They are not affiliated with Anthropic. Verify live details on support.claude.com.

Overview

This is the largest CCAO-F domain by weight. Your job is to **judge outputs before they reach customers, executives, or systems of record**. Official blueprint verbs include evaluate accuracy and completeness, and identify hallucinations, inconsistencies, and biases. You must have They also include apply fact-checking, decide when human review , and edit, adapt, refine, and compare for audience. Organise information and select formats (Artifacts, inline, structured data).

Core exam truth: **Claude can be fluent and wrong**. AI Fluency maps here to Discernment (you spot what is unsafe to ship) and Diligence (you build review habits that match the risk). Eloquence is not evidence. An Artifact is not pre-validated. A citation is not proof that the linked text supports the claim.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Accuracy & completeness	evaluate, assess	Claim vs source. Missing sections. Silent omissions
Hallucinations / inconsistencies / biases	identify, spot	Fabrications, math/timeline drift, stereotypes, one-sidedness
Fact-checking & validation	verify, cross-check	Numbers, names, dates, quotes, URLs, policy quotes
Human review gates	determine, escalate	L1–L4 risk tiers. When peer/expert review is mandatory
Edit / adapt / refine / compare	revise, curate	Audience rewrite. Rubric compare. Ship/revise/reject
Format fitness	organise, select	Inline vs Artifact vs structured (tables/Sheets-ready)

Deep notes

1. What “evaluation” means on this exam

Evaluation is a **three-way compare**. (1) the ask / success criteria, (2) sources of truth (uploaded files, Project knowledge, Research citations, policy of record), and (3) the draft. Score across:

- **Correctness** — are load-bearing claims true and supported?
- **Completeness** — is anything material missing relative to the ask or outline?
- **Consistency** — do numbers, dates, owners, and table↔prose agree?
- **Calibration** — does confidence match evidence? Are unknowns labeled?
- **Audience fit** — tone, length, jargon, decision-usefulness for the reader?
- **Policy / safety** — PII, commitments, legal-effect language, refusals honored?
- **Format fit** — is delivery mode (inline / Artifact / structured) appropriate?

Fail any one of these and you may still “revise” rather than “ship.” Completeness failures hide better than wrong claims. A polished one-pager that drops the biggest customer objection is a common Domain 03 trap.

2. Risk tiers (proportional review)

Match review depth to consequence. Exam multiple-response items often pair a **skip-review-on-high-risk** distractor with an **over-validate-trivia** distractor. Choose the proportional set.

Tier	Typical work	Review depth
L1 Low	Labeled brainstorming, icebreakers, internal ideation	Skim. Keep “draft / speculative” label

Tier	Typical work	Review depth
L2 Medium	Internal summary of a short known doc. Team status rewrite	Spot-check key claims + completeness vs outline
L3 High	Customer email with commitments. External publish. Ops runbook change	Full claim map + peer review. Quote sources of truth
L4 Critical	Legal / medical / finance / HR / safety. Employment decisions. Irreversible connector sends	Expert human review + primary sources. Claude is drafting aid only

General rule: if the output creates an **external commitment**, moves **money/people/safety**, or hits a **system of record**, treat at least L3. Irreversible Connector actions (send email, update ticket, write to CRM) always need a **human gate** before execution. This is true even if the draft looks perfect.

3. Claim maps (the Domain 03 signature skill)

A **claim map** turns fluent prose into a checklist you can verify. Method:

1. Extract every load-bearing claim (numbers, names, dates, quotes, attributions, commitments, causal statements, “per Policy X”).
2. Tag each: SUPPORTED (quote or cite the evidence), UNSUPPORTED, CONTRADICTED, AMBIGUOUS, OUT_OF_SCOPE.
3. Decide per claim: keep / rewrite / drop / escalate.
4. Only then polish tone.

Priority scan order (poisoned-number drill): numbers → names → dates → commitments/SLAs → legal or medical assertions → attributions/citations → soft opinions. Exams hide one bad figure inside otherwise fluent paragraphs.

Claim extract script (prompt pattern you can reuse): “List every factual claim as a bullet. For each: claim text | evidence needed | status (SUPPORTED/UNSUPPORTED/...) | suggested fix. Do not rewrite the memo until the map is complete.”

Self-check YES/NO checklists at the end of outputs speed human review. They **do not** replace L3/L4 review.

4. Hallucination types

Know the taxonomy. Stems rarely say “hallucination.” They show symptoms.

1. **Fabricated facts** — statistics, market sizes, “47% per Gartner 2026” with no real support.
2. **Fabricated or mismatched sources** — real-looking URLs that 404. Citations whose body does not support the claim. Invented study titles.
3. **Fabricated attributions** — quoting a person or policy section that does not exist (e.g., “per Policy 4.2” when 4.2 is unrelated or missing).
4. Overconfident inference: you treat a guess as a settled fact. Fake precision. (“exactly 12.47%”).
5. **Context bleed** — mixing details from another chat, Project file, or earlier example into this task.
6. **Instruction amnesia** — ignoring constraints (audience, banned claims, “quote only from file”).

Citations are not automatic validation. Research outputs still need **citation-support checks**: open the source, confirm the sentence actually backs the claim, confirm date/version relevance. Irrelevant but real links still fail validation.

5. Inconsistencies and bias

Inconsistencies: incorrect arithmetic. Timeline contradictions. Table totals $\neq$ prose. Constraint drift mid-answer (“no discounts” then offers 15%). Owner named in summary but absent from transcript.

Bias patterns the exam cares about: stereotype defaults in examples. Position bias (the output gives too much weight to the first source). Fake neutrality that hides a preferred option. One-sided analysis missing counterarguments or risks. Fix by requiring counterarguments, decision criteria, and explicit unknowns. Do not only ask “are you sure?” once.

6. Fact-checking workflow (ship / revise / reject / escalate)

Standard flow for consequential drafts:

1. **Risk-tier** the task (L1–L4).
2. **Extract claims** (map).
3. **Map evidence** to Project knowledge / uploaded policy / Research citations / primary systems.
4. **Spot-check** high-severity claims first (numbers, commitments, legal-effect).
5. **Completeness pass** against the original outline or ask.
6. **Decide:** ship (often labeled) / revise / reject unsupported block / escalate to human expert.
7. **Format** only after content passes—or format choice is itself part of fitness (see below).

Red-team Claude as **assistance, not authority**: Adversarial prompts (‘attack this draft for unsupported commitments’) help you find issues. Outsourcing final regulated judgment to the model fails Diligence.

7. Human-review gates (when you must stop)

Mandatory human / expert review when any of these apply:

- Legal, medical, finance, HR, safety, or employment-effect content
- External customer or public publish
- Irreversible Connector / automation actions
- PII or sensitive personal data in the draft
- Low evidence coverage on load-bearing claims
- Topic beyond your or Claude’s demonstrated competence
- High ambiguity with material downside

If you use Claude to grade Claude, you only find issues. Treating that grade as legal sign-off is an exam-wrong answer. “Are you sure?” once is **not** a validation method.

8. Audience edit, adapt, refine, compare

Adapt $\neq$ invent. Audience rewrite changes length, jargon, structure, and emphasis. It does not change facts. If the exec brief needs a number you do not have, mark UNKNOWN or escalate. Do not fabricate a crisp KPI.

Compare drafts with a rubric table:

Criterion	Draft A	Draft B	Winner / merge note
Accuracy vs source	...	...	...
Completeness vs outline	...	...	...
Calibration / unknowns	...	...	...
Audience fit	...	...	...
Policy / commitments	...	...	...

Merge strengths. Do not prefer a draft only because it is longer or more eloquent. Style-only polish after a failed claim map is a trap answer.

9. Format choice: Artifacts / inline / structured

Wrong format fails **fit-for-purpose** even when content is good.

Format	Use when	Anti-pattern
Inline chat	Short answers, quick clarifications, iterative brainstorming	You send a board pack as raw chat.
Artifact	Standalone editable deliverable (doc, app, interactive table, HTML) you will revise or share	You force a two-word answer into an Artifact 'app'.
Structured (table, bullets, CSV/Sheets-ready)	Output that feeds another step. Easy claim checking. Ops handoff	Prose paragraphs when the next step is a spreadsheet
Research + citations	Multi-source investigation needing cited landscape	Research for a single already-uploaded PDF you should quote
Project knowledge (after approval)	A source of truth for recurring work	You paste unvalidated drafts into knowledge as 'truth'.

Organise and curate: headings, claim lists, decision/evidence/risks sections for leaders. Attractive dashboards with wrong metric definitions fail evaluation. Use **semantics before aesthetics**.

10. Bridges to other domains

- Bad prompts create difficult eval work (02).
- Wrong features raise error rates (01).
- Stale Project knowledge yields true-to-file, false-to-world (05).
- If you ship without gates, you fail governance (06).
- Repeated eval misses become troubleshooting patterns (07).

Fluency without Discernment is a business risk. The **21% weight is intentional**: Anthropic certifies associates who stop fluent errors from becoming organizational facts.

11. Personal SOP (Diligence system)

Keep a one-page rubric. Pre-define L1–L4 rules per workflow. Store known failure examples in a Project. Record almost-shipped failures monthly. Lab loop: Research brief → map ~15 claims → decide ship/revise/reject → Artifact only after pass → write three exam stems from mistakes found.

Validation scripts that operationalize Discernment (they also serve as Domain 02 performance tools): claim extract. Adversarial attack. Audience rewrite diff. Commitment/obligation scanner. Privacy/PII redaction scanner.

Decision trees

Situation	Action
Unsourced critical external claim	Revise or reject. Do not ship
Internal contradiction (35 vs 28, table≠prose)	Revise for consistency
Missing required section vs outline	Completeness fail → revise
Audience mismatch, facts OK	Adapt (tone/structure), do not invent
Labeled low-stakes brainstorm	Ship labeled (L1)
Connector send / write action	Human gate before execution
Employment / legal / medical / finance memo	L4 expert review
Citation present but unsupported	Validation fail (same as no citation)
Safety refusal on disallowed ask	Appropriate—do not “fix” as a bug
Pretty Artifact, wrong metric defs	Revise semantics before ship

Tier path: L1 skim → L2 spot-check → L3 full claim map + peer → L4 expert + primary sources.

Timed claim-map drills (90-second stems)

Work each stem in ~90 seconds: Extract 3 to 6 load-bearing claims, tag the status, then select ship, revise, reject, or escalate. Do **not** rewrite the whole memo in the drill. Map first. Answers follow each stem.

Drill A — Vendor email

Stem: Draft thanks Acme and cites “47% faster close rates per Gartner 2026” linking a lifestyle blog. Offers a 60-day pilot with SLA credits “as discussed” (not in thread). **Map focus:** statistic + source. SLA commitment. **Answer:** Reject/revise unsupported Gartner claim. Strip or rewrite SLA until evidence exists. Get legal/commercial review before send (L3).

Drill B — Meeting notes

Stem: Notes assign Jordan as owner of “migrate billing by Friday”. Transcript never names Jordan. Jordan was not in the invite. **Map focus:** attribution / action owner. **Answer:** Fail attribution. Mark OWNER UNCLEAR. Do not invent owners. Escalate to the facilitator.

Drill C — Refund reply vs policy file

Stem: Chat answers “always refund within 90 days from general knowledge”. Uploaded Refund_Policy_v3.pdf says 30 days for digital goods. **Map focus:** policy conflict. **Answer:** Reject general-knowledge answer. Quote the policy file. Use an L3 customer commitment gate.

Drill D — Research brief citation

Stem: Research claims “EU guidance banned feature X in 2024” with a citation that discusses an unrelated privacy cookie rule. **Map focus:** citation support. **Answer:** Unsupported (mismatched citation). Revise the claim or find a supporting source. Do not ship on link presence alone.

Drill E — Dashboard Artifact

Stem: Interactive Artifact “ARR by segment” uses “bookings” field labeled as ARR. Finance dictionary defines ARR differently. **Map focus:** metric semantics. **Answer:** Revise definitions before aesthetics. Use a structured/finance owner check (L3/L4).

Drill F — Campaign brainstorm

Stem: Clearly labeled “speculative concepts only—not approved messaging”. Playful but risky slogans. **Map focus:** risk tier + label. **Answer:** Ship as labeled L1 draft. Do not treat as brand-approved copy.

Drill G — Exec brief completeness

Stem: Ask required Decision / Evidence / Risks / Ask. Draft has Decision and Evidence only. Omits Risks and Ask. **Map focus:** completeness. **Answer:** Completeness fail → revise. Polished half-briefs still fail.

Drill H — Timeline inconsistency

Stem: Prose says pilot starts 15 Sep. Table says 5 Oct. Both presented as firm. **Map focus:** consistency. **Answer:** Revise. Reconcile dates against source of truth before external share.

Drill I — Biased shortlist

Stem: The vendor comparison praises one tool with strong adjectives. No criteria table. Competitors dismissed in one line. **Map focus:** bias / fake neutrality. **Answer:** Revise to criteria-based compare + counterarguments. Do not ship as “objective analysis.”

Drill J — Connector send

Stem: Draft support reply looks correct. User prompt: “send via Gmail connector now.” **Map focus:** human gate. **Answer:** Human gate before send. Evaluation of text is not authorization to execute.

Drill K — Fabricated policy section

Stem: Memo says “per Employee Handbook §4.2 remote stipend is \$500”. Handbook has no §4.2 stipend language. **Map focus:** fabricated attribution. **Answer:** Reject claim. Quote the real section or mark unsupported. Use HR/expert review (L4).

Drill L — Poisoned number in fluent prose

Stem: Three paragraphs of solid summary. One sentence says “NPS rose from 42 to 71” but source table shows 42 → 47. **Map focus:** number priority scan. **Answer:** Catch on first number pass. Revise 71→47. Ship only after the fix.

(Twelve unique stems—practice rotation, not copy-paste padding.)

12. Mini-cases (apply the full loop)

Case 1 — Customer success email. Claude drafts a renewal note promising “priority roadmap influence” and citing “NPS 72 from last quarter.” Project knowledge has NPS 61. Roadmap influence was never approved. Tier L3. Claim-map both promises. Revise: quote real NPS or drop. Remove unapproved influence language. Use peer review before send. Format: inline for the email body. Optional structured bullet of approved talking points stored in Project after approval—not before.

Case 2 — Competitive landscape Artifact. Research produces a polished HTML Artifact comparing four vendors. Two citations are solid. One statistic traces to a press release restating the vendor’s own claim. One “analyst quote” has no matching source text. Completeness vs ask is good. Correctness is mixed. Action: keep Artifact shell. Rewrite unsupported cells to Unverified. Swap the press-release stat for a primary filing, or drop it. Then ship labeled “draft for review.” Semantics and evidence are more important than visual polish.

Case 3 — HR FAQ. Prompt asks for a friendly FAQ on parental leave. Claude invents a “six weeks fully paid for all contractors” bullet. Handbook covers employees only. Contractors are out of scope. This is L4-adjacent employment-effect content. Reject invented contractor benefit. Escalate to HR. Do not publish FAQ from chat alone. Safety/policy Diligence outweighs helpful tone.

Case 4 — Ops runbook. Structured table of on-call steps looks consistent, but step 4 says “page Secondary if no ack in 5 minutes” while prose intro says 15 minutes. Inconsistency fail. Revise to one SLA from the source runbook. Keep structured format because ops will execute from the table.

These cases rehearse the same loop: tier → map → evidence → completeness/consistency → ship/re-vise/reject/escalate → format.

13. Multiple-response pattern bank

Expect Select TWO/THREE items that mix a correct proportional action with distractors. Common wrong pairs to eliminate:

- Use “Skip review because Sonnet/Opus ” + “Run a full legal review on a labeled brainstorm” (prestige bias + over-validation).
- “Ship because there are citations” + “Delete all citations to reduce clutter” (blind trust + anti-evidence)
- “Autosend via Connector to save time” + “Never use Connectors” (gate failure + absolute ban)
- “Ask Claude if it is sure” as sole control + “Only humans may ever draft” (fake validation + anti-Delegation)

Correct patterns usually combine **claim-level verification** with **risk-proportional human gates** and **format fit**.

Exam traps

1. Eloquence = accuracy
 2. Confidence / fake precision = evidence
 3. Skipping completeness because the prose “looks finished”
 4. Blind trust in citations or Research footnotes
 5. Self-grade / “are you sure?” as legal or HR sign-off
 6. Autosend via Connectors without a human gate
 7. Ignoring audience while “fixing facts later”
 8. You treat safety refusals as defects that you must not defeat.
 9. Style-only polish after a failed claim map
 10. Assuming Artifacts are pre-validated because they look like products
 11. Over-validating L1 trivia while skipping L3 commitments (disproportionality)
 12. Inventing KPIs during an “audience adapt” rewrite
-

Practice Q&A (25) — answers with brief rationales

Q1. Heaviest CCAO-F domain by weight? **A:** Output Evaluation and Validation (~21%). Judgment dominates the associate exam.

Q2. Research footnote URL returns 404. First validation call? **A:** Validation fail for that claim. Treat as unsupported until a working supporting source exists.

Q3. Select TWO strong hallucination signals. **A:** Specific numbers with no verifiable source. Attributions to missing people/policy sections. (Fluent tone is not a signal of truth.)

Q4. Which output most clearly needs human expert review before use? **A:** Employment / HR memo with legal-effect language (L4).

Q5. First step when you evaluate a consequential customer email draft? **A:** Build a claim map (then evidence tags). Do this before tone edits.

Q6. Summary says 35 open issues. Table totals 28. Primary failure type? **A:** Inconsistency (internal contradiction).

Q7. Select TWO completeness checks. **A:** Compare against required outline/sections. Confirm required disclaimers/risks/asks are present.

Q8. Why is a Research brief still not auto-approved? **A:** Citations need support checks. Links can be real yet irrelevant or mismatched.

Q9. Best way to choose between Draft A and Draft B? **A:** Criterion rubric table (accuracy, completeness, calibration, audience, policy). Merge strengths.

Q10. Output must feed a Google Sheet next. Best format? **A:** Structured table / Sheets-ready data. Do not use a long prose Artifact.

Q11. Claude refuses a disallowed medical dosing request. Correct interpretation? **A:** Appropriate safety behavior. It is not a defect to “fix.”

Q12. Select TWO high-risk claim classes. **A:** Customer commitments/SLAs. Legal-effect or employment claims.

- Q13.** Memo cites “Policy 4.2” but 4.2 does not exist. Failure? **A:** Fabricated attribution / unsupported claim → reject or rewrite from real source.
- Q14.** Adapting a technical note for executives means? **A:** Change structure/jargon/length. Do not invent missing metrics.
- Q15.** Role of red-team / adversarial critique prompts? **A:** Assistance to surface issues. Not final regulated authority.
- Q16.** Draft labels three items Unknown—needs finance. Good or bad? **A:** Good calibration. Prefer labeled unknowns over fake precision.
- Q17.** Select TWO correct format matches. **A:** Artifact for interactive standalone deliverable. Inline for short iterative answers.
- Q18.** User asks Claude to send the approved-looking reply via email Connector. Required control? **A:** Human gate before irreversible send.
- Q19.** One-sided vendor write-up. Fix? **A:** Add explicit criteria, counterarguments, and evidence tags. Do not add more adjectives.
- Q20.** Board brief omits Risks and Ask sections required by template. Result? **A:** Completeness fail → revise even if Decision prose is eloquent.
- Q21.** Which AI Fluency pair is central to this domain? **A:** Discernment + Diligence.
- Q22.** End-of-output YES/NO self-check finds two unmet constraints. Next? **A:** Revise to meet constraints. The checklist aids humans. It does not replace L3/L4 review.
- Q23.** Select TWO risks of equating eloquence with accuracy. **A:** Shipping fluent false stats. Skipping claim maps because the draft “sounds right.”
- Q24.** Claim tagged UNSUPPORTED in an external customer commitment email. Action? **A:** Revise or reject that claim before send. Escalate if commercially sensitive.
- Q25.** Leader asks for a one-page decision brief. Best curation pattern? **A:** Decision / Evidence / Risks / Ask—organised for action, after claim validation.
-

Quick review checklist (pre-exam)

- ☐ Risk-tier every consequential task (L1–L4)
 - ☐ Claim-map numbers, names, dates, commitments first
 - ☐ Verify citation **support**, not citation **presence**
 - ☐ Consistency pass: math, timelines, table↔prose
 - ☐ Completeness vs outline / required sections
 - ☐ Human gates for legal/HR/finance/safety, external publish, Connector writes, PII
 - ☐ Audience adapt without inventing facts
 - ☐ Format: inline vs Artifact vs structured—on purpose
 - ☐ Honor safety refusals. Do not treat as bugs
 - ☐ Second-pass / red-team Claude as aid, not sign-off
 - ☐ Proportional review: neither skip L3 nor over-police L1
-

Glossary

Term	Meaning
Hallucination	Fluent false or unsupported content (facts, sources, attributions, overconfident inference)
Claim mapping	Extract → evidence-tag → decide keep/rewrite/drop/escalate
Calibration	Confidence matched to evidence. Unknowns labeled
Completeness	Material requirements present vs ask/outline—not merely “correct sentences”
Consistency	Internal agreement across prose, tables, numbers, timelines
Human-in-the-loop	Required human/expert gate before high-consequence use or irreversible action
Red-team	Adversarial critique to surface failures. Assistance only
Fit for purpose	Content and format suitable for audience and next workflow step
Unsupported	Claim lacks adequate evidence from an allowed source of truth
Risk tier (L1–L4)	Proportional review depth from labeled brainstorm to critical expert review

Worked evaluations (unique examples — once)

1. **Vendor email** invents “47% per Gartner 2026” with unrelated blog citation → reject claim. Rewrite with allowed proof. Legal/commercial review.
2. **Meeting notes** invent action owner absent from transcript → fail attribution. Enforce OWNER UNCLEAR.
3. **Speculative campaign** clearly labeled → ship as draft only (L1).
4. **Refund answer** from general knowledge conflicts uploaded policy → reject. Quote policy file.
5. An attractive Artifact dashboard uses metric definitions that differ from the finance dictionary → revise semantics before aesthetics.

Poisoned-number drill: Check numbers, names, dates, and commitments before you check tone. Exams hide one bad figure in fluent prose.

Validation scripts (once)

Claim extract. Adversarial attack. Audience rewrite diff. Commitment/obligation scanner. Privacy/PII redaction scanner. Self-check YES/NO lists speed review. They do not replace L3/L4 gates.

Personal SOP lab (once)

Research brief → map ~15 claims → decide → Artifact only after pass → write three exam stems from mistakes found. Store failure examples in a Project. Log near-misses monthly as organizational

Diligence.

Domain 03 — Evaluating & Validating Claude’s Output

Maps to official CCAO-F Output Evaluation and Validation (~21%, heaviest, ~13 questions)

Dedup note (Aug 2026): This file was rewritten as a single primary-study copy. Earlier pack builds repeated the same drill blocks ~15×; those duplicates are removed.

Disclaimer

Original CCAO-F study notes for non-developers using claude.ai (Projects, Artifacts, Skills, Connectors, Research). Grounded in public Anthropic Help Center & product docs, public Claude Academy (Claude 101, AI Fluency 4D), and the published CCAO-F blueprint. Independent; not affiliated with Anthropic. Verify live details on support.claude.com.

Overview

Largest CCAO-F domain by weight. Your job is to **judge outputs before they reach customers, executives, or systems of record**. Official blueprint verbs: evaluate accuracy and completeness; identify hallucinations, inconsistencies, and biases; apply fact-checking; decide when human review is required; edit, adapt, refine, and compare for audience; organise information and select formats (Artifacts, inline, structured data).

Core exam truth: **Claude can be fluent and wrong**. AI Fluency maps here to **Discernment** (spotting what is unsafe to ship) and **Diligence** (building review habits that scale with risk). Eloquence is not evidence. An Artifact is not pre-validated. A citation is not proof that the linked text supports the claim.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Accuracy & completeness	evaluate, assess	Claim vs source; missing sections; silent omissions
Hallucinations / inconsistencies / biases	identify, spot	Fabrications, math/timeline drift, stereotypes, one-sidedness
Fact-checking & validation	verify, cross-check	Numbers, names, dates, quotes, URLs, policy quotes
Human review gates	determine, escalate	L1–L4 risk tiers; when peer/expert review is mandatory
Edit / adapt / refine / compare	revise, curate	Audience rewrite; rubric compare; ship/revise/reject
Format fitness	organise, select	Inline vs Artifact vs structured (tables/Sheets-ready)

Deep notes

1. What “evaluation” means on this exam

Evaluation is a **three-way compare**: (1) the ask / success criteria, (2) sources of truth (uploaded files, Project knowledge, Research citations, policy of record), and (3) the draft. Score across:

- **Correctness** — are load-bearing claims true and supported?
- **Completeness** — is anything material missing relative to the ask or outline?
- **Consistency** — do numbers, dates, owners, and table↔prose agree?
- **Calibration** — does confidence match evidence? Are unknowns labeled?
- **Audience fit** — tone, length, jargon, decision-usefulness for the reader?
- **Policy / safety** — PII, commitments, legal-effect language, refusals honored?
- **Format fit** — is delivery mode (inline / Artifact / structured) appropriate?

Fail any one of these and you may still “revise” rather than “ship.” Completeness failures hide better than wrong claims: a polished one-pager that drops the biggest customer objection is a classic Domain 03 trap.

2. Risk tiers (proportional review)

Match review depth to consequence. Exam multiple-response items often pair a **skip-review-on-high-risk** distractor with an **over-validate-trivia** distractor. Choose the proportional set.

Tier	Typical work	Review depth
L1 Low	Labeled brainstorming, icebreakers, internal ideation	Skim; keep “draft / speculative” label
L2 Medium	Internal summary of a short known doc; team status rewrite	Spot-check key claims + completeness vs outline
L3 High	Customer email with commitments; external publish; ops runbook change	Full claim map + peer review; quote sources of truth
L4 Critical	Legal / medical / finance / HR / safety; employment decisions; irreversible connector sends	Expert human review + primary sources; Claude is drafting aid only

Rule of thumb: if the output creates an **external commitment**, moves **money/people/safety**, or hits a **system of record**, treat at least L3. Irreversible Connector actions (send email, update ticket, write to CRM) always need a **human gate** before execution—even if the draft looks perfect.

3. Claim maps (the Domain 03 signature skill)

A **claim map** turns fluent prose into a checklist you can verify. Method:

1. Extract every load-bearing claim (numbers, names, dates, quotes, attributions, commitments, causal statements, “per Policy X”).
2. Tag each: SUPPORTED (quote or cite the evidence), UNSUPPORTED, CONTRADICTED, AMBIGUOUS, OUT_OF_SCOPE.

3. Decide per claim: keep / rewrite / drop / escalate.
4. Only then polish tone.

Priority scan order (poisoned-number drill): numbers → names → dates → commitments/SLAs → legal or medical assertions → attributions/citations → soft opinions. Exams hide one bad figure inside otherwise fluent paragraphs.

Claim extract script (prompt pattern you can reuse): “List every factual claim as a bullet. For each: claim text | evidence needed | status (SUPPORTED/UNSUPPORTED/...) | suggested fix. Do not rewrite the memo until the map is complete.”

Self-check YES/NO checklists at the end of outputs speed human review. They **do not** replace L3/L4 review.

4. Hallucination types

Know the taxonomy; stems rarely say “hallucination”—they show symptoms.

1. **Fabricated facts** — statistics, market sizes, “47% per Gartner 2026” with no real support.
2. **Fabricated or mismatched sources** — real-looking URLs that 404; citations whose body does not support the claim; invented study titles.
3. **Fabricated attributions** — quoting a person or policy section that does not exist (e.g., “per Policy 4.2” when 4.2 is unrelated or missing).
4. **Overconfident inference** — treating a guess as settled fact; fake precision (“exactly 12.47%”).
5. **Context bleed** — mixing details from another chat, Project file, or earlier example into this task.
6. **Instruction amnesia** — ignoring constraints (audience, banned claims, “quote only from file”).

Citations are not automatic validation. Research outputs still need **citation-support checks**: open the source, confirm the sentence actually backs the claim, confirm date/version relevance. Irrelevant but real links still fail validation.

5. Inconsistencies and bias

Inconsistencies: math that does not add; timeline contradictions; table totals ≠ prose; constraint drift mid-answer (“no discounts” then offers 15%); owner named in summary but absent from transcript.

Bias patterns the exam cares about: stereotype defaults in examples; position bias (over-weighting the first source); fake neutrality that hides a preferred option; one-sided analysis missing counterarguments or risks. Fix by requiring counterarguments, decision criteria, and explicit unknowns—not by asking “are you sure?” once.

6. Fact-checking workflow (ship / revise / reject / escalate)

Standard flow for consequential drafts:

1. **Risk-tier** the task (L1–L4).
2. **Extract claims** (map).
3. **Map evidence** to Project knowledge / uploaded policy / Research citations / primary systems.
4. **Spot-check** high-severity claims first (numbers, commitments, legal-effect).
5. **Completeness pass** against the original outline or ask.
6. **Decide:** ship (often labeled) / revise / reject unsupported block / escalate to human expert.
7. **Format** only after content passes—or format choice is itself part of fitness (see below).

Red-team Claude as **assistance, not authority**: adversarial prompts (“attack this draft for unsupported commitments”) help surface issues. Outsourcing final regulated judgment to the model fails Diligence.

7. Human-review gates (when you must stop)

Mandatory human / expert review when any of these apply:

- Legal, medical, finance, HR, safety, or employment-effect content
- External customer or public publish
- Irreversible Connector / automation actions
- PII or sensitive personal data in the draft
- Low evidence coverage on load-bearing claims
- Topic beyond your or Claude’s demonstrated competence
- High ambiguity with material downside

Using Claude to **grade Claude** is fine for surfacing issues. Treating that grade as legal sign-off is an exam-wrong answer. “Are you sure?” once is **not** a validation method.

8. Audience edit, adapt, refine, compare

Adapt ≠ invent. Audience rewrite changes length, jargon, structure, and emphasis—not facts. If the exec brief needs a number you do not have, mark UNKNOWN or escalate; do not fabricate a crisp KPI.

Compare drafts with a rubric table:

Criterion	Draft A	Draft B	Winner / merge note
Accuracy vs source	...	...	...
Completeness vs outline	...	...	...
Calibration / unknowns	...	...	...
Audience fit	...	...	...
Policy / commitments	...	...	...

Merge strengths instead of worshipping length or eloquence. Style-only polish after a failed claim map is a trap answer.

9. Format choice: Artifacts / inline / structured

Wrong format fails **fit-for-purpose** even when content is good.

Format	Use when	Anti-pattern
Inline chat	Short answers, quick clarifications, iterative brainstorming	Dumping a board pack as raw chat
Artifact	Standalone editable deliverable (doc, app, interactive table, HTML) you will revise or share	Forcing a two-word answer into an Artifact “app”
Structured (table, bullets, CSV/Sheets-ready)	Output that feeds another step; easy claim checking; ops handoff	Prose paragraphs when the next step is a spreadsheet

Format	Use when	Anti-pattern
Research + citations	Multi-source investigation needing cited landscape	Research for a single already-uploaded PDF you should quote
Project knowledge (after approval)	Living source of truth for recurring work	Pasting unvalidated drafts into knowledge as “truth”

Organise and curate: headings, claim lists, decision/evidence/risks sections for leaders. Pretty dashboards with wrong metric definitions fail evaluation—**semantics before aesthetics**.

10. Bridges to other domains

- Bad prompts create eval nightmares **(02)**.
- Wrong features raise error rates **(01)**.
- Stale Project knowledge yields true-to-file, false-to-world **(05)**.
- Shipping without gates fails governance **(06)**.
- Repeated eval misses become troubleshooting patterns **(07)**.

Fluency without Discernment is a business risk. The **21% weight is intentional**: Anthropic certifies associates who stop fluent errors from becoming organizational facts.

11. Personal SOP (Diligence system)

Keep a one-page rubric. Pre-define L1–L4 rules per workflow. Store known failure examples in a Project. Log near-misses monthly. Lab loop: Research brief → map ~15 claims → decide ship/revise/reject → Artifact only after pass → write three exam stems from mistakes found.

Validation scripts that operationalize Discernment (and double as Domain 02 performance tools): claim extract; adversarial attack; audience rewrite diff; commitment/obligation scanner; privacy/PII redaction scanner.

Decision trees

Situation	Action
Unsourced critical external claim	Revise or reject; do not ship
Internal contradiction (35 vs 28, table≠prose)	Revise for consistency
Missing required section vs outline	Completeness fail → revise
Audience mismatch, facts OK	Adapt (tone/structure), do not invent
Labeled low-stakes brainstorm	Ship labeled (L1)
Connector send / write action	Human gate before execute
Employment / legal / medical / finance memo	L4 expert review
Citation present but unsupported	Validation fail (same as no citation)
Safety refusal on disallowed ask	Appropriate—do not “fix” as a bug
Pretty Artifact, wrong metric defs	Revise semantics before ship

Tier path: L1 skim → L2 spot-check → L3 full claim map + peer → L4 expert + primary sources.

Timed claim-map drills (90-second stems)

Work each stem in ~90 seconds: extract 3–6 load-bearing claims, tag status, pick ship / revise / reject / escalate. Do **not** rewrite the whole memo in the drill—map first. Answers follow each stem.

Drill A — Vendor email

Stem: Draft thanks Acme and cites “47% faster close rates per Gartner 2026” linking a lifestyle blog; offers a 60-day pilot with SLA credits “as discussed” (not in thread). **Map focus:** statistic + source; SLA commitment. **Answer:** Reject/revise unsupported Gartner claim; strip or rewrite SLA until evidence exists; legal/commercial review before send (L3).

Drill B — Meeting notes

Stem: Notes assign Jordan as owner of “migrate billing by Friday”; transcript never names Jordan; Jordan was not in the invite. **Map focus:** attribution / action owner. **Answer:** Fail attribution; mark OWNER UNCLEAR; do not invent owners—escalate to facilitator.

Drill C — Refund reply vs policy file

Stem: Chat answers “always refund within 90 days from general knowledge”; uploaded Refund_Policy_v3.pdf says 30 days for digital goods. **Map focus:** policy conflict. **Answer:** Reject general-knowledge answer; quote policy file; L3 customer commitment gate.

Drill D — Research brief citation

Stem: Research claims “EU guidance banned feature X in 2024” with a citation that discusses an unrelated privacy cookie rule. **Map focus:** citation support. **Answer:** Unsupported (mismatched citation); revise claim or find supporting source; do not ship on link presence alone.

Drill E — Dashboard Artifact

Stem: Interactive Artifact “ARR by segment” uses “bookings” field labeled as ARR; finance dictionary defines ARR differently. **Map focus:** metric semantics. **Answer:** Revise definitions before aesthetics; structured/finance owner check (L3/L4).

Drill F — Campaign brainstorm

Stem: Clearly labeled “speculative concepts only—not approved messaging”; fun but edgy slogans. **Map focus:** risk tier + label. **Answer:** Ship as labeled L1 draft; do not treat as brand-approved copy.

Drill G — Exec brief completeness

Stem: Ask required Decision / Evidence / Risks / Ask; draft has Decision and Evidence only; omits Risks and Ask. **Map focus:** completeness. **Answer:** Completeness fail → revise; polished half-briefs still fail.

Drill H — Timeline inconsistency

Stem: Prose says pilot starts 15 Sep; table says 5 Oct; both presented as firm. **Map focus:** consistency. **Answer:** Revise; reconcile dates against source of truth before external share.

Drill I — Biased shortlist

Stem: Vendor comparison praises one tool with glowing adjectives; no criteria table; competitors dismissed in one line. **Map focus:** bias / fake neutrality. **Answer:** Revise to criteria-based compare + counterarguments; do not ship as “objective analysis.”

Drill J — Connector send

Stem: Draft support reply looks correct; user prompt: “send via Gmail connector now.” **Map focus:** human gate. **Answer:** Human gate before send; evaluation of text ≠ authorization to execute.

Drill K — Fabricated policy section

Stem: Memo says “per Employee Handbook §4.2 remote stipend is \$500”; handbook has no §4.2 stipend language. **Map focus:** fabricated attribution. **Answer:** Reject claim; quote real section or mark unsupported; HR/expert review (L4).

Drill L — Poisoned number in fluent prose

Stem: Three paragraphs of solid summary; one sentence says “NPS rose from 42 to 71” but source table shows 42 → 47. **Map focus:** number priority scan. **Answer:** Catch on first number pass; revise 71→47; ship only after fix.

(Twelve unique stems—practice rotation, not copy-paste padding.)

12. Mini-cases (apply the full loop)

Case 1 — Customer success email. Claude drafts a renewal note promising “priority roadmap influence” and citing “NPS 72 from last quarter.” Project knowledge has NPS 61; roadmap influence was never approved. Tier L3. Claim-map both promises. Revise: quote real NPS or drop; remove unapproved influence language; peer review before send. Format: inline for the email body; optional structured bullet of approved talking points stored in Project after approval—not before.

Case 2 — Competitive landscape Artifact. Research produces a polished HTML Artifact comparing four vendors. Two citations are solid; one statistic traces to a press release restating the vendor’s own claim; one “analyst quote” has no matching source text. Completeness vs ask is good; correctness is mixed. Action: keep Artifact shell, rewrite unsupported cells to Unverified, swap press-release stat for primary filing or drop, then ship labeled “draft for review.” Semantics and evidence beat visual polish.

Case 3 — HR FAQ. Prompt asks for a friendly FAQ on parental leave. Claude invents a “six weeks fully paid for all contractors” bullet. Handbook covers employees only; contractors are out of scope. This is L4-adjacent employment-effect content. Reject invented contractor benefit; escalate to HR; do not publish FAQ from chat alone. Safety/policy Diligence outweighs helpful tone.

Case 4 — Ops runbook. Structured table of on-call steps looks consistent, but step 4 says “page Secondary if no ack in 5 minutes” while prose intro says 15 minutes. Inconsistency fail. Revise to one SLA from the source runbook; keep structured format because ops will execute from the table.

These cases rehearse the same loop: tier → map → evidence → completeness/consistency → ship/re-
vise/reject/escalate → format.

13. Multiple-response pattern bank

Expect Select TWO/THREE items that mix a correct proportional action with distractors. Common wrong pairs to eliminate:

- “Skip review because Sonnet/Opus was used” + “Run a full legal review on a labeled brainstorm” (prestige bias + over-validation)
- “Ship because there are citations” + “Delete all citations to reduce clutter” (blind trust + anti-evidence)
- “Autosend via Connector to save time” + “Never use Connectors” (gate failure + absolute ban)
- “Ask Claude if it is sure” as sole control + “Only humans may ever draft” (fake validation + anti-Delegation)

Correct patterns usually combine **claim-level verification** with **risk-proportional human gates** and **format fit**.

Exam traps

1. Eloquence = accuracy
2. Confidence / fake precision = evidence
3. Skipping completeness because the prose “looks finished”
4. Blind trust in citations or Research footnotes
5. Self-grade / “are you sure?” as legal or HR sign-off
6. Autosend via Connectors without a human gate
7. Ignoring audience while “fixing facts later”
8. Treating safety refusals as bugs to jailbreak
9. Style-only polish after a failed claim map
10. Assuming Artifacts are pre-validated because they look like products
11. Over-validating L1 trivia while skipping L3 commitments (disproportionality)
12. Inventing KPIs during an “audience adapt” rewrite

Practice Q&A (25) — answers with brief rationales

Q1. Heaviest CCAO-F domain by weight? **A:** Output Evaluation and Validation (~21%). Judgment dominates the associate exam.

Q2. Research footnote URL returns 404. First validation call? **A:** Validation fail for that claim—treat as unsupported until a working supporting source exists.

Q3. Select TWO strong hallucination signals. **A:** Specific numbers with no verifiable source; attributions to missing people/policy sections. (Fluent tone is not a signal of truth.)

Q4. Which output most clearly needs human expert review before use? **A:** Employment / HR memo with legal-effect language (L4).

- Q5.** First step when evaluating a consequential customer email draft? **A:** Build a claim map (then evidence tags)—before tone edits.
- Q6.** Summary says 35 open issues; table totals 28. Primary failure type? **A:** Inconsistency (internal contradiction).
- Q7.** Select TWO completeness checks. **A:** Compare against required outline/sections; confirm required disclaimers/risks/asks are present.
- Q8.** Why is a Research brief still not auto-approved? **A:** Citations need support checks—links can be real yet irrelevant or mismatched.
- Q9.** Best way to choose between Draft A and Draft B? **A:** Criterion rubric table (accuracy, completeness, calibration, audience, policy)—merge strengths.
- Q10.** Output must feed a Google Sheet next. Best format? **A:** Structured table / Sheets-ready data—not a long prose Artifact.
- Q11.** Claude refuses a disallowed medical dosing request. Correct interpretation? **A:** Appropriate safety behavior—not a defect to “fix.”
- Q12.** Select TWO high-risk claim classes. **A:** Customer commitments/SLAs; legal-effect or employment claims.
- Q13.** Memo cites “Policy 4.2” but 4.2 does not exist. Failure? **A:** Fabricated attribution / unsupported claim → reject or rewrite from real source.
- Q14.** Adapting a technical note for executives means? **A:** Change structure/jargon/length—not invent missing metrics.
- Q15.** Role of red-team / adversarial critique prompts? **A:** Assistance to surface issues; not final regulated authority.
- Q16.** Draft labels three items Unknown—needs finance. Good or bad? **A:** Good calibration—prefer labeled unknowns over fake precision.
- Q17.** Select TWO correct format matches. **A:** Artifact for interactive standalone deliverable; inline for short iterative answers.
- Q18.** User asks Claude to send the approved-looking reply via email Connector. Required control? **A:** Human gate before irreversible send.
- Q19.** One-sided vendor write-up; fix? **A:** Add explicit criteria, counterarguments, and evidence tags—not more adjectives.
- Q20.** Board brief omits Risks and Ask sections required by template. Result? **A:** Completeness fail → revise even if Decision prose is eloquent.
- Q21.** Which AI Fluency pair is central to this domain? **A:** Discernment + Diligence.
- Q22.** End-of-output YES/NO self-check finds two unmet constraints. Next? **A:** Revise to meet constraints; checklist aids humans—does not replace L3/L4 review.
- Q23.** Select TWO risks of equating eloquence with accuracy. **A:** Shipping fluent false stats; skipping claim maps because the draft “sounds right.”
- Q24.** Claim tagged UNSUPPORTED in an external customer commitment email. Action? **A:** Revise or reject that claim before send; escalate if commercially sensitive.

Q25. Leader asks for a one-page decision brief. Best curation pattern? **A:** Decision / Evidence / Risks / Ask—organised for action, after claim validation.

Quick review checklist (pre-exam)

- ☐ Risk-tier every consequential task (L1–L4)
 - ☐ Claim-map numbers, names, dates, commitments first
 - ☐ Verify citation **support**, not citation **presence**
 - ☐ Consistency pass: math, timelines, table↔prose
 - ☐ Completeness vs outline / required sections
 - ☐ Human gates for legal/HR/finance/safety, external publish, Connector writes, PII
 - ☐ Audience adapt without inventing facts
 - ☐ Format: inline vs Artifact vs structured—on purpose
 - ☐ Honor safety refusals; do not treat as bugs
 - ☐ Second-pass / red-team Claude as aid, not sign-off
 - ☐ Proportional review: neither skip L3 nor over-police L1
-

Glossary

Term	Meaning
Hallucination	Fluent false or unsupported content (facts, sources, attributions, overconfident inference)
Claim mapping	Extract → evidence-tag → decide keep/rewrite/drop/escalate
Calibration	Confidence matched to evidence; unknowns labeled
Completeness	Material requirements present vs ask/outline—not merely “correct sentences”
Consistency	Internal agreement across prose, tables, numbers, timelines
Human-in-the-loop	Required human/expert gate before high-consequence use or irreversible action
Red-team	Adversarial critique to surface failures; assistance only
Fit for purpose	Content and format suitable for audience and next workflow step
Unsupported	Claim lacks adequate evidence from an allowed source of truth
Risk tier (L1–L4)	Proportional review depth from labeled brainstorm to critical expert review

Worked evaluations (unique examples — once)

1. **Vendor email** invents “47% per Gartner 2026” with unrelated blog citation → reject claim; rewrite with allowed proof; legal/commercial review.
2. **Meeting notes** invent action owner absent from transcript → fail attribution; enforce OWNER UNCLEAR.
3. **Speculative campaign** clearly labeled → ship as draft only (L1).
4. **Refund answer** from general knowledge conflicts uploaded policy → reject; quote policy file.
5. **Pretty Artifact dashboard** uses wrong metric definitions vs finance dictionary → revise semantics before aesthetics.

Poisoned-number drill: train eyes to hit numbers, names, dates, commitments before tone. Exams hide one bad figure in fluent prose.

Validation scripts (once)

Claim extract; adversarial attack; audience rewrite diff; commitment/obligation scanner; privacy/PII redaction scanner. Self-check YES/NO lists speed review—they do not replace L3/L4 gates.

Personal SOP lab (once)

Research brief → map ~15 claims → decide → Artifact only after pass → write three exam stems from mistakes found. Store failure examples in a Project; log near-misses monthly as organizational Diligence.

Domain 04 — Workflow Integration & Solution Design

Maps to official CCAO-F Workflow Integration and Solution Design (~16%, ~10 questions)

Note: This edition follows the ASD-STE100 Simplified Technical English writing rules. Simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Projects, Artifacts, Skills, Connectors, Research) are exceptions and stay as written.

Dedup note (2026-08-23): This file is one primary-study copy. Earlier builds repeated the same drill blocks about 7 times. Those duplicates are gone. The content now matches the Domain 03 depth.

Disclaimer

These notes are original CCAO-F study notes. They are for people who are not developers. They use claude.ai (Projects, Artifacts, Skills, Connectors, Research). The notes use public Anthropic Help Center and product docs. They also use public Claude Academy (Claude 101, AI Fluency 4D) and the published CCAO-F blueprint. The notes are independent. They are not affiliated with Anthropic. Check live product details on support.claude.com.

Overview

This domain has the second-highest exam weight. You design how Claude fits actual work. Official blueprint verbs: **apply** Claude to analyze requirements and use cases. **Leverage** Claude for research, planning, and process optimization. **Use** Claude to support solution design, development, and iteration. **Integrate** Claude into existing workflows to augment or redesign them. **Communicate** Claude's value **and limitations** to stakeholders. For people who are not developers, solutions use Projects, Skills, Artifacts, Connectors, Research, code execution, and review gates. Solutions do not use API architecture.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Requirements & use cases	analyze, elicit	The work to complete, constraints, use-case classes
Research / planning / optimization	leverage, plan	Research that leads to a validated brief. Process maps
Solution design & iteration	design, compose, pilot	Feature composition patterns. Pilot, then measure, then tighten
Integrate: augment vs redesign	integrate, decide	Where Claude drafts, retrieves, or transforms. Gates
Stakeholder communication	communicate, disclose	Value + limits + risks + metrics

Deep notes

1. Requirements and use-case analysis first

Collect requirements before you select tools. Record the trigger event, inputs, outputs, users, frequency, latency need, sensitivity class, downstream systems, definition of done, and error budget. If stakeholders cannot answer, Claude can help you interview and outline. Humans confirm the answers. Tool selection before requirements is a common exam error.

Classify the use case: **generate, transform, retrieve, decide, act**. Decide and act need stronger gates. Decisions usually stay with humans. Generate and transform still need Discernment that matches the stakes. Map the steps of the current workflow. Mark where Claude drafts, retrieves, transforms, computes, or (rarely) decides.

2. Solution pattern catalog

- **Pattern A — Knowledge Hub:** A Project with policies, examples, and instructions. Team edit rights. Sonnet as the default. An evaluation checklist inside the instructions.
- **Pattern B — Landscape to Decision:** Research with citations. Then a Domain 03 claim map. Then a decision memo Artifact.
- **Pattern C — Intake Desk:** A Connector reads a mailbox or form. Claude drafts ticket fields. A human confirms. Then the system updates. The human gate before the write is the point.

- **Pattern D — Playbook Skill:** A Skill holds the procedure (for example, incident updates). The Project holds severity definitions.
- **Pattern E — Org Brain:** Enterprise Search for “what did we decide,” with citations back to Slack or Drive.
- **Pattern F — Data-to-Deliverable:** You upload data. Code execution computes totals, trends, and charts. Then it generates the Excel, PDF, or slides deliverable. A human reviews numbers against the source. Use this pattern when the core of the workflow is *computation or file production*, not prose. Code execution and file creation are available on all plans (verified on support.claude.com, Aug 2026).

Compose patterns. One feature is rarely a full solution. Exam answers that name a careful composition score better than answers that name one feature as a complete solution.

3. Augment vs redesign

Augment: Keep the process. Insert Claude at draft, retrieve, or compute steps. Change-management cost stays low. This is the default for regulated or high-stakes flows. Add audit trails. **Redesign:** Remove steps that AI and connectors make unnecessary. Payoff is higher. Redesign needs change management, training, and clear ownership.

Decision logic: High manual waste, low risk, and willing owners make a redesign candidate. For regulated, external-facing, or irreversible flows, augment first with human gates. Earn redesign with pilot evidence. Never design full autopilot for customer-facing sends. Draft plus human send is the exam-correct shape.

4. Research, planning, and process optimization

Use Research for the external landscape during planning. Validate Research per Domain 03 before it informs decisions. Use Projects to keep the plan context. Use code execution to quantify (“which step consumes the hours?”). Optimization targets follow measurement, not guesses. First measure time-to-draft, revision cycles, and error types. Then redesign the worst step with the smallest correct change.

5. Stakeholder communication (value AND limitations — blueprint-explicit)

Write a one-page brief: problem, proposed workflow diagram, human gates, data flows, metrics, risks, and the request. Demonstrate with real messy inputs, not polished sample data. A *caught* hallucination builds more trust than a perfect run. Always include hallucination risk and the review SOP. Include data boundaries (what may enter which plan or feature, and what may not — Domain 06). Include what Claude should not own (final decisions, regulated judgment). Include success metrics and rollback.

Objection handling: “It will replace us” → show augmentation metrics and how you move hours to higher-value work. “It hallucinates” → show the evaluation SOP and error rates after review. “Data leaves our systems” → show plan controls, connector auth scopes, and org policies. Claims of value without limits fail this domain. Fear without a pilot plan also fails this domain.

6. Measurement and iteration

Track time saved, revision cycles to ship, error types (factual, completeness, tone, policy), adoption, and usage-limit consumption. Use the error taxonomy to update instructions, knowledge, and Skills.

Do not use it to blame users. Pilot one team. Expand after metrics pass. Treat solutions as products: version them, write a changelog, and name an owner.

Integration failure modes: stale knowledge, unclear owners, gates skipped under deadline pressure, and ignored adoption (“nobody uses the solution”). Design for the actual organization. Do not design for the organization on the slide.

7. Mini-cases (apply the full loop)

Case 1 — Weekly exec digest. Communications spends four hours each Friday to compile a digest from Slack, Drive docs, and two dashboards. Design: Connectors read the sources. A Project holds the digest template, voice examples, and inclusion criteria. Claude drafts on Friday morning. The communications lead validates claims (Domain 03) and sends. Metrics: prep time, and corrections after send. After four clean weeks, this is a redesign candidate. Remove the manual dashboard screenshot step. Export the data so code execution can chart it.

Case 2 — RFP factory. Sales answers 20 RFPs a quarter from a wiki that nobody updates. Solution: a curated answer library in Project knowledge (owned by sales ops, versioned). A Skill holds the answer-assembly procedure. An extraction schema covers RFP questions. A human review gate sits before submission, because answers are contractual commitments. Reject the anti-pattern “auto-fill and submit.” External commitments keep a human gate.

Case 3 — Support triage. Goal: cut first-response time. A Connector reads the queue. Claude classifies severity and drafts responses that cite the policy Project. Agents review and send. Sev-1 routes straight to humans with a Claude-drafted context brief. The severity *decision* for high-impact tickets stays with humans. Drafting and context assembly is where Claude adds speed with safety.

Case 4 — Board-pack analysis. The CFO wants trend commentary from monthly exports. Use Data-to-Deliverable. Code execution computes trends and builds charts plus the PDF appendix. The controller verifies computed figures against the source. Opus or Sonnet drafts commentary *from the computed numbers only*. A human owns the narrative. Composition: code execution (math) + Project (voice, definitions) + human gate (regulated audience).

Each case practices the same loop: requirements → use-case class → pattern composition → gates → metrics → iterate.

8. Multiple-response pattern bank

Common wrong pairs to remove: “automate the send to save time” plus “keep everything manual forever” (gate failure plus anti-Delegation). Also “promise stakeholders zero errors” plus “lead with everything that can go wrong” (over-claim plus fear — both omit the SOP-and-metrics middle). Also “roll out org-wide immediately” plus “pilot indefinitely without expansion criteria” (no evidence plus no decision rule). Also “design a custom API microservice” (wrong track). Correct combinations pair a **composition** choice with a **communication or measurement** choice. Composition examples: Project + connector + gate. Research + validation. Code execution + review. Communication or measurement examples: you disclose limits, you define a metric, and you name an owner.

Decision trees

Situation	Action
Irreversible external action in flow	Human gate before execution. This is not optional.
Recurring work with shared context	Project (+ Skill if the procedure is rich)
Multi-source public discovery	Research (validated). For internal sources: Enterprise Search
Workflow core is computation / file output	Code execution pattern F
Stakeholder needs interactive tool	Artifact
Unclear requirements	Do a discovery interview and a process map first
Regulated / high-stakes process	Augment with gates and an audit trail. Redesign only with evidence
Success undefined	Define metrics before you build
Pilot succeeded	Expand in steps. Keep measuring. Assign an owner

Exam traps

1. Full autopilot on customer-facing sends (no human gate)
2. You skip the limitations half of stakeholder communication
3. You sell Claude as error-free, or you refuse pilots out of fear
4. You ignore change management and training on redesigns
5. API or microservice architecture answers on a product-track exam
6. No success metrics or rollback plan
7. One-feature answers where composition is needed
8. Tool-first design with no requirements or process map
9. You optimize steps that nobody measured
10. You forget adoption: solutions without an owner decay

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A director says “put Claude into our invoicing process” with no further detail. First step? **A:** Requirements and use-case analysis. Map the process, inputs and outputs, sensitivity, and definition of done before you select features. Tool selection before requirements is the trap.

Q2. Design a customer-support email workflow with Claude. Exam-correct shape? **A:** Claude drafts from Project policy knowledge. A human reviews and sends. Customer-facing sends keep a human gate.

Q3. Select TWO components for a weekly competitive brief solution. **A:** A Project that holds the brief template and instructions. Research (with citation validation) for the landscape sweep.

Q4. Your rollout slide only lists benefits. Add What must to pass this domain’s bar? **A:** Limitations: hallucination risk and review SOP, data boundaries, what stays with humans, metrics, and rollback. Communication of limits is explicit in the blueprint.

Q5. Finance wants monthly variance analysis from exported CSVs, delivered as Excel. Which pattern?

A: Data-to-Deliverable: code execution computes variances and generates the workbook. A human checks numbers against the source before circulation.

Q6. When does redesign beat augmentation? **A:** High manual waste, low risk, willing owners, and pilot evidence. Remove steps that AI makes unnecessary. Regulated flows start as augmentation with audit trails.

Q7. Which metric pair best shows workflow value? **A:** Time-to-shipped-draft plus error rate after review. Speed without quality (or quality without speed) is an incomplete case.

Q8. Ops wants a shared interactive SOP checker teammates can click through. Feature? **A:** An Artifact. It is a standalone interactive deliverable. Viewers consume their own limits. You still validate content first.

Q9. A pilot's outputs show recurring tone misses and two factual errors traced to an outdated policy PDF. Iteration? **A:** Update the knowledge file (staleness). Add tone examples to instructions or the Skill. Fix the system that caused each miss type.

Q10. An exec asks "will this replace the team?" Best response frame? **A:** Change the frame to augmentation. Show which steps Claude absorbs, the human judgment you keep, the metrics, and where the freed hours go.

Q11. Select TWO risks that must be disclosed to stakeholders in a solution proposal. **A:** Hallucination and error risk with its mitigation. Data-sensitivity boundaries and handling rules.

Q12. The team wants to skip a pilot and roll out org-wide "to move fast." Why is this the wrong option? **A:** There is no error taxonomy, no adoption evidence, and no rollback. Pilots keep failure local while instructions and knowledge are still raw.

Q13. Intake emails should become CRM tickets. Where is the gate? **A:** Claude drafts ticket fields from the email that the connector reads. A human confirms before the CRM write. Reads can be broad. Writes are gated.

Q14. "What did we agree with vendor X last quarter?" keeps failing in chat. Solution? **A:** Enterprise Search over connected Slack and Drive. This is org-internal cited lookup. It is not Research. It is not a bigger model.

Q15. The team re-explains a recurring RFP-response procedure (12 steps, strict order) every week. Solution component? **A:** A Skill that holds the procedure, with the boilerplate and source material in Project knowledge.

Q16. How do you present a demo that builds durable trust? **A:** Use real messy inputs. Show the evaluation step. Show a caught error with its fix. This proves the error-catching step, not only the impressive output.

Q17. Post-rollout, usage is high but revision cycles doubled. Diagnosis path? **A:** Segment the error taxonomy. Find which miss types grew and which workflow steps produce them. Tighten instructions and knowledge for those steps. Measure → target → cheapest lever.

Q18. Select TWO signs a proposed solution is over-automated for its risk level. **A:** Irreversible external actions execute without human confirmation. Regulated judgments (HR, legal, finance) go to Claude's output directly.

Quick review checklist (pre-exam)

- ☐ Collect requirements and classify the use case (generate/transform/retrieve/decide/act) first
 - ☐ Six solution patterns, including Data-to-Deliverable via code execution
 - ☐ Augment vs redesign logic and change management
 - ☐ Human gates on irreversible and external actions — always
 - ☐ Stakeholder communication = value + limits + risks + metrics + rollback
 - ☐ Pilot → error taxonomy → tighten → expand. Named owner
 - ☐ No API-architecture answers. Compose product features
-

Glossary

Term	Meaning
Use case classes	Generate, transform, retrieve, decide, act. Gates scale with the last two.
Augmentation	You insert Claude into an existing process at draft, retrieve, or compute steps.
Redesign	You remove process steps that AI and connectors make unnecessary.
Human gate	Required human confirmation before irreversible or external actions.
Pilot	A bounded rollout that generates the error taxonomy before you scale.
Error taxonomy	Factual / completeness / tone / policy miss classification that drives fixes.
Success metric	A predefined measure (time-to-draft, post-review error rate, adoption).
Change management	Training, ownership, and communication that make redesigns stay in place.

Domain 04 — Workflow Integration & Solution Design

Maps to official CCAO-F Workflow Integration and Solution Design (~16%, ~10 questions)

Dedup note (2026-08-23): Rewritten as a single primary-study copy. Earlier builds repeated the same drill blocks ~7×; duplicates removed and content deepened to the Domain 03 standard.

Disclaimer

Original CCAO-F study notes for non-developers using claude.ai (Projects, Artifacts, Skills, Connectors, Research). Grounded in public Anthropic Help Center & product docs, public Claude Academy (Claude 101, AI Fluency 4D), and the published CCAO-F blueprint. Independent; not affiliated with Anthropic. Verify live product details on support.claude.com.

Overview

Second-heaviest domain. Design how Claude fits real work. Official blueprint verbs: **apply** Claude to analyze requirements and use cases; **leverage** Claude for research, planning, and process optimization; **use** Claude to support solution design, development, and iteration; **integrate** Claude into existing workflows to augment or redesign them; **communicate** Claude's value **and limitations** to stakeholders. Non-developer solutions mean Projects, Skills, Artifacts, Connectors, Research, code execution, and review gates — not API architecture.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Requirements & use cases	analyze, elicit	Job-to-be-done, constraints, use-case classes
Research / planning / optimization	leverage, plan	Research→validated brief; process mapping
Solution design & iteration	design, compose, pilot	Feature composition patterns; pilot→measure→tighten
Integrate: augment vs redesign	integrate, decide	Where Claude drafts/retrieves/transforms; gates
Stakeholder communication	communicate, disclose	Value + limits + risks + metrics

Deep notes

1. Requirements and use-case analysis first

Elicit before tooling: trigger event, inputs, outputs, users, frequency, latency need, sensitivity class, downstream systems, definition of done, error budget. If stakeholders can't answer, Claude can help interview and outline — but humans confirm. Premature tooling is the classic fail.

Classify the use case: **generate, transform, retrieve, decide, act**. Decide/act demand stronger gates (decisions usually stay human). Generate/transform still need Discernment proportional to stakes. Map the current workflow's steps and mark where Claude drafts, retrieves, transforms, computes, or (rarely) decides.

2. Solution pattern catalog

- **Pattern A — Knowledge Hub:** Project with policies/examples/instructions; team edit rights; Sonnet default; evaluation checklist inside instructions.
- **Pattern B — Landscape to Decision:** Research with citations → Domain 03 claim map → decision memo Artifact.
- **Pattern C — Intake Desk:** Connector reads a mailbox/form → Claude drafts ticket fields → human confirms → system update. The human gate before the write is the point.
- **Pattern D — Playbook Skill:** Skill encodes procedure (e.g., incident updates); Project holds severity definitions.

- **Pattern E — Org Brain:** Enterprise Search for “what did we decide” with citations back to Slack/Drive.
- **Pattern F — Data-to-Deliverable:** upload data → code execution computes (totals, trends, charts) → generates the Excel/PDF/slides deliverable → human reviews numbers against source. Use when the workflow’s core is *computation or file production*, not prose. (Code execution and file creation are available on all plans — verified support.claude.com, Aug 2026.)

Compose patterns; rarely is one feature a full solution. Exam answers naming a thoughtful composition beat single-feature silver bullets.

3. Augment vs redesign

Augment: keep the process, insert Claude at draft/retrieve/compute steps. Low change-management cost; the default for regulated or high-stakes flows (plus audit trails). **Redesign:** remove steps made unnecessary by AI + connectors. Higher payoff, but needs change management, training, and clear ownership.

Decision logic: high manual waste + low risk + willing owners → redesign candidate. Regulated, external-facing, or irreversible flows → augment first with human gates; earn redesign with pilot evidence. Never design “full autopilot” for customer-facing sends — draft + human send is the exam-correct shape.

4. Research, planning, and process optimization

Use Research for the external landscape during planning (validated per Domain 03 before it informs decisions); Projects to persist the plan’s context; code execution to quantify (“which step consumes the hours?”). Optimization targets follow measurement, not vibes: instrument time-to-draft, revision cycles, error types first, then redesign the worst step with the cheapest lever.

5. Stakeholder communication (value AND limitations — blueprint-explicit)

One-pager: problem, proposed workflow diagram, human gates, data flows, metrics, risks, ask. Demo with real messy inputs, not polished toys; showing a *caught* hallucination builds more trust than a perfect run. Always include: hallucination risk and the review SOP; data boundaries (what may/may not enter which plan/feature — Domain 06); what Claude should not own (final decisions, regulated judgment); success metrics and rollback.

Objection handling: “It will replace us” → augmentation metrics and higher-value work reallocation. “It hallucinates” → show the evaluation SOP and error rates after review. “Data leaves our systems” → plan controls, connector auth scopes, org policies. Hype without limits fails this domain; so does doom without a pilot plan.

6. Measurement and iteration

Track: time saved, revision cycles to ship, error types (factual, completeness, tone, policy), adoption, usage-limit consumption. Use the error taxonomy to update instructions/knowledge/Skills — not to blame users. Pilot one team; expand after metrics clear. Solutions are products: version them, changelog them, name an owner.

Integration failure modes: stale knowledge, unclear owners, gates bypassed under deadline pressure, adoption ignored (“we built it, nobody came”). Design for the real org, not the slide org.

7. Mini-cases (apply the full loop)

Case 1 — Weekly exec digest. Comms spends four hours each Friday compiling a digest from Slack, Drive docs, and two dashboards. Design: connectors read the sources; a Project holds the digest template, voice examples, and inclusion criteria; Claude drafts Friday morning; the comms lead validates claims (Domain 03) and sends. Metrics: prep time, corrections after send. Redesign candidate after four clean weeks: drop the manual dashboard screenshot step by exporting data for code execution to chart.

Case 2 — RFP factory. Sales answers 20 RFPs a quarter from a wiki nobody updates. Solution: curated answer library in Project knowledge (owned by sales ops, versioned); a Skill encoding the answer-assembly procedure; extraction schema for RFP questions; human review gate before submission because answers are contractual commitments. Anti-pattern rejected: “auto-fill and submit” — external commitments keep a human gate.

Case 3 — Support triage. Goal: cut first-response time. Connector reads the queue; Claude classifies severity and drafts responses citing the policy Project; agents review-and-send; Sev-1 routes straight to humans with a Claude-drafted context brief. The severity *decision* for high-impact tickets stays human; the drafting and context assembly is where Claude adds speed safely.

Case 4 — Board-pack analysis. CFO wants trend commentary from monthly exports. Data-to-Deliverable: code execution computes trends and builds charts + the PDF appendix; the controller verifies computed figures against source; Opus/Sonnet drafts commentary *from the computed numbers only*; human owns the narrative. Composition: code execution (math) + Project (voice, definitions) + human gate (regulated audience).

Each case rehearses the same loop: requirements → use-case class → pattern composition → gates → metrics → iterate.

8. Multiple-response pattern bank

Common wrong pairs to eliminate: “automate the send to save time” + “keep everything manual forever” (gate failure + anti-Delegation); “promise stakeholders zero errors” + “lead with everything that can go wrong” (hype + doom — both omit the SOP-and-metrics middle); “roll out org-wide immediately” + “pilot indefinitely without expansion criteria” (no evidence + no decision rule); “design a custom API microservice” (wrong track). Correct combinations pair a **composition** choice (Project + connector + gate; Research + validation; code execution + review) with a **communication or measurement** choice (limits disclosed, metric defined, owner named).

Decision trees

Situation	Action
Irreversible external action in flow	Human gate before execution — non-negotiable
Recurring work with shared context	Project (+ Skill if procedure-rich)
Multi-source public discovery	Research (validated) — internal: Enterprise Search
Workflow core is computation / file output	Code execution pattern F
Stakeholder needs interactive tool	Artifact
Unclear requirements	Discovery interview + process map first

Situation	Action
Regulated / high-stakes process	Augment with gates + audit trail; redesign only with evidence
Success undefined	Define metrics before building
Pilot succeeded	Expand gradually; keep measuring; assign owner

Exam traps

1. Full autopilot on customer-facing sends (no human gate)
2. Skipping the limitations half of stakeholder communication
3. Selling Claude as error-free; or refusing pilots out of fear
4. Ignoring change management and training on redesigns
5. API/microservice architecture answers on a product-track exam
6. No success metrics or rollback plan
7. Single-feature silver bullets where composition is needed
8. Tool-first design with no requirements or process map
9. Optimizing steps nobody measured
10. Forgetting adoption: unowned solutions rot

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A director says “put Claude into our invoicing process” with no further detail. First step? **A:** Requirements and use-case analysis — map the process, inputs/outputs, sensitivity, and definition of done before choosing features. Tooling before requirements is the trap.

Q2. Design a customer-support email workflow with Claude. Exam-correct shape? **A:** Claude drafts from Project policy knowledge → human reviews and sends. Customer-facing sends keep a human gate.

Q3. Select TWO components for a weekly competitive brief solution. **A:** A Project holding the brief template/instructions; Research (with citation validation) for the landscape sweep.

Q4. Your rollout slide only lists benefits. What must be added to pass this domain’s bar? **A:** Limitations: hallucination risk + review SOP, data boundaries, what stays human, metrics and rollback. Communicating limits is blueprint-explicit.

Q5. Finance wants monthly variance analysis from exported CSVs, delivered as Excel. Which pattern? **A:** Data-to-Deliverable: code execution computes variances and generates the workbook; a human checks numbers against source before circulation.

Q6. When does redesign beat augmentation? **A:** High manual waste, low risk, willing owners, and pilot evidence — remove steps AI makes unnecessary. Regulated flows start as augmentation with audit trails.

Q7. Which metric pair best evidences workflow value? **A:** Time-to-shipped-draft plus error rate after review. Speed without quality (or the reverse) is half a case.

Q8. Ops wants a shared interactive SOP checker teammates can click through. Feature? **A:** An Artifact — standalone interactive deliverable; viewers consume their own limits; content still validated first.

Q9. A pilot’s outputs show recurring tone misses and two factual errors traced to an outdated policy PDF. Iteration? **A:** Update the knowledge file (staleness) and add tone examples to instructions/Skill — fix the system that caused each miss type.

Q10. An exec asks “will this replace the team?” Best response frame? **A:** Reframe to augmentation: show which steps Claude absorbs, the human judgment retained, metrics, and where freed hours go.

Q11. Select TWO risks that must be disclosed to stakeholders in a solution proposal. **A:** Hallucination/error risk with its mitigation; data-sensitivity boundaries and handling rules.

Q12. The team wants to skip a pilot and roll out org-wide “to move fast.” Why is this the wrong option? **A:** No error taxonomy, no adoption evidence, no rollback; pilots localize failure while instructions/knowledge are still raw.

Q13. Intake emails should become CRM tickets. Where’s the gate? **A:** Claude drafts ticket fields from the connector-read email; a human confirms before the CRM write. Reads can be liberal; writes are gated.

Q14. “What did we agree with vendor X last quarter?” keeps hitting chat dead-ends. Solution? **A:** Enterprise Search over connected Slack/Drive — org-internal cited lookup, not Research and not bigger models.

Q15. A recurring RFP-response procedure (12 steps, strict order) is being re-explained weekly. Solution component? **A:** A Skill encoding the procedure, with the boilerplate/source material in Project knowledge.

Q16. How do you present a demo that builds durable trust? **A:** Real messy inputs, show the evaluation step, and show a caught error with its fix — proves the safety net, not just the magic.

Q17. Post-rollout, usage is high but revision cycles doubled. Diagnosis path? **A:** Segment the error taxonomy: which miss types grew, which workflow steps produce them; tighten instructions/knowledge for those steps. Measure → target → cheapest lever.

Q18. Select TWO signs a proposed solution is over-automated for its risk level. **A:** Irreversible external actions execute without human confirmation; regulated judgments (HR/legal/finance) are delegated to Claude’s output directly.

Quick review checklist (pre-exam)

- ☐ Elicit requirements + classify use case (generate/transform/retrieve/decide/act) first
 - ☐ Six solution patterns, including Data-to-Deliverable via code execution
 - ☐ Augment vs redesign logic + change management
 - ☐ Human gates on irreversible/external actions — always
 - ☐ Stakeholder comms = value + limits + risks + metrics + rollback
 - ☐ Pilot → error taxonomy → tighten → expand; named owner
 - ☐ No API-architecture answers; compose product features
-

Glossary

Term	Meaning
Use case classes	Generate, transform, retrieve, decide, act — gates scale with the last two
Augmentation	Claude inserted into an existing process at draft/retrieve/compute steps
Redesign	Removing process steps made unnecessary by AI + connectors
Human gate	Required human confirmation before irreversible/external actions
Pilot	Bounded rollout that generates the error taxonomy before scaling
Error taxonomy	Factual / completeness / tone / policy miss classification driving fixes
Success metric	Predefined measure (time-to-draft, post-review error rate, adoption)
Change management	Training, ownership, and communication that make redesigns stick

Domain 05 — Configuration & Knowledge Management

Maps to official CCAO-F Configuration and Knowledge Management (~12%, ~7 questions)

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, MCP, Projects, RAG, Memory) are exceptions and stay as written. This is one primary-study copy. Earlier builds repeated the same drill blocks about 7 times. Those duplicates are gone. The content now matches the Domain 03 depth. Check Plan facts against support.claude.com. This edition adds a full **Memory** section. Memory is a named exam-guide topic. Coverage was near zero before.

Disclaimer

These notes are original CCAO-F study notes. They are for people who are not developers. They use claude.ai (Projects, Artifacts, Skills, Connectors, Research). The notes use public Anthropic Help Center and product docs. They also use public Claude Academy (Claude 101, AI Fluency 4D) and the published CCAO-F blueprint. The notes are independent. They are not affiliated with Anthropic. Check live product details on support.claude.com.

Overview

Configure Claude so that recurring work stays consistent. Official blueprint verbs: **configure** Claude Projects with instructions and knowledge sources. **Manage** uploaded knowledge and connectors (the guide names Google Drive and Gmail). **Create** effective system-level instructions. **Inform, maintain,**

and update configurations, knowledge sources, and instructions over time. This domain helps you make the setup reliable for the team. Memory is now part of this domain. You also manage what Claude remembers.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Configure Projects	configure, set up	Instructions + knowledge + sharing + ownership
Manage knowledge & connectors	manage, curate, authorize	Upload vs connect. Hygiene. Least privilege
Effective instructions	create, structure	Short, prioritized, conflict rules, schemas
Maintain & update	maintain, refresh, retire	Cadence, changelogs, archiving, Memory curation

Deep notes

1. Project setup that survives contact with a team

Checklist: purpose. Owner. Sensitivity label. Instructions. Knowledge set. Sharing permissions. Model-default guidance. Evaluation notes.

Verified plan facts (support.claude.com, Aug 2026): Projects exist on **all plans**. **Free** caps at five projects. **Enhanced knowledge via RAG is paid-only**. RAG expands capacity up to about **10x**. RAG starts when knowledge approaches context limits. **Sharing is on Team and Enterprise**. Read “Can view” -only. “Can edit” lets you change knowledge and instructions. You grant access to one person, to a group, or to the whole org.

2. Instruction engineering

Good instructions specify role, audience defaults, output schemas, and citation rules. Add escalation, conflict priority, and file pointers by name.

Bad instructions tell the company story. They contradict each other. They hide critical rules.

Keep instructions **short, stable, and prioritized**. Instructions compete with task tokens. Put bulky policy in knowledge files.

Test instructions with three golden tasks and three adversarial tasks. In the adversarial tasks, ask for answers that are not in knowledge. Check that Claude answers “Not found”. Update the instructions when failures cluster.

Styles/preferences vs Project instructions: Styles hold global personal taste. Project instructions hold workspace rules. A Style cannot carry a Project’s legal constraints.

Skills vs instructions: Skills hold procedures. Instructions hold standing rules. Do not paste whole Skills into instructions.

3. Knowledge hygiene playbook

- **Naming:** Use Policy-Refunds-v4-2026-06.pdf. Do not use finalFINAL.pdf. Put version dates in filenames. Ask by filename.
- **Curation:** Remove superseded docs on the day the new version arrives. A stale file is a silent production bug. The answer agrees with the stale file but disagrees with current policy. Do not keep duplicate or conflicting versions. Retrieval confusion is a problem you create.
- **Structure:** Split unrelated domains (legal / HR / marketing) into separate Projects. Keep a manifest note. The note lists each document and its purpose.
- **Quality:** Prefer files that yield extractable text. Scans with unreadable text make retrieval fail.
- **RAG behavior:** When knowledge is large, expect search-like retrieval. Ask with a narrow question. Name files.

4. Connectors: upload vs connect, and least privilege

Upload = a frozen snapshot. You must refresh it. Connector = live data with auth and permission complexity.

Select with intent. Living documents that change weekly → connector. Stable policy of record → curated upload with version discipline.

Governance basics (Domain 06 covers this topic in depth): use least privilege. Prefer **read-only** for analysis. Write scopes are a governance decision. Write scopes need human gates. Team/Enterprise pattern: **admin enables, each user authenticates**. Document which connectors a Project expects. Then “Claude cannot see Drive” is an auth problem. It is not a model failure. **Custom connectors (remote MCP) exist on every plan. Free is limited to one custom connector** (verified Aug 2026).

5. Memory — configuring what Claude remembers (added. Named in the exam guide)

Facts verified against support.claude.com (Aug 2026 — this feature evolves. Re-check before exam day):

- **What it is:** Claude builds **memory entries** from your chats. The entries hold professional context, role, preferences, and ongoing work. Claude organizes them into categories. Claude applies them to later conversations. This replaces the old mental model “chats are ephemeral. Nothing persists between threads.”
- **Availability:** Memory generation exists on **Free, Pro, Max** (newer experience) and **Enterprise** (legacy 24-hour synthesis experience). **Chat search** (Claude retrieves your past chats) is **Pro and above**. Team plans have **no org-level memory controls**. Members manage their own memory.
- **Project scoping:** Every Project has its **own separate memory** and summary. This keeps client work, confidential threads, and general chats apart. This is the configuration answer for keeping contexts separate.
- **Controls (this domain’s core):** Settings → Memory: **view** entries, **edit**, **delete individual entries**, or **reset all**. Memory and search **toggle independently**. Citations show which chat a remembered fact came from.
- **Incognito chats:** Ghost icon. All plans. Outside Projects. Excluded from history, memory, and search. (Enterprise retains incognito chats a minimum period for safety and includes them in data exports — this is a governance detail.)
- Include **Portability:** Memory in data exports. Experimental import/export can move memory between assistants.

Management doctrine: Memory is a *personal context layer*. It is not a knowledge base. Curate it like knowledge. Review entries on a schedule. Delete wrong or stale entries. A bad memory entry is configuration drift. You carry that drift into every chat. Disable memory or use incognito for sensitive matters. Keep important team facts in **Project knowledge**. Curated files with owners beat inferred per-user memories. When Claude “remembers wrong,” edit or delete the entry. Do not select a bigger model (Domain 07 bridge).

6. Maintenance, lifecycle, ownership

Every shared Project needs a named owner. That owner is accountable for updates. Without an owner, configuration drift is guaranteed.

Cadence: review instructions monthly. Refresh knowledge when policies change. Replace the file. Announce the change. Update a version note in the instructions. Keep a changelog in knowledge for shared Projects. Archive dead Projects. On Free, the five-project cap forces this.

Onboarding teammates: share the Project. Explain instructions. Show the evaluation checklist. Confirm connector auth. Supervise a first task.

7. Failure modes that look like model bugs

Claude “ignores policy.” A superseded PDF is still in knowledge.

Claude “mixes brands.” Two style guides exist. There is no priority rule.

Claude “forgot our project context.” The user relied on personal Memory instead of Project knowledge. Or the memory entry was deleted. Or the memory is scoped to another Project.

Claude “cannot see Drive.” The connector is not authenticated.

Claude “remembers something wrong about me.” The memory entry is stale. Edit or delete it.

Fix configuration before you escalate models.

8. Mini-cases (configuration audits)

Case 1 — The mystery brand voice. Marketing’s shared Project alternates between formal copy and short, direct copy. Audit finds `Brand-Voice-2025.pdf` and `Voice-Refresh-2026.docx` both in knowledge. There is no priority rule. There is no owner. Fix: delete the 2025 file. Add “voice per `Voice-Refresh-2026.docx`” to instructions. Name an owner. Start a changelog. One audit fixes every user.

Case 2 — The consultant’s bleed risk. A solo consultant serves two competing clients from general chat. The consultant worries that context leaks. Fix: one Project per client (memory is project-scoped. Knowledge is separated). Keep general Memory for personal preferences only. Review memory entries on a schedule. Use incognito for exploratory conflict-sensitive questions.

Case 3 — The Friday-report Project that rotted. Built in March. Praised in April. Wrong by August. Policy v5 shipped. v3 is still in knowledge. The builder left. Nobody owns it. This is common drift. Fix: assign an owner. Replace the file. Add a version note to instructions. Put a monthly review on the calendar. Prevention costs less than the bad report that showed the problem.

Case 4 — Onboarding that failed. A new analyst gets Project access. Their first outputs cite nothing from Drive. Audit: the connector is admin-enabled. The analyst never authenticated. Also they

used their personal account, not the workspace. Fix: workspace seat. Per-user auth walkthrough. Supervised first task against the evaluation checklist.

9. Multiple-response pattern bank

Remove these recurring wrong options:

- “Paste the whole policy into instructions” (bloat — that is knowledge’s job).
- “Keep the old file for reference” (superseded = delete or archive with a clear label).
- “Memory will keep the team aligned” (per-user, not shared).
- “Grant edit to everyone so nobody is blocked” (this is a permission failure).
- “Re-upload weekly instead of connecting” when the doc is living (or its mirror: “connect everything,” including stable policy).

Correct combinations pair a **placement** choice (instructions vs knowledge vs Skill vs connector vs Memory) with a **lifecycle** choice (owner, version note, cadence, archive).

Decision trees

Situation	Action
Recurring team context	Project (instructions + knowledge + owner)
Bulky reference material	Knowledge file, named and versioned — not instructions
Live, frequently changing docs	Connector (read-only unless writes are governed)
Policy updated	Replace file + announce + version note
Personal cross-chat preferences	Memory — review/edit entries in Settings
Confidential single-use chat	Incognito (excluded from memory/history/search)
Contexts must not bleed	Separate Projects (memory is project-scoped)
Free near five-project cap	Archive or merge
Wrong remembered fact	Edit/delete the memory entry
Shared Project misbehaving	Check owner, duplicates, staleness before model blame

Exam traps

1. Everything crammed into instructions (novel-length, self-contradicting)
2. Never updating knowledge. Superseded PDFs left in place
3. Treating personal Memory as the team’s knowledge base
4. Assuming memory crosses Project boundaries (it is scoped per Project)
5. Write-scope connectors by default for analysis-only work
6. Ignoring sharing permissions (org-wide edit on restricted knowledge)
7. Duplicate conflicting files causing retrieval confusion
8. Shared Project with no owner (guaranteed drift)
9. Styles doing a Project’s legal-compliance job
10. Forgetting incognito exists for sensitive one-offs

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A team needs standing tone rules plus a 40-page policy PDF applied to weekly work. Configuration? **A:** Project: short tone rules in instructions. The PDF in knowledge, named with version and date. Instructions point to it by filename.

Q2. A Free-plan consultant hits the project cap when the consultant starts client #6. Options? **A:** Archive or merge an inactive project, or upgrade. Free caps at five (verified Aug 2026. Re-check current caps at support.claude.com).

Q3. Select TWO knowledge-hygiene practices. **A:** Version dates in filenames. Removing superseded documents immediately. (Duplicates and finalFINAL naming are the anti-patterns.)

Q4. Claude cites last quarter's refund window. The v4 policy shipped yesterday. Root cause and fix? **A:** Stale knowledge. Replace v3 with v4. Announce the change. Bump the version note in the instructions. True-to-file, false-to-world is a maintenance failure.

Q5. When does Project RAG matter, and who gets it? **A:** Paid plans, when knowledge approaches context limits. Retrieval expands capacity up to about 10x. Help it with clear naming and narrow questions.

Q6. A user expects a new chat to know yesterday's project decisions automatically. What is the right configuration answer? **A:** Put decisions in Project knowledge (team truth). Memory may recall some context (per-user, plan-dependent, project-scoped). Memory is not a reliable or shared store for load-bearing facts.

Q7. Analysis-only Drive workflow: which connector scope? **A:** Read-only / least privilege. Write scopes need governance approval and human gates.

Q8. Claude keeps addressing a manager by a project they left months ago. Fix? **A:** Settings → Memory: edit or delete the stale entry. Memory is user-curated configuration, not fate.

Q9. "Can view" vs "Can edit" on a shared Project — the difference? **A:** View = use the Project read-only. Edit = modify knowledge and instructions. Grant edit to a small set of people. Sharing is a governance control.

Q10. Where do rich multi-step procedures belong: instructions or a Skill? **A:** A Skill. Procedures load when they are relevant. Instructions carry short standing rules. If you copy a Skill into instructions, you bloat context.

Q11. Two conflicting style guides live in one Project's knowledge. Symptom and fix? **A:** Inconsistent brand outputs from retrieval confusion. Remove one guide, or add an explicit priority rule in instructions.

Q12. An exec wants sensitive board prep with zero trace in memory or history. Recommendation? **A:** Incognito chat (all plans, outside Projects). Exclude It from history, memory, and search. For recurring board work, a dedicated Project with tight sharing beats repeated incognito.

Q13. Enterprise Search returns nothing for a source the team swears is connected. Checks? **A:** Confirm the admin enabled the connector. Confirm that the user who asks has authenticated. Confirm the source is in scope. Check config before you blame the model.

Q14. Who maintains a shared Project, and what artifact proves it? **A:** A named owner. A changelog note in knowledge that records file swaps and instruction versions.

Q15. A Free-plan user wants three custom MCP connectors. Reality? **A:** Custom connectors exist on Free but are limited to one. Multiple custom connectors need a paid plan (verified Aug 2026).

Q16. Personal Styles vs Project instructions — when is Styles the wrong answer? **A:** Whenever the rule is workspace truth (legal constraints, team schemas). Styles are personal taste. Styles do not govern a shared Project.

Q17. Select TWO reasons memory being *project-scoped* matters to configuration. **A:** Client contexts cannot bleed between Projects. Sensitive project discussions stay out of general-chat memory. (This is also why “keep each client in its own Project” is the standing design rule.)

Q18. During teammate onboarding to a configured Project, what two steps prevent week-one chaos? **A:** Confirm their connector authentication. Run a supervised first task against the evaluation checklist.

Quick review checklist (pre-exam)

- ☐ Project setup: purpose, owner, sensitivity, instructions, knowledge, sharing
- ☐ Instructions: short, prioritized, conflict rules, “Not found” escalation, file pointers
- ☐ Knowledge: versioned names, no duplicates, replace superseded, manifest
- ☐ Upload vs connector. Least privilege. Admin-enable + user-auth
- ☐ Free caps: five projects, one custom connector. RAG paid ~10×
- ☐ Memory: plans, project-scoped, view/edit/delete/reset, independent toggles
- ☐ Incognito: all plans, no history/memory/search
- ☐ Maintenance cadence + changelog + named owner. Archive dead Projects

Glossary

Term	Meaning
Project instructions	Standing behavioral rules scoped to a workspace
Project knowledge	Curated reference files. The team’s source of truth
RAG (Projects)	Paid retrieval that expands knowledge capacity near context limits
Connector	Live integration (e.g., Drive, Gmail). Admin-enabled, user-authenticated
Least privilege	Minimum scope that does the job. Read-only default
Memory	Per-user, project-scoped remembered context. Curated in Settings
Memory entry	One editable/deletable remembered fact with chat citation
Incognito chat	Chat excluded from history, memory, and search
Knowledge hygiene	Naming, versioning, duplicate removal, and pruning discipline
Configuration drift	Slow divergence of instructions/knowledge/memory from reality

Domain 05 — Configuration & Knowledge Management

Maps to official CCAO-F Configuration and Knowledge Management (~12%, ~7 questions)

Dedup note (2026-08-23): Rewritten as a single primary-study copy. Earlier builds repeated the same drill blocks ~7x; duplicates removed, content deepened to the Domain 03 standard, plan facts re-verified against support.claude.com, and a full **Memory** section added (it was a named exam-guide topic with near-zero coverage).

Disclaimer

Original CCAO-F study notes for non-developers using claude.ai (Projects, Artifacts, Skills, Connectors, Research). Grounded in public Anthropic Help Center & product docs, public Claude Academy (Claude 101, AI Fluency 4D), and the published CCAO-F blueprint. Independent; not affiliated with Anthropic. Verify live product details on support.claude.com.

Overview

Configure Claude so recurring work stays consistent. Official blueprint verbs: **configure** Claude Projects with instructions and knowledge sources; **manage** uploaded knowledge and connectors (the guide names Google Drive and Gmail); **create** effective system-level instructions; **inform, maintain, and update** configurations, knowledge sources, and instructions over time. This is the “make it reliable for the team” domain — and, since Memory shipped, the “manage what Claude remembers” domain too.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Configure Projects	configure, set up	Instructions + knowledge + sharing + ownership
Manage knowledge & connectors	manage, curate, authorize	Upload vs connect; hygiene; least privilege
Effective instructions	create, structure	Short, prioritized, conflict rules, schemas
Maintain & update	maintain, refresh, retire	Cadence, changelogs, archiving, Memory curation

Deep notes

1. Project setup that survives contact with a team

Checklist: purpose; owner; sensitivity label; instructions; knowledge set; sharing permissions; model-default guidance; evaluation notes. Verified plan facts (support.claude.com, Aug 2026): Projects exist

on **all plans**; **Free caps at five projects**; **enhanced knowledge via RAG is paid-only and expands capacity up to ~10x**, activating as knowledge approaches context limits; **sharing is Team/Enterprise** with “Can view” (read-only) vs “Can edit” (modify knowledge/instructions), granted individually, in bulk, or org-wide.

2. Instruction engineering

Good instructions specify: role; audience defaults; output schemas; citation rules; escalation (“if not in knowledge, say **Not found**”); conflict priority (legal > brand); pointers to files by name. Bad instructions narrate the company novel, contradict themselves, or bury critical rules. Keep them **short, stable, prioritized** — instructions compete with task tokens; bulky policy belongs in knowledge files.

Test instructions with three golden tasks and three adversarial tasks (ask for out-of-knowledge answers and see whether “Not found” fires). Update when failures cluster. **Styles/preferences vs Project instructions**: global personal taste vs workspace rules — a Style can’t carry a Project’s legal constraints. **Skills vs instructions**: procedures vs standing rules; don’t paste whole Skills into instructions.

3. Knowledge hygiene playbook

- **Naming**: Policy-Refunds-v4-2026-06.pdf beats finalFINAL.pdf. Version dates in file-names; ask by filename.
- **Curation**: remove superseded docs the day the new version lands — a stale file is a silent production bug (true to file, false to world). No duplicate/conflicting versions; retrieval confusion is self-inflicted.
- **Structure**: split unrelated domains (legal / HR / marketing) into separate Projects; keep a manifest note listing each document and its purpose.
- **Quality**: prefer text-extractable files; OCR-junk scans poison retrieval.
- **RAG behavior**: when knowledge is large, expect search-like retrieval — ask narrowly and name files.

4. Connectors: upload vs connect, and least privilege

Upload = a frozen snapshot you must refresh; connector = live data with auth and permission complexity. Choose deliberately: living documents that change weekly → connector; stable policy of record → curated upload with version discipline.

Governance basics (Domain 06 owns the deep end): least privilege — prefer **read-only** for analysis; write scopes are a governance decision with human gates. Team/Enterprise pattern: **admin enables, each user authenticates**. Document which connectors a Project expects, so “Claude can’t see Drive” gets diagnosed as auth, not model failure. **Custom connectors (remote MCP) exist on every plan; Free is limited to one custom connector** (verified Aug 2026).

5. Memory — configuring what Claude remembers (added; named in the exam guide)

Facts verified against support.claude.com (Aug 2026 — this feature evolves; re-check before exam day):

- **What it is**: Claude builds **memory entries** from your chats — professional context, role, preferences, ongoing work — organized into categories and applied to later conversations. This replaces the old mental model “chats are ephemeral; nothing persists between threads.”

- **Availability:** memory generation on **Free, Pro, Max** (newer experience) and **Enterprise** (legacy 24-hour synthesis experience); **chat search** (Claude retrieving your past chats) is **Pro and above**. Team plans have **no org-level memory controls** — members manage their own.
- **Project scoping:** every Project has its **own separate memory** and summary, keeping client work, confidential threads, and general chats apart. This is the configuration answer to “keep contexts from bleeding.”
- **Controls (this domain’s core):** Settings → Memory: **view** entries, **edit**, **delete individual entries**, or **reset all**; memory and search **toggle independently**; citations show which chat a remembered fact came from.
- **Incognito chats:** ghost icon, all plans, outside Projects; excluded from history, memory, and search. (Enterprise retains incognito chats a minimum period for safety and includes them in data exports — governance detail.)
- **Portability:** memory is included in data exports; experimental import/export can move memory between assistants.

Management doctrine: Memory is a *personal context layer*, not a knowledge base. Curate it like knowledge: review entries periodically; delete wrong or stale ones (a bad memory entry is configuration drift you carry into every chat); disable or use incognito for sensitive matters; and keep load-bearing team truth in **Project knowledge** — curated files with owners beat inferred per-user memories. When Claude “remembers wrong,” the fix is edit/delete the entry, not a bigger model (Domain 07 bridge).

6. Maintenance, lifecycle, ownership

Every shared Project needs a named owner accountable for updates; without one, configuration drift is guaranteed. Cadence: review instructions monthly; refresh knowledge when policies change (replace file + announce + bump a version note in instructions); changelog in knowledge for shared Projects; archive dead Projects — on Free, the five-project cap forces it. Onboarding teammates: share the Project, explain instructions, show the evaluation checklist, confirm connector auth, supervise a first task.

7. Failure modes that look like model bugs

Claude “ignores policy” → superseded PDF still in knowledge. “Mixes brands” → two style guides with no priority rule. “Forgot our project context” → user relied on personal Memory instead of Project knowledge, or memory entry was deleted/scoped to another Project. “Can’t see Drive” → connector not authenticated. “Remembers something wrong about me” → stale memory entry; edit or delete it. Fix configuration before escalating models.

8. Mini-cases (configuration audits)

Case 1 — The mystery brand voice. Marketing’s shared Project alternates between formal and punchy copy. Audit finds Brand-Voice-2025.pdf and Voice-Refresh-2026.docx both in knowledge, no priority rule, no owner. Fix: delete the 2025 file, add “voice per Voice-Refresh-2026.docx” to instructions, name an owner, start a changelog. One audit, every user fixed.

Case 2 — The consultant’s bleed risk. A solo consultant serves two competing clients from general chat and worries context will leak. Fix: one Project per client (memory is project-scoped; knowledge separated), general Memory kept for personal preferences only, periodic review of memory entries, incognito for exploratory conflict-sensitive questions.

Case 3 — The Friday-report Project that rotted. Built in March, praised in April, wrong by August: policy v5 shipped but v3 is still in knowledge; the builder left; nobody owns it. Classic drift. Fix: assign an owner, replace the file, add a version note to instructions, calendar a monthly review. Prevention was cheaper than the bad report that surfaced it.

Case 4 — Onboarding gone sideways. A new analyst gets Project access, but their first outputs cite nothing from Drive. Audit: connector admin-enabled but the analyst never authenticated; also they used their personal account, not the workspace. Fix: workspace seat, per-user auth walkthrough, supervised first task against the evaluation checklist.

9. Multiple-response pattern bank

Eliminate these recurring wrong options: “paste the whole policy into instructions” (bloat — knowledge’s job); “keep the old file for reference” (superseded = delete or clearly archive); “memory will keep the team aligned” (per-user, not shared); “grant edit to everyone so nobody’s blocked” (permission failure); “re-upload weekly instead of connecting” when the doc is living (or its mirror: “connect everything” including stable policy). Correct combinations pair a **placement** choice (instructions vs knowledge vs Skill vs connector vs Memory) with a **lifecycle** choice (owner, version note, cadence, archive).

Decision trees

Situation	Action
Recurring team context	Project (instructions + knowledge + owner)
Bulky reference material	Knowledge file, named and versioned — not instructions
Live, frequently changing docs	Connector (read-only unless writes are governed)
Policy updated	Replace file + announce + version note
Personal cross-chat preferences	Memory — review/edit entries in Settings
Confidential one-off chat	Incognito (excluded from memory/history/search)
Contexts must not bleed	Separate Projects (memory is project-scoped)
Free near five-project cap	Archive or merge
Wrong remembered fact	Edit/delete the memory entry
Shared Project misbehaving	Check owner, duplicates, staleness before model blame

Exam traps

1. Everything crammed into instructions (novel-length, self-contradicting)
2. Never updating knowledge; superseded PDFs left in place
3. Treating personal Memory as the team’s knowledge base
4. Assuming memory crosses Project boundaries (it’s scoped per Project)
5. Write-scope connectors by default for analysis-only work
6. Ignoring sharing permissions (org-wide edit on restricted knowledge)
7. Duplicate conflicting files causing retrieval confusion
8. Shared Project with no owner (guaranteed drift)

9. Styles doing a Project's legal-compliance job
 10. Forgetting incognito exists for sensitive one-offs
-

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A team needs standing tone rules plus a 40-page policy PDF applied to weekly work. Configuration?
A: Project: short tone rules in instructions; the PDF in knowledge, named with version and date; instructions point to it by filename.

Q2. A Free-plan consultant hits the project cap starting client #6. Options? **A:** Archive or merge an inactive project, or upgrade — Free caps at five (verified Aug 2026; re-check current caps at support.claude.com).

Q3. Select TWO knowledge-hygiene practices. **A:** Version dates in filenames; removing superseded documents immediately. (Duplicates and finalFINAL naming are the anti-patterns.)

Q4. Claude cites last quarter's refund window. The v4 policy shipped yesterday. Root cause and fix?
A: Stale knowledge — replace v3 with v4, announce the change, bump the instructions' version note. True-to-file, false-to-world is a maintenance failure.

Q5. When does Project RAG matter, and who gets it? **A:** Paid plans, when knowledge approaches context limits — retrieval expands capacity up to ~10x; help it with clear naming and narrow asks.

Q6. A user expects a new chat to know yesterday's project decisions automatically. What's the right configuration answer? **A:** Put decisions in Project knowledge (team truth) — Memory may recall some context (per-user, plan-dependent, project-scoped) but is not a reliable or shared store for load-bearing facts.

Q7. Analysis-only Drive workflow: which connector scope? **A:** Read-only / least privilege. Write scopes need governance approval and human gates.

Q8. Claude keeps addressing a manager by a project they left months ago. Fix? **A:** Settings → Memory: edit or delete the stale entry. Memory is user-curated configuration, not fate.

Q9. "Can view" vs "Can edit" on a shared Project — the difference? **A:** View = use the Project read-only; Edit = modify knowledge and instructions. Grant edit narrowly; sharing is a governance control.

Q10. Where do rich multi-step procedures belong: instructions or a Skill? **A:** A Skill — procedures load when relevant; instructions carry short standing rules. Duplicating a Skill inside instructions bloats context.

Q11. Two conflicting style guides live in one Project's knowledge. Symptom and fix? **A:** Inconsistent brand outputs from retrieval confusion — remove one or add an explicit priority rule in instructions.

Q12. An exec wants sensitive board prep with zero trace in memory or history. Recommendation?
A: Incognito chat (all plans, outside Projects) — excluded from history, memory, and search. For recurring board work, a dedicated Project with tight sharing beats repeated incognito.

Q13. Enterprise Search returns nothing for a source the team swears is connected. Checks? **A:** Admin actually enabled the connector; the asking user authenticated; the source is in scope. Config before model blame.

- Q14.** Who maintains a shared Project, and what artifact proves it? **A:** A named owner; a changelog note in knowledge recording file swaps and instruction versions.
- Q15.** A Free-plan user wants three custom MCP connectors. Reality? **A:** Custom connectors exist on Free but are limited to one; multiple custom connectors need a paid plan (verified Aug 2026).
- Q16.** Personal Styles vs Project instructions — when is Styles the wrong answer? **A:** Whenever the rule is workspace truth (legal constraints, team schemas): Styles are personal taste and don’t govern a shared Project.
- Q17.** Select TWO reasons memory being *project-scoped* matters to configuration. **A:** Client contexts can’t bleed between Projects; sensitive project discussions stay out of general-chat memory. (It’s also why “keep each client in its own Project” is the standing design rule.)
- Q18.** During teammate onboarding to a configured Project, what two steps prevent week-one chaos? **A:** Confirm their connector authentication, and run a supervised first task against the evaluation checklist.

Quick review checklist (pre-exam)

- ☐ Project setup: purpose, owner, sensitivity, instructions, knowledge, sharing
- ☐ Instructions: short, prioritized, conflict rules, “Not found” escalation, file pointers
- ☐ Knowledge: versioned names, no duplicates, replace superseded, manifest
- ☐ Upload vs connector; least privilege; admin-enable + user-auth
- ☐ Free caps: five projects, one custom connector; RAG paid ~10×
- ☐ Memory: plans, project-scoped, view/edit/delete/reset, independent toggles
- ☐ Incognito: all plans, no history/memory/search
- ☐ Maintenance cadence + changelog + named owner; archive dead Projects

Glossary

Term	Meaning
Project instructions	Standing behavioral rules scoped to a workspace
Project knowledge	Curated reference files; the team’s source of truth
RAG (Projects)	Paid retrieval expanding knowledge capacity near context limits
Connector	Live integration (e.g., Drive, Gmail); admin-enabled, user-authed
Least privilege	Minimum scope that does the job; read-only default
Memory	Per-user, project-scoped remembered context; curated in Settings
Memory entry	One editable/deletable remembered fact with chat citation
Incognito chat	Chat excluded from history, memory, and search
Knowledge hygiene	Naming, versioning, dedup, and pruning discipline

Term	Meaning
Configuration drift	Slow divergence of instructions/knowledge/memory from reality

Domain 06 — Governance, Risk & Responsible Use

Maps to official CCAO-F Governance, Risk, and Responsible Use (~15%, ~9 questions)

Note: This edition follows the ASD-STE100 Simplified Technical English writing rules. Simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Projects, Artifacts, Skills, Connectors, Research, Memory, PII, GDPR, MCP) are exceptions and stay as written.

Dedup note (2026-08-23): This is one primary-study copy. Earlier builds repeated the same drill blocks about 7 times. Those duplicates are gone. The content now matches the Domain 03 depth.

Disclaimer

These notes are original CCAO-F study notes. They are for people who are not developers. They use claude.ai (Projects, Artifacts, Skills, Connectors, Research). The notes use public Anthropic Usage Policy themes and privacy documentation. They also use public Claude Academy (AI Fluency Diligence) and the published CCAO-F blueprint. The notes are independent. They are not affiliated with Anthropic. Nothing here is legal advice. Check live policy and privacy details on support.claude.com and the Anthropic policy pages.

Overview

The official blueprint verbs are: **identify** appropriate and inappropriate use cases. **Apply** data sensitivity, regulatory, and privacy considerations. **Follow** organizational AI policies and governance standards. **Understand** the ethical implications of AI usage. You need practical judgment. You are not a lawyer. Know what to refuse. Know what needs safeguards. Know which data may go where. Know who owns deployed outputs. The official sample question rewards this action: **anonymize regulated identifiers before upload, consistent with policy**. That judgment pattern applies to many questions.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Use-case appropriateness	identify, refuse, escalate	Disallowed vs high-risk-with-safeguards vs ordinary
Data sensitivity & privacy	apply, classify, minimize	Data tiers. PII minimization. Venue choice

Official objective	Exam verbs	What you practice
Org policy compliance	follow, check, escalate	Policy owners. Shadow IT avoidance
Ethics	understand, disclose	Bias, transparency, ownership of outputs

Deep notes

1. The policy reasoning pattern (memorize the pattern, not clauses)

1. **Disallowed?** Refuse. Do not look for phrasing tricks. Universal categories include violent crimes, child exploitation, scams/fraud, malicious cyber operations, and serious privacy invasions.
2. **High-risk but allowed?** Legal, financial, employment, and medical-adjacent decisions affect people. Proceed **only with safeguards**: human-in-the-loop, disclosure, and evaluation gates.
3. **Ordinary business?** Still apply privacy minimization and org rules.
4. **Uncertain?** Escalate to the policy owner. Do not invent a legal answer.

On the exam, jailbreak-style options are always wrong. An example is “rephrase to get around the refusal.” A safety refusal on genuinely disallowed content is correct behavior. It is not a defect. The policy honors context. Defensive security work with the owner’s consent differs from malicious hacking. You can state benign intent plainly (Domain 07).

2. Data sensitivity in practice

Classify data before you paste it: **public / internal / confidential / restricted**. Then:

- **Minimize:** Share the minimum that the task needs. Anonymize or redact regulated identifiers (names, account numbers) before upload when policy restricts them. This is the official sample answer.
- **Venue:** Personal Free accounts are usually the wrong venue for company confidential data. Managed Team/Enterprise workspaces exist to carry organizational controls.
- **Training/retention defaults:** Consumer offerings and commercial offerings differ. Anthropic typically does **not** train on commercial/organizational data by default. Check current terms in the privacy docs. Do not repeat unverified claims. “Instruct Claude not to retain it” is **not** a policy control. That line is a sample-question distractor.
- **Memory & retention surface (new):** Claude’s Memory stores context from chats. Treat memory entries as data. Put sensitive single-use questions in **incognito chats**. Exclude Incognito chats from history, memory, and search. On Enterprise, the org still retains incognito chats for a minimum safety period. Admin data exports include them. Incognito is a *memory/history* control. It is not a way around org visibility. Data exports include Memory. Review and delete entries per policy (see 05–... §Memory).

3. Connector and sharing governance

Connectors expand the range of harm. Before you connect Gmail, Drive, or Slack, know what Claude can **read**. Know whether **writes** are possible. Prefer read-only for analysis. Write scopes (send email, update HRIS/CRM) are governance decisions. They need approval and human gates. Enterprise-managed auth and domain restrictions may apply. Project sharing is itself a control: “Can view” vs

“Can edit”. Do not share restricted knowledge org-wide. Shared Artifacts can expose data. Read the confirmation dialogs. Free/personal accounts are the wrong venue for company secrets.

4. Transparency, disclosure, and ownership (Diligence)

The Diligence triad is **Creation** (quality of AI-assisted work), **Transparency** (honesty about AI’s role), and **Deployment** (you own what you release). Disclose AI assistance when policy, high-risk context, or customer-facing use requires it. Customer-facing chatbots disclose that they are AI. “Claude said it is fine” is never a compliance defense. The deployer owns the output. Fabricated compliance claims are a violation. An example is “this is GDPR-approved.”

5. Ethics scenarios that recur

- **Hiring/employment assist:** Ranking of candidates needs bias scrutiny, a human decision, and often disclosure. Fully automated hire/reject is the canonical inappropriate use.
- **Finance/legal/medical-sounding advice to customers:** Licensed-human review plus disclaimers.
- **Bias:** Stereotype defaults, skewed examples, and one-sided analyses. Evaluate impacts on people. Require counterarguments and criteria (Domain 03 methods that serve Domain 06 goals).
- **Synthetic media/deception:** Non-consensual deepfakes and impersonation are disallowed.
- **Copyright/confidentiality:** Do not paste third-party secrets or bulk copyrighted text into unmanaged contexts.
- **Accessibility & inclusion:** Examples and outputs must not leave people out without notice.

6. Following org policy without becoming shadow IT

Help teams adopt Claude **inside policy**. Publish allowed tools and venues. Publish data tiers, review requirements, example prompts, and escalation paths. Training works better than warnings that cause fear. Metrics must count policy incidents, not only speed wins. When org policy and personal convenience conflict, policy wins on the exam every time. When policy is silent, escalate to its owner. Do not invent a rule. Report harmful or broken outputs through product channels when the case warrants it.

7. Mini-cases (judgment reps)

Case 1 — The helpful shortcut. An ops manager pastes the full customer database extract into chat “to find churn patterns.” The extract includes names, emails, and account balances. Policy classifies this data as restricted. Correct path: aggregate or anonymize first. Drop direct identifiers. Keep behavioral columns. Run the work in the managed workspace. Involve the data owner. The analysis is legitimate. The handling is not.

Case 2 — The quiet chatbot. A team ships a Claude-backed FAQ widget. There is no AI disclosure. There is no human escalation path. Two failures exist. Transparency: customer-facing AI discloses. Deployment ownership: there is no route for wrong answers to reach humans. Fix this before you scale. Do not wait for the first complaint.

Case 3 — The eager connector. An admin grants a write-scope HRIS connector so Claude can “tidy records.” There is no governance review. There is no gate. Even with good intent, this is the range-of-harm trap. Writes of people-data need the system owner’s approval. They need least privilege.

They need human confirmation per change. Read-only reporting delivers 90% of the value at 10% of the risk.

Case 4 — The confident compliance memo. Claude drafts “our data flows are GDPR-compliant” for a client email. Nobody with authority reviews it. Fabricated compliance claims are a governance violation even when they sound plausible. Regulated assertions ship only with the accountable owner’s sign-off (the Domain 03 L4 gate that serves Domain 06).

Case 5 — The screening spreadsheet. A recruiter asks Claude to score 400 resumes and auto-email rejections to the bottom half. Split the work. Claude may structure and summarize applications. Ranking feeds a human decision with bias checks. Rejection *decisions and sends* stay human. This is high-risk-with-safeguards, split correctly.

8. Multiple-response pattern bank

Wrong pairs that recur:

- “Rephrase until it answers” + “report the refusal as an outage” (jailbreak + refusal-as-bug).
- “Upload as-is, it is internal” + “never analyze customer data at all” (policy breach + paralysis). Anonymization is the middle path.
- “Claude reviewed it, ship it” + “humans must draft everything” (fake sign-off + anti-Delegation).
- “Ban connectors” + “connect with write access everywhere” (absolutes).

Correct combinations pair a **data-handling** move (classify, minimize, right venue) with an **oversight** move (human gate, disclosure, escalation to the policy owner).

Decision trees

Situation	Action
Disallowed harmful request	Refuse. Do not rephrase to bypass the refusal.
High-risk decisioning (employment/finance/legal)	Human oversight + disclosure + evaluation gates
Regulated identifiers in a file to analyze	Anonymize or redact first, per policy
Confidential data, personal Free account	Wrong venue. Use the managed workspace.
Customer-facing AI	Disclosure norms apply
Sensitive one-off question	Incognito chat. On Enterprise, admin export/retention still applies.
Write-scope connector on a sensitive system	Governance approval + human gate
Uncertain legality/policy	Escalate to the policy owner
Safety refusal on allowed work	Clarify benign intent within policy (Domain 07). Never jailbreak.

Exam traps

1. “Claude said it is fine” as legal/compliance cover
2. You paste restricted data into a personal account against policy
3. “Instruct Claude not to retain it” as a privacy control

4. Jailbreak options presented as cleverness
 5. You skip disclosure on customer-facing or high-risk use
 6. You treat safety refusals as bugs to fix
 7. Fully automated employment/credit decisions
 8. Write connectors on sensitive systems without approval
 9. You believe incognito hides activity from Enterprise governance (it is a memory/history control)
 10. Fabricated compliance claims (“certified GDPR-safe output”)
-

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A user asks Claude for malware to “test” a competitor’s network. Response pattern? **A:** Refuse — malicious cyber operations are disallowed. There is no consent and no authorization. Defensive work with the system owner’s consent is the different, legitimate case.

Q2. A PM wants to upload a spreadsheet with customer names and account numbers. Policy restricts regulated personal data. Most appropriate action? **A:** Remove or anonymize the identifiers before upload, consistent with policy. The analysis proceeds without exposure of protected data (official sample answer pattern).

Q3. Select TWO valid privacy moves for routine work. **A:** Minimize PII to what the task needs. Follow the org’s data classification for venue and handling. (“Tell Claude to forget it” is not a control.)

Q4. HR wants Claude to auto-reject bottom-scored resumes with no human step. Verdict? **A:** Inappropriate. Employment decisions are high-risk. They need human oversight, bias scrutiny, and disclosure. Claude may assist (summarize, structure). Claude may not decide.

Q5. A customer-facing support bot built on Claude — what governance requirement applies to the interface? **A:** Disclose that customers talk to AI. Keep escalation to humans available. Transparency norms apply to customer-facing use.

Q6. An analyst uses their personal Free account for board-confidential financials “to save time.” Problem? **A:** Wrong venue. Personal accounts lack org controls. Policy on confidential data likely prohibits this. Use the managed workspace. Convenience never outranks classification.

Q7. Does commercial/organizational Claude usage train the model on your data by default? **A:** Typically not by default for commercial offerings. Check current privacy documentation. Do not recite folklore. “Verify, then rely” is the exam-safe posture.

Q8. Ops wants a connector that can write to the HRIS to “save clicks.” Governance shape? **A:** Approval by the system’s governance owner. Least-privilege scope. A human gate before writes. Reads for analysis are the easier grant. Writes move people-data.

Q9. A hiring-assist workflow shows the model favoring one university in rankings. Required response? **A:** Bias scrutiny. Test with counterfactual candidates. Add criteria. Keep humans deciding. Document the check. If you ship it without a check, you fail ethics and possibly law.

Q10. Claude refuses a request that seems clearly legitimate. Wrong answer vs right answer? **A:** Wrong: jailbreak or rephrase-to-trick. Right: state the benign, authorized context plainly within policy. Escalate if it persists (Domain 07).

Q11. When must AI assistance be disclosed in deliverables? **A:** When org policy, high-risk context, or customer-facing use requires it. Transparency is one third of Diligence. The deployer owns the call.

Q12. A teammate asks Claude whether their data-sharing plan is legal in the EU and acts on the answer. Failure? **A:** The teammate treats Claude as counsel. Escalate uncertain legality to the compliance/policy owner. Claude can draft the question well. Claude cannot own the answer.

Q13. Select TWO facts about incognito chats relevant to governance. **A:** Exclude Them from history, memory, and search. On Enterprise the org still retains them for a minimum safety period. Data exports include them. They are not invisible to the org.

Q14. Someone shares a Project holding restricted contracts org-wide with edit rights. Assessment? **A:** This is a permission failure two times. Restricted knowledge goes to an org-wide audience. The share grants edit rights, not view. Sharing scope is a governance control.

Q15. Marketing wants Claude to fabricate “9 out of 10 dentists” style stats for copy. Call? **A:** Refuse the fabrication. Deceptive claims are inappropriate use. Supply real sourced figures or drop the claim (Domain 03 enforcement of a Domain 06 rule).

Q16. A near-miss: an associate almost emailed AI-drafted financial advice to a client unreviewed. What controls were missing? **A:** High-risk safeguards: licensed-human review, disclosure, and a human gate on the send. High-risk allowed does not equal high-risk unsupervised.

Q17. What belongs in a team’s responsible-use enablement pack? **A:** Allowed tools/venues. Data tiers with examples. Review requirements by risk. Example prompts. Escalation paths. Adoption **inside policy** beats **Shadow IT**.

Q18. Select TWO cases that are disallowed outright (refuse. Do not safeguard). **A:** Non-consensual deepfake/impersonation content. Instructions that enable scams or malicious cyber operations. (Employment decisioning is high-risk-with-safeguards, not refuse-outright.)

Quick review checklist (pre-exam)

- ☐ Policy pattern: disallowed → refuse. High-risk → safeguards. Ordinary → minimize. Uncertain → escalate
 - ☐ Data tiers + anonymize regulated identifiers before upload
 - ☐ Venue discipline: personal Free ≠ confidential work
 - ☐ Training/retention: commercial typically not by default — verify current docs
 - ☐ Incognito = memory/history control, not governance invisibility
 - ☐ Connectors: read-only default. Writes = approval + human gate
 - ☐ Disclosure for customer-facing and high-risk use. Deployer owns outputs
 - ☐ Refusals on disallowed content are correct behavior
-

Glossary

Term	Meaning
Usage Policy	Anthropic’s rules: universal disallowed categories + high-risk requirements
High-risk use case	Allowed use that affects people’s rights, money, or health — needs safeguards

Term	Meaning
Human-in-the-loop	Required human review/decision inside the workflow
Data classification	Public / internal / confidential / restricted handling tiers
PII minimization	Share the least personal data the task needs. Redact or anonymize
Disclosure	Be honest about AI's role where it matters
Diligence triad	Creation, Transparency, Deployment — you own what you ship
Shadow IT	Unsanctioned tool use around policy. Fix with enablement, not fear
Constitutional AI	High-level: principles-based training that guides safer model behavior

Domain 06 — Governance, Risk & Responsible Use

Maps to official CCAO-F Governance, Risk, and Responsible Use (~15%, ~9 questions)

Dedup note (2026-08-23): Rewritten as a single primary-study copy. Earlier builds repeated the same drill blocks ~7x; duplicates removed and content deepened to the Domain 03 standard.

Disclaimer

Original CCAO-F study notes for non-developers using claude.ai (Projects, Artifacts, Skills, Connectors, Research). Grounded in public Anthropic Usage Policy themes, privacy documentation, public Claude Academy (AI Fluency Diligence), and the published CCAO-F blueprint. Independent; not affiliated with Anthropic; nothing here is legal advice. Verify live policy and privacy details on support.claude.com and the Anthropic policy pages.

Overview

Official blueprint verbs: **identify** appropriate and inappropriate use cases; **apply** data sensitivity, regulatory, and privacy considerations; **follow** organizational AI policies and governance standards; **understand** the ethical implications of AI usage. Non-lawyer practical judgment for associates: know what to refuse, what needs safeguards, what data may go where, and who owns deployed outputs. The official sample question for this domain rewards *anonymizing regulated identifiers before upload, consistent with policy* — that judgment pattern generalizes.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Use-case appropriateness	identify, refuse, escalate	Disallowed vs high-risk-with-safeguards vs ordinary
Data sensitivity & privacy	apply, classify, minimize	Data tiers; PII minimization; venue choice
Org policy compliance	follow, check, escalate	Policy owners; shadow-IT avoidance
Ethics	understand, disclose	Bias, transparency, ownership of outputs

Deep notes

1. The policy reasoning pattern (memorize the pattern, not clauses)

1. **Disallowed?** → refuse; do not look for phrasing tricks. Universal categories include violent crimes, child exploitation, scams/fraud, malicious cyber operations, and serious privacy invasions.
2. **High-risk but allowed?** (legal, financial, employment, medical-adjacent decisions affecting people) → proceed **only with safeguards**: human-in-the-loop, disclosure, evaluation gates.
3. **Ordinary business?** → still apply privacy minimization and org rules.
4. **Uncertain?** → escalate to the policy owner; don't improvise legality.

On the exam, jailbreak-flavored options (“rephrase to get around the refusal”) are always wrong; a safety refusal on genuinely disallowed content is correct behavior, not a defect. Nuance the policy does honor: context matters — defensive security work with the owner’s consent differs from malicious hacking; benign intent can be stated plainly (Domain 07).

2. Data sensitivity in practice

Classify before pasting: **public / internal / confidential / restricted**. Then:

- **Minimize**: share the minimum needed; anonymize or redact regulated identifiers (names, account numbers) before upload when policy restricts them — the official sample answer.
- **Venue**: personal Free accounts are usually the wrong venue for company confidential data; managed Team/Enterprise workspaces exist precisely to carry organizational controls.
- **Training/retention defaults**: consumer vs commercial offerings differ; commercial/organizational offerings are typically **not** trained on by default — verify current terms in the privacy docs rather than reciting folklore. “Instruct Claude not to retain it” is **not** a policy control (sample question distractor).
- **Memory & retention surface (new)**: Claude’s Memory stores context from chats — treat memory entries as data. Sensitive one-offs belong in **incognito chats** (excluded from history, memory, search); on Enterprise, incognito chats are still retained for a minimum safety period and appear in admin data exports — incognito is a *memory/history* control, not a way around org visibility. Memory is included in data exports; review and delete entries per policy (see 05—... §Memory).

3. Connector and sharing governance

Connectors expand the blast radius: before connecting Gmail/Drive/Slack, know what Claude can **read** and whether **writes** are possible. Prefer read-only for analysis; write scopes (send email, update HRIS/CRM) are governance decisions requiring approval and human gates. Enterprise-managed auth and domain restrictions may apply. Project sharing is itself a control: “Can view” vs “Can edit”; avoid org-wide sharing of restricted knowledge; shared Artifacts can expose data — read the confirmation dialogs. Free/personal accounts are the wrong venue for company secrets.

4. Transparency, disclosure, and ownership (Diligence)

The Diligence triad: **Creation** (quality of AI-assisted work), **Transparency** (honesty about AI’s role), **Deployment** (you own what you release). Disclose AI assistance when policy, high-risk context, or customer-facing use requires it — customer-facing chatbots disclose that they’re AI. “Claude said it’s fine” is never a compliance defense; the deployer owns the output. Fabricating compliance claims (“this is GDPR-approved”) is itself a violation.

5. Ethics scenarios that recur

- **Hiring/employment assist:** ranking candidates demands bias scrutiny, human decision, and often disclosure — fully automated hire/reject is the canonical inappropriate use.
- **Finance/legal/medical-sounding advice to customers:** licensed-human review + disclaimers.
- **Bias:** stereotype defaults, skewed examples, one-sided analyses — evaluate impacts on people, require counterarguments and criteria (Domain 03 machinery serving Domain 06 goals).
- **Synthetic media/deception:** non-consensual deepfakes and impersonation are disallowed territory.
- **Copyright/confidentiality:** don’t paste third-party secrets or bulk copyrighted text into unmanaged contexts.
- **Accessibility & inclusion:** examples and outputs shouldn’t quietly exclude.

6. Following org policy without becoming shadow IT

Help teams adopt Claude inside the lines: publish allowed tools and venues, data tiers, review requirements, example prompts, escalation paths. Training beats scare posters. Metrics should count policy incidents, not only speed wins. When org policy and personal convenience conflict, policy wins on the exam every time; when policy is silent, escalate to its owner rather than improvising. Report harmful or broken outputs through product channels when warranted.

7. Mini-cases (judgment reps)

Case 1 — The helpful shortcut. An ops manager pastes the full customer database extract (names, emails, account balances) into chat “to find churn patterns.” Policy classifies this restricted. Correct path: aggregate or anonymize first (drop direct identifiers, keep behavioral columns), run in the managed workspace, and involve the data owner. The analysis was legitimate; the handling wasn’t.

Case 2 — The quiet chatbot. A team ships a Claude-backed FAQ widget with no AI disclosure and no human escalation path. Two failures: transparency (customer-facing AI discloses) and deployment ownership (no route for wrong answers to reach humans). Fix before scale, not after the first complaint.

Case 3 — The eager connector. An admin grants a write-scope HRIS connector so Claude can “tidy records.” No governance review, no gate. Even with good intent, this is the blast-radius trap:

people-data writes need the system owner’s approval, least privilege, and human confirmation per change. Read-only reporting would have delivered 90% of the value at 10% of the risk.

Case 4 — The confident compliance memo. Claude drafts “our data flows are GDPR-compliant” for a client email. Nobody with authority reviewed it. Fabricated compliance claims are a governance violation even when plausible; regulated assertions ship only with the accountable owner’s sign-off (Domain 03 L4 gate serving Domain 06).

Case 5 — The screening spreadsheet. A recruiter asks Claude to score 400 resumes and auto-email rejections to the bottom half. Split it: Claude may structure and summarize applications; ranking feeds a human decision with bias checks; rejection *decisions and sends* stay human. High-risk-with-safeguards, decomposed correctly.

8. Multiple-response pattern bank

Recurring wrong pairs: “rephrase until it answers” + “report the refusal as an outage” (jailbreak + refusal-as-bug); “upload as-is, it’s internal” + “never analyze customer data at all” (policy breach + paralysis — anonymization is the middle); “Claude reviewed it, ship it” + “humans must draft everything” (fake sign-off + anti-Delegation); “ban connectors” + “connect with write access everywhere” (absolutes). Correct combinations pair a **data-handling** move (classify, minimize, right venue) with an **oversight** move (human gate, disclosure, escalation to the policy owner).

Decision trees

Situation	Action
Disallowed harmful request	Refuse; no rephrasing games
High-risk decisioning (employment/finance/legal)	Human oversight + disclosure + evaluation gates
Regulated identifiers in a file to analyze	Anonymize/redact first, per policy
Confidential data, personal Free account	Wrong venue — use the managed workspace
Customer-facing AI	Disclosure norms apply
Sensitive one-off question	Incognito chat; on Enterprise, remember admin export/retention still applies
Write-scope connector on a sensitive system	Governance approval + human gate
Uncertain legality/policy	Escalate to policy owner
Safety refusal on allowed work	Clarify benign intent within policy (Domain 07); never jailbreak

Exam traps

1. “Claude said it’s fine” as legal/compliance cover
2. Pasting restricted data into a personal account against policy
3. “Instruct Claude not to retain it” as a privacy control
4. Jailbreak options dressed as resourcefulness
5. Skipping disclosure on customer-facing or high-risk use
6. Treating safety refusals as bugs to fix

7. Fully automated employment/credit decisions
 8. Write connectors on sensitive systems without approval
 9. Believing incognito hides activity from Enterprise governance (it's a memory/history control)
 10. Fabricated compliance claims ("certified GDPR-safe output")
-

Practice Q&A (18) — scenario stems with answers and rationales

Q1. A user asks Claude for malware to “test” a competitor’s network. Response pattern? **A:** Refuse — malicious cyber operations are disallowed; no consent, no authorization. Defensive work with the system owner’s consent is the different, legitimate case.

Q2. A PM wants to upload a spreadsheet with customer names and account numbers; policy restricts regulated personal data. Most appropriate action? **A:** Remove or anonymize the identifiers before upload, consistent with policy — the analysis proceeds without exposing protected data (official sample answer pattern).

Q3. Select TWO valid privacy moves for routine work. **A:** Minimize PII to what the task needs; follow the org’s data classification for venue and handling. (“Tell Claude to forget it” is not a control.)

Q4. HR wants Claude to auto-reject bottom-scored resumes with no human step. Verdict? **A:** Inappropriate — employment decisions are high-risk and need human oversight, bias scrutiny, and disclosure. Claude may assist (summarize, structure), not decide.

Q5. A customer-facing support bot built on Claude — what governance requirement applies to the interface? **A:** Disclose that customers are talking to AI, with escalation to humans available — transparency norms for customer-facing use.

Q6. An analyst uses their personal Free account for board-confidential financials “to move fast.” Problem? **A:** Wrong venue: personal accounts lack org controls; policy on confidential data likely prohibits it. Use the managed workspace — convenience never outranks classification.

Q7. Does commercial/organizational Claude usage train the model on your data by default? **A:** Typically not by default for commercial offerings — but verify current privacy documentation rather than reciting folklore; “verify, then rely” is the exam-safe posture.

Q8. Ops wants a connector that can write to the HRIS to “save clicks.” Governance shape? **A:** Approval by the system’s governance owner, least-privilege scope, and a human gate before writes. Reads for analysis are the easier grant; writes move people-data.

Q9. A hiring-assist workflow shows the model favoring one university in rankings. Required response? **A:** Bias scrutiny: test with counterfactual candidates, add criteria, keep humans deciding, document the check. Shipping it unexamined fails ethics and possibly law.

Q10. Claude refuses a request that seems clearly legitimate. Wrong answer vs right answer? **A:** Wrong: jailbreak/rephrase-to-trick. Right: state the benign, authorized context plainly within policy; escalate if it persists (Domain 07).

Q11. When must AI assistance be disclosed in deliverables? **A:** When org policy, high-risk context, or customer-facing use requires it — Transparency is one third of Diligence, and the deployer owns the call.

Q12. A teammate asks Claude whether their data-sharing plan is legal in the EU and acts on the answer. Failure? **A:** Treating Claude as counsel — uncertain legality escalates to the compliance/policy

owner. Claude can draft the question well; it cannot own the answer.

Q13. Select TWO facts about incognito chats relevant to governance. **A:** They’re excluded from history, memory, and search; on Enterprise they’re still retained for a minimum safety period and included in data exports — not invisible to the org.

Q14. Someone shares a Project holding restricted contracts org-wide with edit rights. Assessment? **A:** Permission failure twice over: restricted knowledge to an org-wide audience, and edit (not view) rights. Sharing scope is a governance control.

Q15. Marketing wants Claude to fabricate “9 out of 10 dentists” style stats for copy. Call? **A:** Refuse the fabrication — deceptive claims are inappropriate use; supply real sourced figures or drop the claim (Domain 03 enforcement of a Domain 06 rule).

Q16. A near-miss: an associate almost emailed AI-drafted financial advice to a client unreviewed. What controls were missing? **A:** High-risk safeguards: licensed-human review, disclosure, and a human gate on the send. High-risk allowed ≠ high-risk unsupervised.

Q17. What belongs in a team’s responsible-use enablement pack? **A:** Allowed tools/venues, data tiers with examples, review requirements by risk, example prompts, escalation paths — adoption inside the lines beats shadow IT.

Q18. Select TWO cases that are disallowed outright (refuse, don’t safeguard). **A:** Non-consensual deep-fake/impersonation content; instructions enabling scams or malicious cyber operations. (Employment decisioning is high-risk-with-safeguards, not refuse-outright.)

Quick review checklist (pre-exam)

- ☐ Policy pattern: disallowed → refuse; high-risk → safeguards; ordinary → minimize; uncertain → escalate
 - ☐ Data tiers + anonymize regulated identifiers before upload
 - ☐ Venue discipline: personal Free ≠ confidential work
 - ☐ Training/retention: commercial typically not by default — verify current docs
 - ☐ Incognito = memory/history control, not governance invisibility
 - ☐ Connectors: read-only default; writes = approval + human gate
 - ☐ Disclosure for customer-facing and high-risk use; deployer owns outputs
 - ☐ Refusals on disallowed content are correct behavior
-

Glossary

Term	Meaning
Usage Policy	Anthropic’s rules: universal disallowed categories + high-risk requirements
High-risk use case	Allowed use affecting people’s rights/money/health — needs safeguards
Human-in-the-loop	Required human review/decision inside the workflow

Term	Meaning
Data classification	Public / internal / confidential / restricted handling tiers
PII minimization	Sharing the least personal data the task needs; redact/anonymize
Disclosure	Being honest about AI's role where it matters
Diligence triad	Creation, Transparency, Deployment — you own what you ship
Shadow IT	Unsanctioned tool use around policy; fix with enablement, not fear
Constitutional AI	High-level: principles-based training guiding safer model behavior

Domain 07 — Troubleshooting & Optimization

Maps to official CCAO-F Troubleshooting and Optimization (~10%, ~6 questions)

Note: This edition follows the ASD-STE100 Simplified Technical English writing rules. Simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Projects, Artifacts, Skills, Connectors, Research, Memory, effort) are exceptions and stay as written.

Dedup note (2026-08-23): This file is one primary-study copy. Earlier builds repeated the same drill blocks about 7 times. Those duplicates are gone. The content now matches the Domain 03 depth.

Disclaimer

These notes are original CCAO-F study notes. They are for people who are not developers. They use claude.ai (Projects, Artifacts, Skills, Connectors, Research). The notes use public Anthropic Help Center and product docs. They also use public Claude Academy (Claude 101, AI Fluency 4D) and the published CCAO-F blueprint. The notes are independent. They are not affiliated with Anthropic. Check live product details on support.claude.com.

Overview

Official blueprint verbs: **identify, diagnose, and resolve** issues with underperforming prompts or poor outputs. **Adjust** the approach based on feedback and results. **Optimize** workflows for efficiency and effectiveness. This is the integration domain. Decide whether the fault is in the platform, inputs, configuration, prompt, context, model, evaluation, or governance. Then fix the right layer. Do not waste usage. Most stems reward the cheapest correct fix. They do not reward the most dramatic one.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Diagnose poor outputs	identify, isolate	The diagnostic ladder. Layer naming
Adjust from feedback	adjust, adapt	Miss taxonomy → targeted system updates
Optimize workflows	optimize, streamline	Levers: model, effort, feature, context, structure

Deep notes

1. The diagnostic ladder (run it in order)

1. **Success criteria** — was the request vague? Maybe nothing is broken. The brief is the problem (Domain 02).
2. **Inputs** — wrong or missing files, unauthenticated connector, web search off, OCR-junk scans, capability toggles.
3. **Configuration** — stale or conflicting knowledge. Contradictory instructions. Wrong Project. Stale **Memory entry** that feeds wrong context (edit or delete it — see 05–... §Memory).
4. **Prompt** — Description gaps: no criteria, no schema, mixed task types.
5. **Context** — thread drift. Summarize and restart.
6. **Model/effort** — under-powered or over-powered tier. Tune effort before you change tiers.
7. **Evaluation gates** — quality “problems” that are really shipped-without-review problems (Domain 03).
8. **Limits/plan** — usage limits, Free caps, feature availability.

Discipline: change **one variable at a time**. Record what fixed it so it becomes a pattern. Do not treat a lucky rerun as a pattern. Two failed tightened prompt iterations mean the wrong layer. Move up or down the ladder.

2. Worked cases

- **“Claude gives the wrong refund answer.”** Which policy file is in the Project? Ask it to quote the section. If the file is stale, update knowledge. If the file is right but ignored, add instructions: “Prefer Refund-Policy-v4. If not in knowledge, say Not found.” Only then consider model escalation.
- **“Research is useless.”** Is web search on? Is the plan paid? Is the question scoped? Are connectors authenticated if you expect internal data? Narrow the scope before you blame the feature.
- **“The numbers in the summary are wrong.”** Large-table arithmetic in plain chat is the wrong tool. Rerun through **code execution** so the math is computed, not pattern-matched. Then compare against the source.
- **“Claude keeps bringing up my old project.”** Stale memory entry. Go to Settings → Memory. Edit or delete the entry. Use separate Projects to scope context. Use incognito for single-use questions.
- **“The team says Claude is random.”** Shared Project without an owner. Conflicting guides. No schema. This is a governance and configuration fix. It is not a bigger model.

- **“Claude refused my legitimate request.”** State the benign, authorized context plainly. Rephrase within policy. Refusals on disallowed content are correct behavior. Jailbreak options are always exam-wrong (Domain 06).

3. Optimization levers (effectiveness × efficiency)

Model tier. Effort. Feature choice (Research vs chat vs code execution). Context size. Project-persisted context vs re-pasting. Skill reuse. Schemas to cut revision cycles. Batching similar tasks. Pruning noisy knowledge. Capability toggles. Connector scope. Optimize **both** quality and cost/time. If you cut review on L4 work, that is false efficiency. If you run Opus on ticket triage, that is false quality.

Scoreboard: median revisions-to-ship. % factual misses. Usage per workflow. Time-to-first-draft. Improve the worst metric with the **cheapest lever**. That phrase decides exam options.

4. Feedback → system updates (miss taxonomy)

Tag every miss. Then fix the system that caused it. **Factual** → knowledge, citations, or code execution for math. **Format** → instructions or schema. **Tone** → examples in Project or Skill. **Policy** → governance plus instructions. **Incomplete** → checklist or decomposition. **Auth/access** → IT enablement. **Wrong remembered context** → memory curation. When you fix the causing system, you practice Diligence. The anti-pattern is coaching users so they compensate for a broken configuration.

5. Bridge summary across all domains

01 pick tools and models. 02 describe tasks. 03 validate outputs. 04 design workflows. 05 configure knowledge and memory. 06 govern risk. 07 diagnose and optimize. Real failures are usually multi-domain. Name the **primary** layer in your answer. Fix it. Then measure again.

6. 90-second diagnosis drills

Work each in about 90 seconds: name the layer, then the cheapest fix. Answers follow.

- **Drill A:** “Claude did our weekly report well before. Since Marta added her files, it mixes two template styles.” → Configuration: conflicting templates in knowledge. Remove one, or set priority in instructions.
- **Drill B:** “Research mode grays out for one analyst.” → Platform/plan: their seat, plan, or capability toggle. This is not a prompt problem.
- **Drill C:** “The extraction is perfect on 10 rows and fails on 5,000.” → Tool/context: move from chat-window pasting to an uploaded file plus code execution.
- **Drill D:** “Claude answered from the March price list. April’s is uploaded too.” → Knowledge hygiene: a superseded file is still present. Purge it.
- **Drill E:** “Every chat, Claude assumes I am still on the Berlin project.” → Memory: stale entry. Edit or delete it in Settings → Memory.
- **Drill F:** “Long thread: it now ignores the banned-words list from message 3.” → Context drift: summarize and restart, with rules persisted in the Project.
- **Drill G:** “Quality is fine but we use up the day’s limit by 2 p.m.” → Optimization: lower the tier or effort on routine tasks. Batch. Persist context instead of re-pasting.
- **Drill H:** “It refuses to summarize our pen-test report.” → Governance-aware rephrase: state the authorized defensive context. Never jailbreak.

The core skill: symptom → layer → cheapest fix, in one sentence. If your explanation needs a paragraph, you probably have not found the layer.

7. Multiple-response pattern bank

Recurring wrong pairs: “switch to the top model” plus “give up on the task” (status plus giving up — the ladder is the middle). Also “add three more paragraphs of instructions” plus “delete all instructions” (word count as a cure). Also “remove the review step to speed up” plus “review every trivial output” (false efficiency plus disproportion). Also “restart every chat hourly” plus “never restart” (ritual plus sunk cost). Correct combinations pair a **diagnostic** move with a **structural** fix. Diagnostic moves: check auth and toggles, quote-the-file test, isolate one variable. Structural fixes: schema, knowledge purge, memory edit, correct tier size.

Decision trees

Symptom	Likely layer → action
Wrong facts “from” files	Knowledge/pointing → name the file, purge superseded versions
Ignores required format	Prompt → schema/Performance constraints
Great early, degrades late in thread	Context → summarize + restart
Wrong arithmetic on data	Tool → code execution, verify vs source
Slow/expensive workflow	Optimization → lighter model/effort, less Research, templates
Feature “does not work”	Inputs/platform → plan, capability toggle, auth, web search
Wrong personal context recalled	Memory → edit or delete the entry. Project scoping
Refusal on allowed work	Governance-aware rephrase. Never jailbreak
Chronic misses across users	Workflow/config redesign — not a bigger model
Errors reaching stakeholders	Evaluation gates missing → restore Domain 03 SOP

Exam traps

1. “Escalate to the model with the most capability” as the universal fix
2. You ignore auth, web search, and capability toggles before deep diagnosis
3. You fix symptoms with ever-longer prompts (wrong layer)
4. “Optimizing” by removing human review from high-risk steps
5. You treat safety refusals as outages
6. You never restart drifted threads (sunk-cost threading)
7. You keep conflicting knowledge versions
8. You change five variables at once and learn nothing
9. You blame the model for a stale memory entry or a stale PDF
10. You optimize an unmeasured workflow

Practice Q&A (16) — scenario stems with answers and rationales

- Q1.** Claude answers refund questions from general knowledge despite an uploaded policy. First two moves? **A:** Name the file in the prompt and require quoting. Check knowledge for superseded or duplicate versions. Pointing plus hygiene is better than asking again with more force.
- Q2.** Select TWO optimizations for a high-volume extraction workflow burning limits. **A:** Drop to Haiku (or lower effort). Persist the schema and rules in a Project instead of re-pasting. (Removing review is the trap option.)
- Q3.** Research returns nothing. Fastest diagnostic? **A:** Check that web search is enabled and that the plan supports Research. Capability toggles come before feature blame.
- Q4.** A 200-message thread now contradicts itself. Fix? **A:** Write a handoff summary (decisions, constraints, open items). Start a fresh chat. Replace drifted context. Do not argue with it.
- Q5.** Monthly variance memo has subtle arithmetic errors. Layer and fix? **A:** Tool layer: route the math through code execution on the uploaded data. Then verify totals against the source. Chat prose is not a calculator.
- Q6.** Outputs ignore the required five-section format intermittently. Fix? **A:** Put a hard schema in Project instructions (sections, order, “omit nothing. Write N/A if empty”). Format misses are Performance-constraint gaps, not model failures.
- Q7.** Drive connector “sees nothing” for one user but works for others. Diagnosis? **A:** That user has not authenticated the connector. Admin enablement is org-wide. Auth is per-user.
- Q8.** Claude keeps referencing a client the associate stopped serving last year. Fix? **A:** Memory curation: delete or edit the stale entry in Settings → Memory. Keep current clients in their own Projects so scoping does the work.
- Q9.** Chronic tone misses across the whole team’s outputs. Cheapest durable fix? **A:** Add gold tone examples (and a counterexample) to the shared Project or Skill. One config change fixes every user. Per-chat coaching does not.
- Q10.** Select TWO checks *before* concluding “the model is too weak” on an analysis task. **A:** Was a rubric or criteria provided (prompt layer)? Is the knowledge current and conflict-free (config layer)? Escalate the tier only after both pass.
- Q11.** The team hits usage limits mid-afternoon. Select TWO adjustments. **A:** Route routine tasks to lighter models or lower effort. Cut Research use where a quoted file or a single search is enough. (Also batch similar tasks. An upgrade is a business call, not a reflex.)
- Q12.** A stakeholder demands “faster outputs — just buy the top model.” Better lever set? **A:** Templates and Skills for recurring structure, persisted Project context, and a right-sized model per task. Speed usually lives in workflow structure, not in tier.
- Q13.** Claude refuses to analyze the company’s own phishing simulation results. Move? **A:** Restate the authorized, defensive context plainly and rephrase within policy. Legitimate security work with consent is fine to clarify. Jailbreak framing is always wrong on the exam.
- Q14.** Two conflicting SOP versions live in knowledge. Answers alternate between them. Fix? **A:** Delete the superseded version (or add explicit priority in instructions). Retrieval confusion is configuration debt.

Q15. How do you prove an optimization worked? **A:** Before/after on the scoreboard: revisions-to-ship, factual-miss %, usage per workflow, time-to-first-draft. Unmeasured optimization is a story, not proof.

Q16. After an incident, someone proposes removing the human review step to “streamline.” Verdict? **A:** Reject: review on consequential outputs is effectiveness, not overhead. Restore the gate and optimize elsewhere (lighter model, schema, batching). False efficiency is the trap.

Quick review checklist (pre-exam)

- ☐ Ladder order: criteria → inputs → config (incl. memory) → prompt → context → model/effort → gates → limits
 - ☐ One variable at a time. Two failed tightened iterations = change layer
 - ☐ Wrong math → code execution. Wrong facts → knowledge. Wrong format → schema
 - ☐ Stale memory entries are configuration bugs — curate them
 - ☐ Cheapest lever on the worst measured metric
 - ☐ Miss taxonomy → fix the causing system
 - ☐ Never optimize away human gates on high-risk work
 - ☐ Refusal ≠ outage. Clarify within policy
-

Glossary

Term	Meaning
Diagnostic ladder	Ordered layer checks from criteria to limits
Thread drift	Long-conversation degradation. Fix it with summarize + restart.
Miss taxonomy	Factual / format / tone / policy / incomplete / auth classification
Cheapest lever	Smallest change that moves the worst metric
Configuration drift	Stale knowledge, instructions, or memory that diverges from reality
False efficiency	Speed you gain when you delete necessary review
Effort control.	Depth setting inside a model tier — tune before you switch tiers.
Scoreboard	Revisions-to-ship, miss %, usage, time-to-first-draft

Domain 07 — Troubleshooting & Optimization

Maps to official CCAO-F Troubleshooting and Optimization (~10%, ~6 questions)

Dedup note (2026-08-23): Rewritten as a single primary-study copy. Earlier builds repeated the same drill blocks ~7x; duplicates removed and content deepened to the Domain 03 standard.

Disclaimer

Original CCAO-F study notes for non-developers using claude.ai (Projects, Artifacts, Skills, Connectors, Research). Grounded in public Anthropic Help Center & product docs, public Claude Academy (Claude 101, AI Fluency 4D), and the published CCAO-F blueprint. Independent; not affiliated with Anthropic. Verify live product details on support.claude.com.

Overview

Official blueprint verbs: **identify, diagnose, and resolve** issues with underperforming prompts or poor outputs; **adjust** approach based on feedback and results; **optimize** workflows for efficiency and effectiveness. This is the integration domain: decide whether the break is **platform, inputs, configuration, prompt, context, model, evaluation, or governance** — and fix the right layer without wasting usage. Most stems reward the cheapest correct fix, not the most heroic one.

Key map (objectives ↔ exam verbs)

Official objective	Exam verbs	What you practice
Diagnose poor outputs	identify, isolate	The diagnostic ladder; layer naming
Adjust from feedback	adjust, adapt	Miss taxonomy → targeted system updates
Optimize workflows	optimize, streamline	Levers: model, effort, feature, context, structure

Deep notes

1. The diagnostic ladder (run it in order)

1. **Success criteria** — was the ask vague? Maybe nothing is broken but the brief (Domain 02).
2. **Inputs** — wrong/missing files, unauthenticated connector, web search off, OCR-junk scans, capability toggles.
3. **Configuration** — stale or conflicting knowledge; contradictory instructions; wrong Project; stale **Memory entry** feeding wrong context (edit/delete it — see 05—... §Memory).
4. **Prompt** — Description gaps: no criteria, no schema, mixed task types.
5. **Context** — thread drift; summarize and restart.
6. **Model/effort** — under- or over-powered tier; tune effort before jumping tiers.
7. **Evaluation gates** — quality “problems” that are really shipped-without-review problems (Domain 03).
8. **Limits/plan** — usage limits, Free caps, feature availability.

Discipline: change **one variable at a time**; note what fixed it so it becomes a pattern, not a lucky rerun. Two failed tightened prompt iterations = wrong layer; move up or down the ladder.

2. Worked cases

- **“Claude gives the wrong refund answer.”** Which policy file is in the Project? Ask it to quote the section. File stale → update knowledge. File right but ignored → instructions: “Prefer Refund-Policy-v4; if not in knowledge, say Not found.” Only then consider model escalation.
- **“Research is useless.”** Web search on? Paid plan? Question scoped? Connectors authed if internal data expected? Narrow the scope before blaming the feature.
- **“The numbers in the summary are wrong.”** Large-table arithmetic in plain chat is the wrong tool — rerun through **code execution** so the math is computed, not pattern-matched; then compare against source.
- **“Claude keeps bringing up my old project.”** Stale memory entry — Settings → Memory, edit/delete; use separate Projects to scope context; incognito for one-offs.
- **“The team says Claude is random.”** Shared Project without owner; conflicting guides; no schema. Governance + configuration fix — not a bigger model.
- **“Claude refused my legitimate request.”** State the benign, authorized context plainly and rephrase within policy; refusals on disallowed content are correct behavior, and jailbreak options are always exam-wrong (Domain 06).

3. Optimization levers (effectiveness × efficiency)

Model tier; effort; feature choice (Research vs chat vs code execution); context size; Project-persisted context vs re-pasting; Skill reuse; schemas to cut revision cycles; batching similar tasks; pruning noisy knowledge; capability toggles; connector scope. Optimize **both** quality and cost/time: cutting review on L4 work is false efficiency; running Opus on ticket triage is false quality.

Scoreboard: median revisions-to-ship; % factual misses; usage per workflow; time-to-first-draft. Improve the worst metric with the **cheapest lever** — that phrase decides exam options.

4. Feedback → system updates (miss taxonomy)

Tag every miss, then fix the system that caused it: **factual** → knowledge/citations/code execution for math; **format** → instructions/schema; **tone** → examples in Project/Skill; **policy** → governance + instructions; **incomplete** → checklist/decomposition; **auth/access** → IT enablement; **wrong remembered context** → memory curation. Closing the loop is Diligence; the anti-pattern is coaching users to compensate for a broken configuration.

5. Bridge summary across all domains

01 pick tools/models; 02 describe tasks; 03 validate outputs; 04 design workflows; 05 configure knowledge/memory; 06 govern risk; 07 diagnose and optimize. Real failures are usually multi-domain — name the **primary** layer in your answer, fix it, and re-measure.

6. 90-second diagnosis drills

Work each in ~90 seconds: name the layer, then the cheapest fix. Answers follow.

- **Drill A:** “Claude used to nail our weekly report; since Marta added her files it mixes two template styles.” → Configuration: conflicting templates in knowledge; remove one or set priority in instructions.
- **Drill B:** “Research mode grays out for one analyst.” → Platform/plan: their seat/plan or capability toggle — not a prompt problem.

- **Drill C:** “The extraction is perfect on 10 rows, garbage on 5,000.” → Tool/context: move from chat-window pasting to uploaded file + code execution.
- **Drill D:** “Claude answered from the March price list; April’s is uploaded too.” → Knowledge hygiene: superseded file still present; purge it.
- **Drill E:** “Every chat, Claude assumes I’m still on the Berlin project.” → Memory: stale entry; edit/delete in Settings → Memory.
- **Drill F:** “Long thread: it now ignores the banned-words list from message 3.” → Context drift: summarize + restart with rules persisted in the Project.
- **Drill G:** “Quality is fine but we burn the day’s limit by 2 p.m.” → Optimization: tier/effort down on routine tasks; batch; persist context instead of re-pasting.
- **Drill H:** “It refuses to summarize our pen-test report.” → Governance-aware rephrase: state the authorized defensive context; never jailbreak.

The meta-skill: symptom → layer → cheapest fix, said in one sentence. If your explanation needs a paragraph, you probably haven’t found the layer.

7. Multiple-response pattern bank

Recurring wrong pairs: “switch to the top model” + “give up on the task” (prestige + defeatism — the ladder is the middle); “add three more paragraphs of instructions” + “delete all instructions” (wordcount as medicine); “remove the review step to speed up” + “review every trivial output” (false efficiency + disproportion); “restart every chat hourly” + “never restart” (ritual + sunk cost). Correct combinations pair a **diagnostic** move (check auth/toggles, quote-the-file test, isolate one variable) with a **structural** fix (schema, knowledge purge, memory edit, tier right-sizing).

Decision trees

Symptom	Likely layer → action
Wrong facts “from” files	Knowledge/pointing → name file, purge superseded versions
Ignores required format	Prompt → schema/Performance constraints
Great early, degrades late in thread	Context → summarize + restart
Wrong arithmetic on data	Tool → code execution, verify vs source
Slow/expensive workflow	Optimization → lighter model/effort, less Research, templates
Feature “doesn’t work”	Inputs/platform → plan, capability toggle, auth, web search
Wrong personal context recalled	Memory → edit/delete entry; Project scoping
Refusal on allowed work	Governance-aware rephrase; never jailbreak
Chronic misses across users	Workflow/config redesign — not a bigger model
Errors reaching stakeholders	Evaluation gates missing → restore Domain 03 SOP

Exam traps

1. “Escalate to the biggest model” as the universal fix
 2. Ignoring auth, web search, and capability toggles before deep diagnosis
 3. Fixing symptoms with ever-longer prompts (wrong layer)
 4. “Optimizing” by removing human review from high-risk steps
 5. Treating safety refusals as outages
 6. Never restarting drifted threads (sunk-cost threading)
 7. Hoarding conflicting knowledge versions
 8. Changing five variables at once, learning nothing
 9. Blaming the model for a stale memory entry or stale PDF
 10. Optimizing an unmeasured workflow
-

Practice Q&A (16) — scenario stems with answers and rationales

Q1. Claude answers refund questions from general knowledge despite an uploaded policy. First two moves? **A:** Name the file in the prompt and require quoting; check knowledge for superseded/duplicate versions. Pointing + hygiene beat re-asking louder.

Q2. Select TWO optimizations for a high-volume extraction workflow burning limits. **A:** Drop to Haiku (or lower effort); persist the schema and rules in a Project instead of re-pasting. (Removing review is the trap option.)

Q3. Research returns nothing. Fastest diagnostic? **A:** Check web search is enabled and the plan supports Research — capability toggles before feature blame.

Q4. A 200-message thread now contradicts itself. Fix? **A:** Handoff summary (decisions, constraints, open items) → fresh chat. Drifted context is replaced, not argued with.

Q5. Monthly variance memo has subtle arithmetic errors. Layer and fix? **A:** Tool layer: route the math through code execution on the uploaded data, then verify totals against source. Chat prose is not a calculator.

Q6. Outputs ignore the required five-section format intermittently. Fix? **A:** Put a hard schema in Project instructions (sections, order, “omit nothing; write N/A if empty”). Format misses are Performance-constraint gaps, not model failures.

Q7. Drive connector “sees nothing” for one user but works for others. Diagnosis? **A:** That user hasn’t authenticated the connector — admin enablement is org-wide, auth is per-user.

Q8. Claude keeps referencing a client the associate stopped serving last year. Fix? **A:** Memory curation: delete/edit the stale entry in Settings → Memory; keep current clients in their own Projects so scoping does the work.

Q9. Chronic tone misses across the whole team’s outputs. Cheapest durable fix? **A:** Add gold tone examples (and a counterexample) to the shared Project/Skill — one config change fixes every user, unlike per-chat coaching.

Q10. Select TWO checks *before* concluding “the model is too weak” on an analysis task. **A:** Was a rubric/criteria provided (prompt layer)? Is the knowledge current and conflict-free (config layer)? Escalate tier only after both pass.

Q11. The team hits usage limits mid-afternoon. Select TWO adjustments. **A:** Route routine tasks to lighter models/lower effort; cut Research use where a quoted file or single search suffices. (Also batching similar tasks; upgrading is a business call, not a reflex.)

Q12. A stakeholder demands “faster outputs — just buy the top model.” Better lever set? **A:** Templates/Skills for recurring structure, persisted Project context, right-sized model per task — speed usually lives in workflow structure, not tier.

Q13. Claude refuses to analyze the company’s own phishing simulation results. Move? **A:** Restate the authorized, defensive context plainly and rephrase within policy — legitimate security work with consent is fine to clarify; jailbreak framing is auto-wrong.

Q14. Two conflicting SOP versions live in knowledge; answers alternate between them. Fix? **A:** Delete the superseded version (or add explicit priority in instructions). Retrieval confusion is configuration debt.

Q15. How do you prove an optimization worked? **A:** Before/after on the scoreboard: revisions-to-ship, factual-miss %, usage per workflow, time-to-first-draft. Unmeasured optimization is anecdote.

Q16. After an incident, someone proposes removing the human review step to “streamline.” Verdict? **A:** Reject: review on consequential outputs is effectiveness, not overhead — restore the gate and optimize elsewhere (lighter model, schema, batching). False efficiency is the trap.

Quick review checklist (pre-exam)

- ☐ Ladder order: criteria → inputs → config (incl. memory) → prompt → context → model/effort → gates → limits
 - ☐ One variable at a time; two failed tightened iterations = change layer
 - ☐ Wrong math → code execution; wrong facts → knowledge; wrong format → schema
 - ☐ Stale memory entries are configuration bugs — curate them
 - ☐ Cheapest lever on the worst measured metric
 - ☐ Miss taxonomy → fix the causing system
 - ☐ Never optimize away human gates on high-risk work
 - ☐ Refusal ≠ outage; clarify within policy
-

Glossary

Term	Meaning
Diagnostic ladder	Ordered layer checks from criteria to limits
Thread drift	Long-conversation degradation; fixed by summarize + restart
Miss taxonomy	Factual / format / tone / policy / incomplete / auth classification
Cheapest lever	Smallest change that moves the worst metric
Configuration drift	Stale knowledge, instructions, or memory diverging from reality
False efficiency	Speed gained by deleting necessary review

Term	Meaning
Effort control	Depth dial inside a model tier — tune before switching tiers
Scoreboard	Revisions-to-ship, miss %, usage, time-to-first-draft

CCAO-F Pro Tips — how to take this exam well

Companion to domain files 01–07; distilled 2026-08-23 from the official Exam Guide v1.0 and this pack. Exam craft only — content lives in the domain files.

What this exam actually rewards: *judgment as a professional user*, not technical knowledge. The heaviest domain by far is Output Evaluation (21%), and the recurring question underneath most items is: **“what should this person do before trusting or shipping Claude’s output?”** When in doubt, the answer that adds a verification step, a human gate, or a configuration fix beats the answer that adds a bigger model or a longer prompt.

The numbers that drive strategy

- **60 items / 120 min → 2 min each.** These stems are shorter than the developer exams’ — the risk isn’t time, it’s overthinking. First defensible read wins; flag genuine coin-flips and move on.
- **720 scaled, no guessing penalty** — answer everything.
- **Triage by weight:** Evaluation 21% + Workflow Design 16% + Governance 15% = 52%. These are all judgment domains — they share one instinct (verify, gate, own), so drilling one strengthens all three.
- **Multiple-response is all-or-nothing;** each item states how many to select.

The four master instincts (most items reduce to one of these)

1. **Verify before it leaves the building.** Citation → open it. Number → recompute or spot-check. Compliance/external send → human review first. (The guide’s own sample: verify the citation before the compliance send.)
2. **Right-size the model.** High-volume, simple, repetitive → lighter/cheaper tier. Deep analysis, high-stakes → stronger tier. “Always use the biggest” is always wrong (the pack even plants Fable-maximalism as a trap).
3. **Fix the configuration before blaming the model.** Weak output in a shared Project → check instructions, stale knowledge files, missing owner, wrong placement — *then* consider model/prompt changes (Domain 07’s whole logic).
4. **Protect the data.** Regulated/PII content → anonymize before upload, use incognito for sensitive one-offs, respect plan-level controls. (Sample question: anonymize regulated PII first.)

If the stem says X → think Y

Stem signal	Reflex
“Should they send/publish/submit it?”	Not yet — name the verification step first

Stem signal	Reflex
Claude cited a source / gave a statistic	Check the source exists and says that; citations can be wrong
Same task, hundreds of times, simple	Lighter model + a reusable prompt/Project, not Opus-by-default
Team keeps getting inconsistent outputs from a shared Project	Knowledge hygiene: duplicate/stale files, no owner, no version note (file 05's mystery-brand-voice case)
Where should this knowledge live?	Pair a placement (instructions vs knowledge vs Skill vs connector vs Memory) with a lifecycle (owner, cadence, archive)
"Claude remembers wrong" / personal context issue	Edit/delete the Memory entry; Memory is per-user — it never aligns a team (Project knowledge does)
Sensitive matter, shouldn't persist	Incognito chat (excluded from history/memory/search)
Data analysis, file outputs, charts	Code execution / file creation — not "ask Claude to imagine a spreadsheet"
Live document that keeps changing	Connector, not weekly re-upload; stable policy docs → upload is fine
"High-risk use" (legal, medical, employment decisions)	Human accountability stays; Claude drafts/supports, a person decides

Distractor archetypes to expect

- **The trust answer:** ship it because it "looks right" or "Claude is usually accurate." Never the answer in a 21%-weight evaluation domain.
- **The bigger-model answer** for a workflow/config problem.
- **The permission overshare:** "grant edit to everyone so nobody's blocked" / org-wide sharing of restricted knowledge. Least privilege wins here too.
- **The memory confusion:** using personal Memory where shared Project knowledge is needed, or vice versa.
- **The plan-fact bluff:** options hinging on invented plan limits. The real anchors you know: Free = 5 Projects, sharing = Team/Enterprise, paid RAG expands knowledge capacity — anything more exotic, judge by principle.

Freshness watchlist (re-verify the week of your exam)

Plan caps and feature availability move fastest here: Projects caps, Memory availability/controls, Research/connector plan gating — 10 minutes on support.claude.com. The pack's verified facts are dated Aug 2026.

The 48-hour plan

1. Re-read file 03 in full (21% + it trains the master instinct), then files 04 and 06.
2. Re-do every file's traps table + the multiple-response pattern banks.
3. Live practice beats re-reading here: evaluate three real Claude outputs with a written ship/re-vise/reject note; build one Project with instructions

- knowledge + a sharing decision.
4. Timed 60Q simulation; review only what you missed.

Day-of (online proctored)

ID, clean desk, room scan, stable connection. Two-minute rhythm: read the question sentence, find who's at risk / what's about to leave the building, pick the verify/gate/own answer. On review passes, change answers only for misread facts — your calibrated instinct is the product of the drills.